

# Parcial de Bases de Datos II

## Unidad 1: Álgebra Relacional

### ✿ 1. Lenguajes de acceso en Bases de Datos Relacionales (BDR)

En una **base de datos relacional**, los datos se guardan en **tablas (relaciones)**.

Para consultar o manipular esos datos, usamos **lenguajes de acceso**.

Estos lenguajes se dividen en dos grandes grupos:

#### ◆ Álgebra Relacional

- Es un **lenguaje procedimental**.

👉 Esto significa que **indica qué se quiere obtener y cómo hacerlo paso a paso**.

■ Ejemplo:

“Quiero los empleados del departamento 10 y después mostrar solo sus nombres”.

En álgebra relacional eso sería:

$\pi$  nombre ( $\sigma$  id\_dep = 10 (Empleado))

- En SQL se expresa igual pero con pasos explícitos:

SELECT nombre

FROM Empleado

WHERE id\_dep = 10;

→ Primero filtra ( $\sigma$  selección), luego proyecta columnas ( $\pi$ ).

💡 “**Procedimental**” = el usuario **da instrucciones detalladas** (como una receta).

#### ◆ Cálculo Relacional

- Es un **lenguaje no procedimental**.

👉 Indica **qué** se quiere obtener, **sin decir cómo** hacerlo.

- Existen **dos variantes**:

1. **Orientado a Tuplas** → trabaja con registros completos (filas).

Ejemplo: “*Devuélveme todas las tuplas donde el salario > 2000*”.

2. **Orientado a Dominios** → trabaja con valores individuales (columnas).

■ El **SQL** moderno combina ambos conceptos, pero su base teórica viene de álgebra relacional.

### ✿ Equivalencia

“Álgebra y Cálculo Relacional son equivalentes en poder expresivo.”

Esto quiere decir que **cualquier consulta que puedas hacer con álgebra relacional también puede hacerse con cálculo relacional**, solo cambia la forma de escribirla.

#### ◆ Lenguajes de Usuario

- **SQL (Structured Query Language)** → basado en álgebra relacional (procedimental).
- **QBE (Query By Example)** → basado en cálculo relacional (no procedimental, usado en interfaces gráficas como Access).

## ■ Introducción

### ■ *Lenguajes de acceso en BDR*

#### ■ *Álgebra Relacional*

- *Lenguaje procedimental (se indica qué y cómo obtenerlo)*

#### ■ *Cálculo Relacional*

- *Lenguaje no procedimental (se indica qué pero no cómo obtenerlo)*

#### ■ *Dos tipos*

- *Orientado a Tuplas*

- *Orientado a Dominios*

### ■ *Álgebra y Cálculo Relacional son equivalentes en poder expresivo*

### ■ *Lenguajes de Usuario*

- *SQL (Structured Query Language), basado en álgebra relacional*

- *QBE (Query By Example), basado en cálculo relacional*



## 2. Definición del Álgebra Relacional

El álgebra relacional es un **conjunto cerrado de operaciones** que trabajan sobre relaciones (tablas) y **devuelven otras relaciones**.

👉 “Cerrado” significa que si aplicás una operación a una tabla, el resultado **también es una tabla**, por lo tanto podés **encadenar operaciones** (como funciones matemáticas).



### Propiedades principales:

- Actúan sobre **relaciones** (tablas).
- Producen **relaciones** como resultado.
- Podés **combinarlas** para crear consultas más complejas.



### 3. Operadores Básicos

Son las **operaciones fundamentales** del álgebra relacional, a partir de las cuales se pueden expresar todas las demás.

| Operador                               | Descripción                       | Ejemplo                          | SQL equivalente                         |
|----------------------------------------|-----------------------------------|----------------------------------|-----------------------------------------|
| <b>Selección (<math>\sigma</math>)</b> | Filtra filas según una condición. | $\sigma$ edad > 30<br>(Empleado) | SELECT * FROM Empleado WHERE edad > 30; |

|                                                  |                                                         |                                 |                                           |
|--------------------------------------------------|---------------------------------------------------------|---------------------------------|-------------------------------------------|
| <b>Proyección (<math>\pi</math>)</b>             | Selecciona columnas específicas.                        | $\pi$ nombre, sueldo (Empleado) | SELECT nombre, sueldo FROM Empleado;      |
| <b>Unión (U)</b>                                 | Combina tuplas de dos relaciones compatibles.           | Empleado1 U Empleado2           | SELECT * FROM E1 UNION SELECT * FROM E2;  |
| <b>Diferencia (-)</b>                            | Resta filas de una tabla respecto a otra.               | E1 - E2                         | SELECT * FROM E1 EXCEPT SELECT * FROM E2; |
| <b>Producto cartesiano (<math>\times</math>)</b> | Combina todas las tuplas posibles entre dos relaciones. | E $\times$ D                    | SELECT * FROM Empleado, Depto;            |

● Estos cinco operadores son **operacionalmente completos**, es decir, **con ellos podés construir cualquier consulta posible**.

#### ✿ 4. Operadores Derivados

Se **forman combinando los operadores básicos**.

No agregan “nuevas capacidades”, pero son **más prácticos** para ciertos tipos de consultas.

| Operador                                | Se forma a partir de               | Sirve para...                        | Ejemplo SQL                                |
|-----------------------------------------|------------------------------------|--------------------------------------|--------------------------------------------|
| <b>Intersección (<math>\cap</math>)</b> | Unión + Diferencia                 | Obtener elementos comunes            | SELECT * FROM A INTERSECT SELECT * FROM B; |
| <b>Join (<math>\bowtie</math>)</b>      | Producto cartesiano + Selección    | Combinar tablas por una condición    | SELECT * FROM A JOIN B ON A.id = B.id;     |
| <b>División (<math>\div</math>)</b>     | Proyección + Diferencia + Producto | “Quién tiene todos los...”           | SELECT alumno FROM Cursa $\div$ Materias;  |
| <b>Asociación</b>                       | Join + Proyección                  | Consultas jerárquicas o dependientes | JOIN anidados                              |

## ■ Definición

### ■ Conjunto cerrado de operaciones

- Actúan sobre relaciones
- Producen relaciones como resultados
- Pueden combinarse para construir expresiones más complejas

#### ■ Operadores Básicos

- Unión
- Diferencia
- Producto Cartesiano
- Selección
- Proyección

- Son operacionalmente completos, permiten expresar cualquier consulta a una BDR

#### ■ Operadores Derivados

- Intersección
- Join
- División
- Asociación

- No añaden nada nuevo
- Se forman combinando los operadores básicos
- Son útiles en determinadas consultas

## Operadores Basicas

### Union:

#### ■ $R \cup S$

- La unión de dos relaciones  $R$  y  $S$ , es otra relación que contiene las tuplas que están en  $R$ , o en  $S$ , o en ambas, eliminándose las tuplas duplicadas
- $R$  y  $S$  deben ser unión-compatible, es decir, definidas sobre el mismo conjunto de atributos

Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

Jefes

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 421 | Jorge  | 48   |

Ingenieros  $\cup$  Jefes

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |
| 421 | Jorge  | 48   |

#### ◆ 1. Concepto de Unión ( $R \cup S$ )

La **unión** combina los registros (tuplas) de **dos relaciones** (tablas) y **devuelve todas las filas que aparecen en una u otra, sin repetir**.

👉 Es como juntar dos tablas con la misma estructura y eliminar duplicados.

#### ■ Definición formal

La unión de dos relaciones  $R$  y  $S$  es otra relación que contiene todas las tuplas que están en  $R$ , o en  $S$ , o en ambas.

- ◆ Si una tupla aparece en las dos, **solo se muestra una vez** (sin duplicados).
- ◆ Si una tupla aparece solo en una de las dos, también se incluye.

## ⚙️ Requisito importante: Unión-compatibilidad

Para poder unir **R U S**, las dos relaciones deben ser **compatibles**, es decir:

1. Tener **el mismo número de atributos (columnas)**.
2. Que los **atributos correspondan en tipo y significado**.
  - Ejemplo: si la primera columna de R es E# (entero), la primera de S también debe ser un número de empleado.

## 🧩 2. Ejemplo visual

→ Observá que el registro de José (320, 34) aparece solo una vez, aunque estaba en ambas tablas.

## 💻 3. Equivalente en SQL

```
SELECT * FROM Ingenieros
```

```
UNION
```

```
SELECT * FROM Jefes;
```

- ♦ **UNION** elimina duplicados automáticamente.
- ♦ Si querés conservar los duplicados, podés usar **UNION ALL**:

```
SELECT * FROM Ingenieros
```

```
UNION ALL
```

```
SELECT * FROM Jefes;
```

## 🧠 4. Interpretación práctica

La unión sirve cuando querés combinar resultados del mismo tipo, por ejemplo:

- Empleados de distintas áreas.
- Listas de clientes de distintas sucursales.
- Usuarios que aparecen en dos tablas distintas (por ejemplo, “registrados” y “suscriptores”).

## 🎯 Resumen rápido

| Elemento                | Descripción                                |
|-------------------------|--------------------------------------------|
| <b>Símbolo</b>          | R U S                                      |
| <b>Tipo de operador</b> | Básico                                     |
| <b>Qué hace</b>         | Combina las tuplas de R y S sin duplicar   |
| <b>Condición</b>        | R y S deben tener mismos atributos y tipos |
| <b>Ejemplo SQL</b>      | SELECT * FROM R UNION SELECT * FROM S;     |

## Diferencia:

## ■ R - S

- La diferencia de dos relaciones  $R$  y  $S$ , es otra relación que contiene las tuplas que están en la relación  $R$ , pero no están en  $S$
- $R$  y  $S$  deben ser unión-compatible

**Ingenieros**

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

**Jefes**

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 421 | Jorge  | 48   |

**Ingenieros - Jefes**

| E#  | Nombre | Edad |
|-----|--------|------|
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

**Jefes - Ingenieros**

| E#  | Nombre | Edad |
|-----|--------|------|
| 421 | Jorge  | 48   |

### ◆ 1. Concepto de Diferencia ( $R - S$ )

La **diferencia** obtiene los registros (tuplas) que están en una tabla **R**, pero **no están** en la tabla **S**.

👉 Es decir, resta una relación de otra.

Te quedás solo con las filas que existen en **R** y no aparecen en **S**.

### ■ Definición formal

La diferencia de dos relaciones **R** y **S** es una nueva relación que contiene **las tuplas que están en R pero no en S**.

- ◆ Si una tupla está en ambas, **se elimina**.
- ◆ Si está solo en **R**, **se mantiene**.

### ⚙️ Requisito importante: Unión-compatibilidad

Igual que en la unión ( $R \cup S$ ), para poder hacer  $R - S$ , las dos tablas deben ser **compatibles**, es decir:

1. Tener el **mismo número de columnas**.
2. Que los **tipos y nombres de columnas correspondan** (por posición y significado).

### 🧩 2. Ejemplo visual

**Resultado: Ingenieros – Jefes**

| E#  | Nombre | Edad |
|-----|--------|------|
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

→ Se eliminaron los registros que estaban en ambas tablas (José, 320).

**Resultado inverso: Jefes – Ingenieros**

| E#  | Nombre | Edad |
|-----|--------|------|
| 421 | Jorge  | 48   |

→ Jorge solo existe en Jefes, no en Ingenieros.



### 3. Equivalente en SQL

SELECT \* FROM Ingenieros

EXCEPT

SELECT \* FROM Jefes;

- ◆ EXCEPT elimina duplicados automáticamente, igual que la diferencia relacional.
- ◆ Si querés mantenerlos, usás EXCEPT ALL (depende del gestor).

### 4. Interpretación práctica

La **diferencia** sirve para encontrar:

- Empleados que **no** son jefes.
- Clientes que **no** hicieron compras.
- Productos que **no** están en stock.

■ Ejemplo práctico:

SELECT \* FROM Clientes

EXCEPT

SELECT \* FROM ClientesConPedidos;

→ Te devuelve los clientes que **todavía no compraron nada**.

### Resumen rápido

| Elemento                | Descripción                                |
|-------------------------|--------------------------------------------|
| <b>Símbolo</b>          | R – S                                      |
| <b>Tipo de operador</b> | Básico                                     |
| <b>Qué hace</b>         | Devuelve tuplas que están en R y no en S   |
| <b>Condición</b>        | R y S deben tener mismos atributos y tipos |
| <b>Ejemplo SQL</b>      | SELECT * FROM R EXCEPT SELECT * FROM S;    |

## Producto Cartesiano:

### ■ R x S

- *Define una relación que es la concatenación de cada una de las filas de la relación R con cada una de las filas de la relación S*

**Ingenieros**

| E#  | Nombre | D# |
|-----|--------|----|
| 320 | José   | D1 |
| 322 | Rosa   | D3 |

**Departamentos**

| D# | Descrip |
|----|---------|
| D1 | Central |
| D3 | I+D     |

**Proyectos**

| Proyecto | Tiempo |
|----------|--------|
| RX338A   | 21     |
| PY254Z   | 32     |

**Ingenieros X Proyectos**

| E#  | Nombre | D# | Proyecto | Tiempo |
|-----|--------|----|----------|--------|
| 320 | José   | D1 | RX338A   | 21     |
| 320 | José   | D1 | PY254Z   | 32     |
| 322 | Rosa   | D3 | RX338A   | 21     |
| 322 | Rosa   | D3 | PY254Z   | 32     |

**Ingenieros X Departamentos**

| E#  | Nombre | D# | DD | Descrip |
|-----|--------|----|----|---------|
| 320 | José   | D1 | D1 | Central |
| 320 | José   | D1 | D3 | I+D     |
| 322 | Rosa   | D3 | D1 | Central |
| 322 | Rosa   | D3 | D3 | I+D     |

## ◆ 1. Concepto del Producto Cartesiano ( $R \times S$ )

El **producto cartesiano** combina **cada fila (tupla)** de una tabla **R** con **cada fila** de otra tabla **S**.

Es decir, **hace todas las combinaciones posibles** entre los registros de las dos relaciones.

📘 En palabras simples:

👉 Si R tiene 3 filas y S tiene 2 filas, el resultado tendrá  $3 \times 2 = 6$  filas.

### 📖 Definición formal

El producto cartesiano entre R y S devuelve una relación que es la **concatenación de cada fila de R con cada fila de S**.

## 🏗️ 2. Ejemplo visual

**Relación R → Ingenieros**

| E#  | Nombre | D# |
|-----|--------|----|
| 320 | José   | D1 |
| 322 | Rosa   | D3 |

**Relación S → Proyectos**

| Proyecto | Tiempo |
|----------|--------|
| RX338A   | 21     |
| PY254Z   | 32     |

**Resultado: Ingenieros  $\times$  Proyectos**

| E#  | Nombre | D# | Proyecto | Tiempo |
|-----|--------|----|----------|--------|
| 320 | José   | D1 | RX338A   | 21     |
| 320 | José   | D1 | PY254Z   | 32     |
| 322 | Rosa   | D3 | RX338A   | 21     |
| 322 | Rosa   | D3 | PY254Z   | 32     |

👉 Se unió **cada ingeniero con cada proyecto**, generando 4 combinaciones ( $2 \times 2$ ).

**Otro ejemplo: Ingenieros  $\times$  Departamentos**

| D# | Descrip |
|----|---------|
| D1 | Central |
| D3 | I+D     |

**Resultado:**

Cada ingeniero se combina con todos los departamentos:

| E#  | Nombre | D# | DD | Descrip |
|-----|--------|----|----|---------|
| 320 | José   | D1 | D1 | Central |
| 320 | José   | D1 | D3 | I+D     |
| 322 | Rosa   | D3 | D1 | Central |
| 322 | Rosa   | D3 | D3 | I+D     |



### 3. Equivalente en SQL

En SQL, el producto cartesiano se genera con una consulta **sin condición de unión**:

```
SELECT *
```

```
FROM Ingenieros, Proyectos;
```

o también:

```
SELECT *
```

```
FROM Ingenieros CROSS JOIN Proyectos;
```

- ◆ Si no agregás un WHERE, se combinan **todas las filas**.
- ◆ Si querés limitar las combinaciones, agregás una **condición** (por ejemplo, usando un JOIN).

### 4. Interpretación práctica

El producto cartesiano se usa como **base de otras operaciones**, especialmente del **JOIN**.

Por sí solo, rara vez se usa directamente, porque produce combinaciones que **no siempre tienen sentido lógico**.

Ejemplo práctico:

```
SELECT *
```

```
FROM Ingenieros, Departamentos
```

```
WHERE Ingenieros.D# = Departamentos.D#;
```

→ Aquí transformás el producto cartesiano en un **producto con condición** (es decir, un **JOIN**).

### 5. Resumen rápido

| Elemento                | Descripción                                   |
|-------------------------|-----------------------------------------------|
| <b>Símbolo</b>          | $R \times S$                                  |
| <b>Tipo de operador</b> | Básico                                        |
| <b>Qué hace</b>         | Combina cada fila de R con cada fila de S     |
| <b>Resultado</b>        | Relación con $(m \times n)$ filas             |
| <b>Ejemplo SQL</b>      | SELECT * FROM R CROSS JOIN S;                 |
| <b>Uso típico</b>       | Base para el JOIN (con condición de igualdad) |

## Selección:

### ■ $\sigma$ predicado (R)

- *Es un operador unario*
- *Define una relación con los mismos atributos que R y que contiene solo aquellas filas de R que satisfacen la condición especificada (predicado)*

Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | Maria  | 25   |

$\sigma_{\text{edad} \geq 35}$  (Ingenieros)

| E#  | Nombre | Edad |
|-----|--------|------|
| 322 | Rosa   | 37   |

$\sigma_{\text{edad} \geq 45}$  (Ingenieros)

| E# | Nombre | Edad |
|----|--------|------|
|    |        |      |

## ◆ 1. Concepto de Selección ( $\sigma$ )

La **selección** permite **filtrar filas (tuplas)** de una tabla según una **condición lógica**. Solo se conservan las filas que **cumplen el predicado** (la condición especificada).

■ En otras palabras:

👉 De todas las filas de una relación, se seleccionan solo las que **satisfacen un criterio**.

### ⚙ Notación formal

$$\sigma_{\text{condición}}(R)$$

- $\sigma$ : símbolo del operador de selección.
- **condición**: el **predicado lógico** que deben cumplir las filas (por ejemplo, edad  $\geq 35$ ).
- **R**: la relación (tabla) sobre la que se aplica.

### ✿ Ejemplo visual

Relación R  $\rightarrow$  Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

Ejemplo 1:

$$\sigma_{\text{edad} \geq 35}(\text{Ingenieros})$$

| E#  | Nombre | Edad |
|-----|--------|------|
| 322 | Rosa   | 37   |

Solo se selecciona la fila donde la edad es **mayor o igual a 35**.

Ejemplo 2:

$$\sigma_{\text{edad} \geq 45}(\text{Ingenieros})$$

| E# | Nombre | Edad |
|----|--------|------|
|----|--------|------|

No devuelve filas, porque **ningún ingeniero tiene edad  $\geq 45$** .

## 🧠 2. Propiedades importantes

- Es un **operador unario**, porque actúa **sobre una sola relación**.
- El resultado tiene **los mismos atributos (columnas)** que la tabla original, solo cambia el número de filas.
- Se puede aplicar **más de una condición** usando conectores lógicos:
  - AND ( $\wedge$ )  $\rightarrow$  intersección de condiciones.
  - OR ( $\vee$ )  $\rightarrow$  unión de condiciones.
  - NOT ( $\neg$ )  $\rightarrow$  negación.

■ Ejemplo:

$$\sigma_{\{\text{edad} > 25 \wedge D\# = 'D3'\}}(\text{Ingenieros})$$

$\rightarrow$  Selecciona los ingenieros mayores de 25 del departamento D3.

### 3. Equivalente en SQL

SELECT \*

FROM Ingenieros

WHERE edad >= 35;

Otro ejemplo con múltiples condiciones:

SELECT \*

FROM Ingenieros

WHERE edad > 25 AND D# = 'D3';

### 4. Resumen rápido

| Elemento         | Descripción                      |
|------------------|----------------------------------|
| Símbolo          | $\sigma$ (sigma)                 |
| Tipo de operador | Unario                           |
| Qué hace         | Filtra filas según una condición |
| Resultado        | Misma estructura, menos filas    |
| Ejemplo SQL      | SELECT * FROM R WHERE condición; |

### 5. Uso práctico

- Buscar empleados mayores de cierta edad.
- Filtrar productos por precio o categoría.
- Seleccionar registros que cumplen una condición específica (como estado = 'Activo').

## Proyección:

### ■ $\Pi_{col1, \dots, coln}(R)$

- Es un operador unario
- Define una relación que contiene un subconjunto vertical de R con los valores de los atributos especificados, eliminando filas duplicadas en el resultado

Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |
| 324 | José   | 29   |

$\Pi_{Nombre, Edad}(\text{Ingenieros})$

| Nombre | Edad |
|--------|------|
| José   | 34   |
| Rosa   | 37   |
| María  | 25   |
| José   | 29   |

$\Pi_{Nombre}(\text{Ingenieros})$

| Nombre |
|--------|
| José   |
| Rosa   |
| María  |

### ◆ 1. Concepto de Proyección ( $\pi$ )

La **proyección** selecciona **columnas (atributos)** específicas de una tabla.

A diferencia de la **selección** ( $\sigma$ ), que filtra **filas**, la proyección **reduce columnas**.

■ En otras palabras:

👉 Si la selección elige “qué filas quiero”, la proyección elige “qué columnas quiero”.

## ⚙️ Notación formal

$$\pi_{col1, col2, \dots, coln}(R)$$

- $\pi$  ( $\pi$ )  $\rightarrow$  símbolo del operador de proyección.
- **col1, col2, ...**  $\rightarrow$  nombres de los atributos (columnas) que querés conservar.
- **R**  $\rightarrow$  relación (tabla) sobre la que se aplica.

## 🧩 2. Ejemplo visual

Relación R  $\rightarrow$  Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |
| 324 | José   | 29   |

Ejemplo 1:

$$\pi_{Nombre, Edad}(Ingenieros)$$

| Nombre | Edad |
|--------|------|
| José   | 34   |
| Rosa   | 37   |
| María  | 25   |
| José   | 29   |

$\rightarrow$  Solo se mantienen las columnas **Nombre** y **Edad**.

Ejemplo 2:

$$\pi_{Nombre}(Ingenieros)$$

| Nombre |
|--------|
| José   |
| Rosa   |
| María  |

$\rightarrow$  Se eliminan los duplicados (José aparece una sola vez).

El álgebra relacional **no admite filas repetidas** en el resultado.

## 🧠 3. Propiedades importantes

- Es un **operador unario** (se aplica sobre una sola tabla).
- El resultado tiene **menos columnas**, pero **las filas pueden repetirse temporalmente** antes de eliminar duplicados.
- Es un **subconjunto vertical** de la relación original.
- **Proyección + Selección** son los dos operadores más usados para construir consultas SQL.

## 💻 4. Equivalente en SQL

```
SELECT Nombre, Edad
FROM Ingenieros;
```

👉 En SQL, **no elimina duplicados por defecto**.

Si querés que el resultado sea como en álgebra relacional (sin repeticiones):

SELECT DISTINCT Nombre, Edad

FROM Ingenieros;

## 🎯 5. Resumen rápido

| Elemento                | Descripción                                      |
|-------------------------|--------------------------------------------------|
| <b>Símbolo</b>          | $\pi$ (pi)                                       |
| <b>Tipo de operador</b> | Unario                                           |
| <b>Qué hace</b>         | Selecciona columnas específicas                  |
| <b>Resultado</b>        | Misma cantidad de filas o menos (sin duplicados) |
| <b>Ejemplo SQL</b>      | SELECT DISTINCT col1, col2 FROM R;               |

## ✂️ 6. Diferencia con Selección ( $\sigma$ )

| Concepto        | Selección ( $\sigma$ ) | Proyección ( $\pi$ ) |
|-----------------|------------------------|----------------------|
| Actúa sobre     | Filas                  | Columnas             |
| Devuelve        | Menos filas            | Menos columnas       |
| Criterio        | Condición lógica       | Lista de atributos   |
| SQL equivalente | WHERE                  | SELECT col1, col2    |

## Operadores Derivados:

### Intersección:

#### ■ $R \cap S$

- Define una relación que contiene el conjunto de todas las filas que están tanto en la relación  $R$  como en  $S$
- $R$  y  $S$  deben ser unión-compatible
- Equivalencia con operadores básicos  

$$R \cap S = R - (R - S)$$

Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

Jefes

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 421 | Jorge  | 48   |

Ingenieros  $\cap$  Jefes

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |

#### ◆ 1. Concepto de Intersección ( $R \cap S$ )

La **intersección** devuelve las filas (tuplas) que están **presentes en ambas relaciones**, es decir, en **R y en S** al mismo tiempo.

■ En otras palabras:

👉 “Lo que R y S tienen en común”.

## ⚙️ Notación formal

$$R \cap S$$

- **R y S:** relaciones (tablas).
- El resultado es una nueva relación con **las filas que están tanto en R como en S**.

## ✖️ Ejemplo visual

### Relación R → Ingenieros

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 322 | Rosa   | 37   |
| 323 | María  | 25   |

### Relación S → Jefes

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |
| 421 | Jorge  | 48   |

### Resultado: Ingenieros $\cap$ Jefes

| E#  | Nombre | Edad |
|-----|--------|------|
| 320 | José   | 34   |

→ Solo aparece la fila **común a ambas relaciones** (José, 34).

## 🧠 2. Propiedades importantes

- Es un **operador binario**, ya que actúa sobre **dos relaciones**.
- **R y S deben ser unión-compatibles**, es decir:
  - Tener el **mismo número de columnas**.
  - Que los atributos correspondan en tipo y significado.
- El resultado siempre es un **subconjunto de R (y también de S)**.

### ♦ Relación con otros operadores

La intersección puede expresarse usando solo operadores básicos:

$$R \cap S = R - (R - S)$$

→ Es decir:

Primero se eliminan de R las filas que **no están** en S, y el resultado es la **intersección**.

## 💻 3. Equivalente en SQL

```
SELECT * FROM Ingenieros
INTERSECT
SELECT * FROM Jefes;
```

- ♦ INTERSECT elimina duplicados automáticamente, igual que el álgebra relacional.
- ♦ Si el motor lo permite, INTERSECT ALL conserva duplicados.

## 🧠 4. Interpretación práctica



La **intersección** sirve para encontrar coincidencias entre dos conjuntos de datos, por ejemplo:

- Clientes que **también son proveedores**.
- Empleados que **figuran en dos departamentos**.
- Estudiantes que **cursan materias en común**.

■ Ejemplo:

SELECT \* FROM Clientes

INTERSECT

SELECT \* FROM Proveedores;

→ Devuelve las personas que están registradas en ambos roles.

## 🔗 5. Resumen rápido

| Elemento         | Descripción                                      |
|------------------|--------------------------------------------------|
| Símbolo          | $R \cap S$                                       |
| Tipo de operador | Derivado                                         |
| Qué hace         | Devuelve las filas que están en ambas relaciones |
| Condición        | R y S deben ser unión-compatibles                |
| Equivalencia     | $R \cap S = R - (R - S)$                         |
| Ejemplo SQL      | SELECT * FROM R INTERSECT SELECT * FROM S;       |

## Division o Cociente:

### ■ $R \div S$

- Define una relación sobre el conjunto de atributos C, incluido en la relación R, y que contiene el conjunto de valores de C, que en las tuplas de R están combinadas con cada una de las tuplas de S
- Condiciones
  - $\text{grado}(R) > \text{grado}(S)$
  - $\text{conjunto atributos de } S \subset \text{conjunto de atributos de } R$
- Equivalencia con operadores básicos
 
$$X_1 = \Pi_C(R); \quad X_2 = \Pi_C((X_1 \times S) - R); \quad X = X_1 - X_2$$

R1

| E#  | Proyecto |
|-----|----------|
| 320 | RX338A   |
| 320 | PY254Z   |
| 323 | RX338A   |
| 323 | NC168T   |
| 323 | PY254Z   |
| 324 | PY254Z   |
| 324 | NC168T   |

R2

| Proyecto |
|----------|
| RX338A   |
| PY254Z   |

R1 ÷ R2

| E#  |
|-----|
| 320 |
| 323 |

### ◆ 1. Concepto de División ( $R \div S$ )

La **división** se usa cuando queremos encontrar los elementos de una relación (R) que están relacionados con **todos** los elementos de otra (S).

■ En palabras simples:

👉 “¿Qué registros de R están asociados con **todos los valores** de S?”



## ⚙️ Notación formal

$$R \div S$$

- **R**: relación principal (tiene más columnas).
- **S**: subconjunto (tiene menos columnas).
- El resultado incluye **solo los valores de R** que están combinados con **todas las tuplas de S**.

## 🏠 2. Ejemplo visual

Relación R1 → *Ingenieros y sus proyectos*

| E#  | Proyecto |
|-----|----------|
| 320 | RX338A   |
| 320 | PY254Z   |
| 323 | RX338A   |
| 323 | PY254Z   |
| 323 | NC168T   |
| 324 | NC168T   |

Relación R2 → *Proyectos requeridos*

| Proyecto |
|----------|
| RX338A   |
| PY254Z   |

Resultado: R1 ÷ R2

| E#  |
|-----|
|     |
| 320 |
| 323 |

→ Los empleados **320 y 323** han trabajado **en todos los proyectos listados en R2 (RX338A y PY254Z)**.

El ingeniero 324 queda fuera porque **no tiene ambos proyectos**.

## 🧠 3. Condiciones necesarias

Para que se pueda aplicar la división:

1. **grado(R) > grado(S)** →  
R debe tener **más atributos** que S.  
(Ejemplo: R tiene *E#*, *Proyecto* y S solo *Proyecto*).
2. **Los atributos de S** deben estar **incluidos** en los de R.  
Es decir, S debe tener columnas que existan dentro de R.

## 🌱 4. Interpretación práctica

El operador **División (÷)** se usa cuando querés responder preguntas del tipo:

- “¿Qué empleado participó en **todos** los proyectos requeridos?”
- “¿Qué alumno aprobó **todas** las materias obligatorias?”

- “¿Qué cliente compró **todos** los productos de una lista?”

 Ejemplo:

-- “¿Qué alumno cursó todas las materias obligatorias?”

R: (Alumno, Materia)

S: (Materia obligatoria)

Resultado = Alumnos que están en todas las materias de S

## 5. Equivalente en SQL

SQL no tiene un operador DIVIDE,

pero se puede simular con **subconsultas** y agrupamientos:

SELECT E#

FROM R1

WHERE Proyecto IN (SELECT Proyecto FROM R2)

GROUP BY E#

HAVING COUNT(DISTINCT Proyecto) = (SELECT COUNT(\*) FROM R2);

→ Cuenta cuántos proyectos de R2 tiene cada empleado,  
y devuelve solo los que los tienen **todos**.

## ♦ 6. Equivalencia con operadores básicos

$$X_1 = \pi_c(R)$$

$$X_2 = \pi_c((X_1 \times S) - R)$$

$$X = X_1 - X_2$$

 En resumen:

1. Se toman los valores de las columnas de R sin S.
2. Se generan todas las combinaciones posibles con S.
3. Se eliminan las combinaciones que no existen en R.
4. Lo que queda son las tuplas que cumplen **todas las combinaciones** → el resultado de la división.

## 7. Resumen rápido

| Elemento                | Descripción                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------|
| <b>Símbolo</b>          | $R \div S$                                                                                |
| <b>Tipo de operador</b> | Derivado                                                                                  |
| <b>Qué hace</b>         | Devuelve los valores de R que están combinados con <b>todas</b> las tuplas de S           |
| <b>Condiciones</b>      | $\text{grado}(R) > \text{grado}(S)$ ; $\text{atributos}(S) \subseteq \text{atributos}(R)$ |
| <b>Ejemplo SQL</b>      | GROUP BY + HAVING COUNT(DISTINCT ...)                                                     |
| <b>Ejemplo práctico</b> | Empleados que trabajaron en todos los proyectos                                           |

## Unión Natural – Natural Join:

### ■ $R \bowtie S$ ó $R * S$

- El resultado es una relación con los atributos de ambas relaciones y se obtiene combinando las tuplas de ambas relaciones que tengan el mismo valor en los atributos comunes
- Normalmente la operación de join se realiza entre los atributos comunes de dos tablas que corresponden a la clave primaria de una tabla y la clave foránea correspondiente de la otra tabla
- Método
  - Se realiza el producto cartesiano  $R \times S$
  - Se seleccionan aquellas filas del producto cartesiano para las que los atributos comunes tengan el mismo valor
  - Se elimina del resultado una ocurrencia (columna) de cada uno de los atributos comunes
- Equivalencia con operadores básicos

$$R \bowtie_F S = \sigma_F(R \times S)$$

#### ◆ 1. Concepto de Unión Natural ( $R \bowtie S$ )

La **Unión Natural (Join Natural)** combina dos relaciones (tablas) uniendo las filas que **tienen el mismo valor en los atributos comunes**.

■ En palabras simples:

👉 “Uní las dos tablas donde los valores coinciden en una o más columnas con nombre igual.”

#### ⚙️ Notación formal

$$R \bowtie S \text{ o también } R * S$$

El resultado es una nueva relación que contiene:

- **Todos los atributos** de ambas tablas.
- **Solo las filas** donde los valores coinciden en los **atributos comunes**.

#### 🧱 2. Método de funcionamiento

El **Join Natural** se puede entender como una **composición de operaciones básicas**:

$$R \bowtie S = \sigma_F(R \times S)$$

Donde:

- $R \times S \rightarrow$  es el **producto cartesiano** (todas las combinaciones posibles).
- $\sigma_F \rightarrow$  aplica una **selección** filtrando solo las filas que cumplen la condición de igualdad entre atributos comunes.
- Luego se **elimina una de las columnas repetidas**.

■ En resumen:

1. Se hace el **producto cartesiano** ( $R \times S$ ).
2. Se **filtran** las filas donde los valores coinciden.
3. Se **elimina una columna repetida** para evitar duplicación.

### ✿ 3. Ejemplo visual

Tabla R → *Ingenieros*

| E#  | Nombre | D# |
|-----|--------|----|
| 320 | José   | D1 |
| 322 | Rosa   | D3 |

Tabla S → *Departamentos*

| D# | Descrip |
|----|---------|
| D1 | Central |
| D3 | I+D     |

Resultado: Ingenieros ⋈ Departamentos

| E#  | Nombre | D# | Descrip |
|-----|--------|----|---------|
| 320 | José   | D1 | Central |
| 322 | Rosa   | D3 | I+D     |

✓ Se unieron las dos tablas por el atributo común D#.

### 💻 4. Equivalente en SQL

En SQL, el **Join Natural** se puede escribir de varias formas según el tipo de unión:

#### ♦ a) JOIN con condición explícita

```
SELECT *
FROM Ingenieros
JOIN Departamentos
ON Ingenieros.D# = Departamentos.D#;
```

#### ♦ b) NATURAL JOIN (automático por nombre de columna)

```
SELECT *
FROM Ingenieros
NATURAL JOIN Departamentos;
```

→ El NATURAL JOIN une automáticamente por las columnas que tienen el **mismo nombre** en ambas tablas.

### 🧠 5. Tipos relacionados de Join

- **INNER JOIN** → solo las filas que coinciden en ambas tablas.
- **LEFT JOIN** → todas las filas de la izquierda, aunque no tengan coincidencia.
- **RIGHT JOIN** → todas las filas de la derecha.
- **FULL JOIN** → todas las filas de ambas tablas, incluso sin coincidencias.

📘 Pero el **Natural Join** siempre se comporta como un **INNER JOIN**, y por igualdad de nombres de columnas.

### 🎯 6. Resumen rápido

| Elemento         | Descripción             |
|------------------|-------------------------|
| Símbolo          | $R \bowtie S$ o $R * S$ |
| Tipo de operador | Derivado                |

|                    |                                                              |
|--------------------|--------------------------------------------------------------|
| <b>Qué hace</b>    | Combina filas de R y S donde los atributos comunes coinciden |
| <b>Basado en</b>   | Producto cartesiano + Selección                              |
| <b>Elimina</b>     | Columnas duplicadas                                          |
| <b>Ejemplo SQL</b> | SELECT * FROM R JOIN S ON R.id = S.id; o NATURAL JOIN        |

## ✖ 7. Aplicación práctica

Se usa para:

- Relacionar tablas por claves primarias y foráneas.
- Unir datos distribuidos en distintas tablas (por ejemplo, empleados y departamentos).
- Crear vistas combinadas con información coherente.

## Outer JOIN:

- *Es una variante del Join en la que se intenta mantener toda la información de los operandos, incluso para aquellas filas que no participan en el Join*
- *Se “rellenan con nulos” las tuplas que no tienen correspondencia en el Join*
- *Tres variantes*
  - *Left*
    - *se tienen en cuenta todas las filas del primer operando*
  - *Right*
    - *se tienen en cuenta todas las filas del segundo operando*
  - *Full*
    - *se tienen en cuenta todas las filas de ambos operandos*

### ◆ 1. Concepto de Outer Join

El **Outer Join** es una variante del Join que **mantiene toda la información** de una o ambas tablas, **aunque no haya coincidencia** entre ellas.

■ En palabras simples:

👉 “Muestra también las filas que no coinciden en la otra tabla, rellenando con valores nulos.”

## 🧠 2. Cómo funciona

En un **Join normal (INNER JOIN)**, solo se muestran las filas **que coinciden** entre las dos tablas.

En cambio, el **Outer Join** agrega las filas que **no tienen pareja**, rellenando los datos faltantes con NULL.

## ✖ 3. Tipos de Outer Join

### ♦ a) LEFT OUTER JOIN

Muestra **todas las filas de la tabla izquierda (R)**, y los datos coincidentes de la tabla derecha (S).

Si no hay coincidencia, los valores de S se completan con NULL.

📘 Ejemplo:

#### Empleados (R)

| ID | Nombre | D# |
|----|--------|----|
| 1  | José   | D1 |
| 2  | Rosa   | D3 |
| 3  | María  | D2 |

#### Departamentos (S)

##### D# NombreDepto

D1 Central

D3 I+D

#### LEFT JOIN →

SELECT \*

FROM Empleados

LEFT JOIN Departamentos

ON Empleados.D# = Departamentos.D#;

#### Resultado:

| ID | Nombre | D# | NombreDepto |
|----|--------|----|-------------|
| 1  | José   | D1 | Central     |
| 2  | Rosa   | D3 | I+D         |
| 3  | María  | D2 | NULL        |

✅ María no tiene departamento asignado → se rellena con NULL.

### ♦ b) RIGHT OUTER JOIN

Muestra **todas las filas de la tabla derecha (S)**, y los datos coincidentes de la izquierda (R).

Si no hay coincidencia, se completa con NULL del lado de R.

📘 Ejemplo:

SELECT \*

FROM Empleados

RIGHT JOIN Departamentos

ON Empleados.D# = Departamentos.D#;

**Resultado:**

| ID   | Nombre | D# | NombreDepto |
|------|--------|----|-------------|
| 1    | José   | D1 | Central     |
| 2    | Rosa   | D3 | I+D         |
| NULL | NULL   | D4 | Ventas      |

✓ Si un departamento no tiene empleados, igual aparece con NULL.

#### ♦ c) FULL OUTER JOIN

Devuelve **todas las filas de ambas tablas**, coincidan o no.

Las no coincidentes se rellenan con NULL del lado que falta.

■ Ejemplo:

SELECT \*

FROM Empleados

FULL OUTER JOIN Departamentos

ON Empleados.D# = Departamentos.D#;

**Resultado:**

| ID   | Nombre | D# | NombreDepto |
|------|--------|----|-------------|
| 1    | José   | D1 | Central     |
| 2    | Rosa   | D3 | I+D         |
| 3    | María  | D2 | NULL        |
| NULL | NULL   | D4 | Ventas      |

✓ Incluye todos los empleados y todos los departamentos, aunque no haya coincidencia.

#### ■ 4. Equivalencia en álgebra relacional

El **Outer Join** no forma parte del álgebra relacional original, pero se puede expresar usando:

- Join ( $\bowtie$ )
- Unión (U)
- Diferencia (-)
- Valores nulos o tuplas extendidas

Por eso se considera una **extensión práctica** del álgebra relacional, implementada en SQL.

#### 🌀 5. Resumen rápido

| Tipo       | Qué conserva                    | Coincidencias faltantes             |
|------------|---------------------------------|-------------------------------------|
| LEFT JOIN  | Todas las filas de la izquierda | Rellenadas con NULL en la derecha   |
| RIGHT JOIN | Todas las filas de la derecha   | Rellenadas con NULL en la izquierda |



|                  |                                 |                                        |
|------------------|---------------------------------|----------------------------------------|
| <b>FULL JOIN</b> | Todas las filas de ambas tablas | Rellenadas con NULL donde falten datos |
|------------------|---------------------------------|----------------------------------------|

## 6. Aplicaciones prácticas

- Ver **todos los empleados**, aunque no tengan departamento asignado (LEFT).
- Ver **todos los departamentos**, aunque no tengan empleados (RIGHT).
- Generar informes completos combinando datos de ambas fuentes (FULL).

|           |        |    |           |         |
|-----------|--------|----|-----------|---------|
| <b>R1</b> |        |    | <b>R2</b> |         |
| E#        | Nombre | D# | D#        | Descrip |
| 320       | José   | D1 | D1        | Central |
| 322       | Rosa   | D3 | D3        | I+D     |
| 323       | María  | D3 | D3        | I+D     |
| 324       | José   | D5 | D4        | Ventas  |

|                     |        |    |         |
|---------------------|--------|----|---------|
| <b>R1 * LEFT R2</b> |        |    |         |
| E#                  | Nombre | D# | Descrip |
| 320                 | José   | D1 | Central |
| 322                 | Rosa   | D3 | I+D     |
| 323                 | María  | D3 | I+D     |
| 324                 | José   | D5 | null    |

|                      |        |    |         |
|----------------------|--------|----|---------|
| <b>R1 * RIGHT R2</b> |        |    |         |
| E#                   | Nombre | D# | Descrip |
| 320                  | José   | D1 | Central |
| 322                  | Rosa   | D3 | I+D     |
| 323                  | María  | D3 | I+D     |
| null                 | null   | D4 | Ventas  |

|                |        |    |         |
|----------------|--------|----|---------|
| <b>R1 * R2</b> |        |    |         |
| E#             | Nombre | D# | Descrip |
| 320            | José   | D1 | Central |
| 322            | Rosa   | D3 | I+D     |
| 323            | María  | D3 | I+D     |

|                     |        |    |         |
|---------------------|--------|----|---------|
| <b>R1 * FULL R2</b> |        |    |         |
| E#                  | Nombre | D# | Descrip |
| 320                 | José   | D1 | Central |
| 322                 | Rosa   | D3 | I+D     |
| 323                 | María  | D3 | I+D     |
| 324                 | José   | D5 | null    |
| null                | null   | D4 | Ventas  |

### ◆ 1. Tablas base

#### Relación R1 (Empleados o Ingenieros)

| E#  | Nombre | D# |
|-----|--------|----|
| 320 | José   | D1 |
| 322 | Rosa   | D3 |
| 323 | María  | D3 |
| 324 | José   | D5 |

#### Relación R2 (Departamentos)

| D# | Descrip |
|----|---------|
| D1 | Central |
| D3 | I+D     |
| D4 | Ventas  |

### 🧩 2. JOIN normal (INNER JOIN o R1 \* R2)

Solo muestra las filas **que tienen coincidencia** en los atributos comunes (D#).

#### Resultado (R1 \* R2):

| E#  | Nombre | D# | Descrip |
|-----|--------|----|---------|
| 320 | José   | D1 | Central |
| 322 | Rosa   | D3 | I+D     |
| 323 | María  | D3 | I+D     |

- ♦ El registro con D5 no aparece porque **no existe en R2**.
- ♦ El D4 de R2 tampoco aparece porque **no existe en R1**.
- ♦ **\*3. LEFT OUTER JOIN (R1 LEFT R2)**

Devuelve **todas las filas de R1**, incluso las que **no tienen coincidencia** en R2.  
Las que no coinciden se **rellenan con NULL**.

| E#  | Nombre | D# | Descrip |
|-----|--------|----|---------|
| 320 | José   | D1 | Central |
| 322 | Rosa   | D3 | I+D     |
| 323 | María  | D3 | I+D     |
| 324 | José   | D5 | NULL    |

✓ **Todas las filas de R1 están presentes.**

→ José (D5) no tiene descripción porque no existe ese departamento → se rellena con NULL.

- ♦ **\*4. RIGHT OUTER JOIN (R1 RIGHT R2)**

Devuelve **todas las filas de R2**, incluso las que **no tienen coincidencia** en R1.  
Las filas sin pareja en R1 se completan con NULL.

| E#   | Nombre | D# | Descrip |
|------|--------|----|---------|
| 320  | José   | D1 | Central |
| 322  | Rosa   | D3 | I+D     |
| 323  | María  | D3 | I+D     |
| NULL | NULL   | D4 | Ventas  |

✓ **Todas las filas de R2 aparecen.**

→ El departamento D4 no tiene empleados → los valores de R1 son NULL.

- ♦ **\*5. FULL OUTER JOIN (R1 FULL R2)**

Combina el LEFT y el RIGHT JOIN:

Devuelve **todas las filas de ambas tablas**, con NULL donde no haya coincidencias.

| E#   | Nombre | D# | Descrip |
|------|--------|----|---------|
| 320  | José   | D1 | Central |
| 322  | Rosa   | D3 | I+D     |
| 323  | María  | D3 | I+D     |
| 324  | José   | D5 | NULL    |
| NULL | NULL   | D4 | Ventas  |

✓ **Muestra toda la información de R1 y R2**, sin perder ningún dato.

## 6. Equivalentes en SQL

-- LEFT JOIN

SELECT \*

FROM R1

LEFT JOIN R2 ON R1.D# = R2.D#;

```
-- RIGHT JOIN
SELECT *
FROM R1
RIGHT JOIN R2 ON R1.D# = R2.D#;
```

```
-- FULL JOIN
SELECT *
FROM R1
FULL OUTER JOIN R2 ON R1.D# = R2.D#;
```

## 7. Resumen comparativo

| Tipo de Join | Qué conserva             | Qué rellena con NULL  | Ejemplo SQL       |
|--------------|--------------------------|-----------------------|-------------------|
| INNER JOIN   | Solo coincidencias       | Nada                  | R1 JOIN R2 ON ... |
| LEFT JOIN    | Todas las filas de R1    | Datos faltantes de R2 | LEFT JOIN         |
| RIGHT JOIN   | Todas las filas de R2    | Datos faltantes de R1 | RIGHT JOIN        |
| FULL JOIN    | Todas las filas de ambas | Ambos lados           | FULL OUTER JOIN   |

## Asociacion:

### ■ Asociación o Theta Join ( $\theta$ -Join)

■  $R \bowtie_F S$  ó  $R *_F S$

- Define una relación que contiene las tuplas que satisfacen el predicado  $F$  en el producto cartesiano de  $R$  y  $S$
- El predicado  $F$  es de la forma  $R.a_i \theta S.b_j$ , donde  $\theta$  representa un operador de comparación ( $<, \leq, >, \geq, =, \neq$ )
- El predicado no tiene por qué definirse sobre atributos comunes
- Equivalencia con operadores básicos

$$R \bowtie_F S = \sigma_F(R \times S)$$

#### ■ Equijoin

- Si el predicado  $F$  contiene únicamente el operador de igualdad

### ◆ 1. Concepto de Asociación o Theta Join ( $\theta$ -Join)

El **Theta Join** ( $\bowtie_x$ ) combina dos relaciones **R** y **S** según una **condición de comparación (predicado F)** que puede usar distintos operadores.

■ En palabras simples:

👉 “Uní las filas de R y S donde se cumpla una condición cualquiera (no solo igualdad).”

### ⚙️ Notación formal

$$R \bowtie_F S \text{ o también } R *_F S$$

Donde:

- **R y S** → son las relaciones que se combinan.
- **F** → es el **predicado** o condición (por ejemplo, R.edad > S.edad).
- **θ** → representa el **operador de comparación** (<, ≤, >, ≥, =, ≠).

### 🧩 2. Equivalencia con operadores básicos

El Theta Join se define como:

$$R \bowtie_F S = \sigma_F(R \times S)$$

👉 Es decir:

1. Se realiza el **producto cartesiano** entre R y S.
2. Se aplica una **selección (σ)** usando la condición F.

### 🌿 3. Tipos de comparaciones posibles

- Igualdad (=)
- Mayor (>)
- Menor (<)
- Mayor o igual (≥)
- Menor o igual (≤)
- Distinto (≠)

■ Esto hace al Theta Join **más general** que el Natural Join, ya que **no necesita que los atributos tengan el mismo nombre ni tipo**.

### ♦ 4. Ejemplo práctico

**Relación R → Empleados**

| ID | Nombre | Edad |
|----|--------|------|
| 1  | José   | 34   |
| 2  | Rosa   | 28   |
| 3  | María  | 40   |

**Relación S → Proyectos**

| PID | Responsable | EdadMin |
|-----|-------------|---------|
| P1  | Central     | 30      |
| P2  | I+D         | 35      |

**Ejemplo de Theta Join:**

$$R \bowtie_{R.Edad > S.EdadMin} S$$

→ Combina empleados con proyectos donde la edad del empleado **sea mayor que la edad mínima requerida**.

| ID | Nombre | Edad | PID | Responsable | EdadMin |
|----|--------|------|-----|-------------|---------|
| 1  | José   | 34   | P1  | Central     | 30      |
| 3  | María  | 40   | P1  | Central     | 30      |
| 3  | María  | 40   | P2  | I+D         | 35      |

### 5. Equivalente en SQL

```
SELECT *
FROM Empleados E
JOIN Proyectos P
ON E.Edad > P.EdadMin;
```

→ SQL te permite cualquier condición en el ON, no solo igualdad.

### 6. Subtipo importante: EquiJoin

Si el predicado **F** contiene solo el operador de igualdad (=), entonces el **Theta Join** se convierte en un **EquiJoin**.

 Por ejemplo:

$R \bowtie_{\{R.D\# = S.D\#\}} S$

Es un **EquiJoin**, y si además eliminamos una columna duplicada, se convierte en un **Natural Join**.

### 7. Resumen rápido

| Elemento                 | Descripción                               |
|--------------------------|-------------------------------------------|
| <b>Símbolo</b>           | $R \bowtie_x S$ o $R *_x S$               |
| <b>Qué hace</b>          | Combina filas que cumplen una condición F |
| <b>Basado en</b>         | Producto cartesiano + Selección           |
| <b>Tipo de condición</b> | Cualquier comparación (<, >, =, etc.)     |
| <b>Subtipo</b>           | EquiJoin (solo usa =)                     |
| <b>SQL equivalente</b>   | SELECT * FROM R JOIN S ON condición;      |

### ◆ 8. Diferencias con otros Join

| Tipo                | Condición               | Atributos comunes | Resultado      |
|---------------------|-------------------------|-------------------|----------------|
| <b>Theta Join</b>   | Cualquier operador      | Opcional          | General        |
| <b>EquiJoin</b>     | Igualdad (=)            | Específicos       | Más común      |
| <b>Natural Join</b> | Igualdad (=) automática | Mismo nombre      | Sin duplicados |

Ejemplos:

- *Obtener los apellidos y teléfono de los alumnos de nombre Rosa*

$\pi_{\text{apellidos, telefono}} (\sigma_{\text{nombre}='Rosa'}(\text{Alumnos}))$

### Asignaturas

| CodA | NombreA  | Precio |
|------|----------|--------|
| 1    | Program. | 15000  |
| 2    | Dibujo   | 20000  |
| 3    | Inglés   | 18000  |

### Notas

| Nmat | CodA | Conv   | Nota |
|------|------|--------|------|
| 0338 | 1    | Feb 02 | 8    |
| 0254 | 2    | Feb 02 | 5    |
| 0168 | 2    | Feb 02 | 3    |
| 0338 | 2    | Feb 02 | 5    |
| 0338 | 3    | Jun 02 | 7    |
| 0254 | 1    | Jun 02 | 6    |
| 0168 | 1    | Jun 02 | 9    |
| 0168 | 3    | Jun 02 | 5    |

### Alumnos

| Nmat | Nombre | Apellidos     | Domicilio       | Telefono |
|------|--------|---------------|-----------------|----------|
| 0338 | Ana    | Pérez Gómez   | C / Julio nº 96 | 1112233  |
| 0254 | Rosa   | López López   | C/ Verano s/n   | 1113344  |
| 0168 | Juan   | García García | C/ Playa nº 1   | 1114455  |

| Apellidos   | Telefono |
|-------------|----------|
| López López | 1113344  |

- *Obtener las notas obtenidas en la asignatura de Inglés*

$\pi_{\text{nombre, apellidos, nota}} (\sigma_{\text{nombreA}='Ingles'}(\text{Alumnos*Notas*Asignaturas}))$

### Asignaturas

| CodA | NombreA  | Precio |
|------|----------|--------|
| 1    | Program. | 15000  |
| 2    | Dibujo   | 20000  |
| 3    | Inglés   | 18000  |

### Notas

| Nmat | CodA | Conv   | Nota |
|------|------|--------|------|
| 0338 | 1    | Feb 02 | 8    |
| 0254 | 2    | Feb 02 | 5    |
| 0168 | 2    | Feb 02 | 3    |
| 0338 | 2    | Feb 02 | 5    |
| 0338 | 3    | Jun 02 | 7    |
| 0254 | 1    | Jun 02 | 6    |
| 0168 | 1    | Jun 02 | 9    |
| 0168 | 3    | Jun 02 | 5    |

### Alumnos

| Nmat | Nombre | Apellidos     | Domicilio       | Telefono |
|------|--------|---------------|-----------------|----------|
| 0338 | Ana    | Pérez Gómez   | C / Julio nº 96 | 1112233  |
| 0254 | Rosa   | López López   | C/ Verano s/n   | 1113344  |
| 0168 | Juan   | García García | C/ Playa nº 1   | 1114455  |

| Nombre | Apellidos     | Nota |
|--------|---------------|------|
| Ana    | Pérez Gómez   | 7    |
| Juan   | García García | 5    |



- *Obtener los alumnos que figuren matriculados en todas las asignaturas*

**Asignaturas**

| CodA | NombreA  | Precio |
|------|----------|--------|
| 1    | Program. | 15000  |
| 2    | Dibujo   | 20000  |
| 3    | Inglés   | 18000  |

**Notas**

| Nmat | CodA | Conv   | Nota |
|------|------|--------|------|
| 0338 | 1    | Feb 02 | 8    |
| 0254 | 2    | Feb 02 | 5    |
| 0168 | 2    | Feb 02 | 3    |
| 0338 | 2    | Feb 02 | 5    |
| 0338 | 3    | Jun 02 | 7    |
| 0254 | 1    | Jun 02 | 6    |
| 0168 | 1    | Jun 02 | 9    |
| 0168 | 3    | Jun 02 | 5    |

**Alumnos**

| Nmat | Nombre | Apellidos     | Domicilio       | Telefono |
|------|--------|---------------|-----------------|----------|
| 0338 | Ana    | Pérez Gómez   | C / Julio nº 96 | 1112233  |
| 0254 | Rosa   | López López   | C/ Verano s/n   | 1113344  |
| 0168 | Juan   | García García | C/ Playa nº 1   | 1114455  |

$$\pi_{Nmat, codA} (Notas) \div \pi_{codA} (Asignaturas)$$

| Nmat |
|------|
| 0338 |
| 0168 |

- *Obtener los alumnos que figuren matriculados en todas las asignaturas*

$$\pi_{nombre, apellidos, (Alumnos * (\pi_{Nmat, codA} (Notas) \div \pi_{codA} (Asignaturas)))}$$

| Nombre | Apellidos     |
|--------|---------------|
| Ana    | Pérez Gómez   |
| Juan   | García García |

*Obtener los alumnos que figuren matriculados en las asignaturas de Inglés y Dibujo*

$$\pi_{Nmat} (\sigma_{nombreA='Inglés'} (Asignaturas) * Alumnos) \cap$$

$$\pi_{Nmat} (\sigma_{nombreA='Dibujo'} (Asignaturas) * Alumnos)$$

| Nmat |
|------|
| 0338 |
| 0168 |



- *Obtener los alumnos que no han suspendido ninguna asignatura*

$$\pi_{Nmat}(\sigma_{nota \geq 5}(Notas)) - \pi_{Nmat}(\sigma_{nota < 5}(Notas))$$

| Nmat |
|------|
| 0338 |
| 0254 |

## Unidad 2: Transacciones y Bloqueo

### ◆ 1. Definición de Transacción

Una **transacción** es una **unidad lógica de trabajo** que agrupa una o más operaciones sobre la base de datos (como inserciones, modificaciones o eliminaciones), que deben ejecutarse **todas o ninguna**.

■ En palabras simples:

👉 Una transacción garantiza que **los datos pasen de un estado consistente a otro**, sin errores intermedios.

Ejemplo:

BEGIN TRANSACTION

UPDATE Cuentas SET saldo = saldo - 100 WHERE id = 1;

UPDATE Cuentas SET saldo = saldo + 100 WHERE id = 2;

COMMIT;

→ Si una de las operaciones falla, **ningún cambio se aplica** (rollback automático).

### ◆ 2. Propiedades ACID

Toda transacción debe cumplir con las propiedades **ACID**:

| Propiedad                          | Descripción                                                | Ejemplo                                                                         |
|------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>A – Atomicidad</b>              | Todas las operaciones se ejecutan completamente o ninguna. | Si una transferencia falla, se deshace todo.                                    |
| <b>C – Consistencia</b>            | El estado de la BD pasa de un estado válido a otro.        | No se puede generar dinero "extra".                                             |
| <b>I – Aislamiento (Isolation)</b> | Una transacción no afecta a otra mientras se ejecuta.      | Dos usuarios no pueden modificar el mismo registro simultáneamente sin control. |
| <b>D – Durabilidad</b>             | Una vez confirmada (COMMIT), la transacción es permanente. | Si se apaga el servidor, los cambios ya grabados no se pierden.                 |

### ◆ 3. Tipos de Transacciones

### a) Implícitas

El gestor de base de datos ejecuta la transacción automáticamente sin necesidad de usar comandos BEGIN o COMMIT.

■ Ejemplo:

```
INSERT INTO Clientes VALUES (1, 'Ana');
```

→ Se ejecuta como una **transacción automática**, con un COMMIT implícito.

### b) Explícitas

El programador controla **cuándo comienza y cuándo termina** la transacción.

■ Ejemplo:

```
BEGIN TRANSACTION
```

```
UPDATE Stock SET cantidad = cantidad - 5 WHERE id_producto = 10;
```

```
UPDATE Ventas SET total = total + 5 WHERE id_venta = 23;
```

```
COMMIT;
```

→ Si algo falla, se puede usar ROLLBACK para revertir los cambios.

### ♦ 4. Transacciones Locales

Se ejecutan **dentro de un mismo servidor o base de datos**.

■ Ejemplo:

Una transacción que afecta solo tablas dentro de una misma BD PostgreSQL o SQL Server.

Ventajas:

- Más rápidas y simples.
- Menor riesgo de fallo en la comunicación.

### ♦ 5. Transacciones Distribuidas

Involucran **más de una base de datos o servidor**, coordinadas por un **gestor distribuido (coordinador de transacciones)**.

■ Ejemplo:

Un sistema bancario donde una transacción actualiza simultáneamente:

- Una BD en Buenos Aires (cuenta de origen).
- Otra BD en Córdoba (cuenta destino).

→ Se usan protocolos como el **Two-Phase Commit (2PC)** para asegurar que **todas las bases confirmen o cancelen juntas**.

### ♦ 6. Transacciones Anidadas

Son **transacciones dentro de otras transacciones**.

■ Ejemplo:

```
BEGIN TRAN
```

```
UPDATE Empleados SET sueldo = sueldo * 1.1 WHERE depto = 'Ventas';
```

```
BEGIN TRAN
```

```
INSERT INTO Historial VALUES ('Aumento aplicado', GETDATE());
```

COMMIT;

COMMIT;

→ Si la **transacción interna falla**, puede hacerse rollback sin afectar la externa (según configuración del SGBD).

Se usan en procesos grandes donde se quiere tener **control por bloques**.

## 7. Resumen rápido

| Tipo               | Descripción                               | Ejemplo                                        |
|--------------------|-------------------------------------------|------------------------------------------------|
| <b>Local</b>       | Afecta una sola BD.                       | Actualizar tablas en un mismo servidor.        |
| <b>Distribuida</b> | Involucra múltiples BDs o nodos.          | Transferencias entre bancos distintos.         |
| <b>Implícita</b>   | Se ejecuta automáticamente.               | INSERT, UPDATE, etc.                           |
| <b>Explícita</b>   | Controlada con BEGIN / COMMIT / ROLLBACK. | Bloque de operaciones atómicas.                |
| <b>Anidada</b>     | Una transacción dentro de otra.           | Registro + auditoría dentro del mismo proceso. |

## Instrucciones:

### ◆ 1. BEGIN TRANSACTION

Inicia una **transacción explícita**.

Desde este punto, todas las operaciones se consideran parte de la transacción hasta que se ejecute un COMMIT o ROLLBACK.

 Ejemplo:

BEGIN TRANSACTION

UPDATE Clientes SET saldo = saldo - 100 WHERE id = 1;

UPDATE Clientes SET saldo = saldo + 100 WHERE id = 2;

COMMIT;

 Si las dos operaciones se ejecutan correctamente → COMMIT confirma los cambios.

 Si ocurre un error → se puede usar ROLLBACK para deshacer todo.

Ej.

# BEGIN TRANSACTION

```
BEGIN { TRAN | TRANSACTION }
[ { nombre_transacción | @variable_tran }
[ WITH MARK [ 'descripcion' ] ] ]
```

## Ejemplo:

```
BEGIN TRANSACTION;
USE AdventureWorks;
DELETE FROM
AdventureWorks.HumanResources.JobCandidate
WHERE JobCandidateID = 13;
COMMIT TRANSACTION;
```

### ◆ 1. Qué hace BEGIN TRANSACTION

Inicia una **transacción explícita**, es decir, un bloque controlado de operaciones SQL que se ejecutan como una unidad.

Desde el momento en que se ejecuta BEGIN TRANSACTION, **todos los cambios** (INSERT, UPDATE, DELETE) quedan **pendientes** hasta que se ejecute un:

-  COMMIT TRANSACTION → para confirmar los cambios, o
-  ROLLBACK TRANSACTION → para deshacerlos.

### ◆ 2. Sintaxis general

```
BEGIN { TRAN | TRANSACTION }
[ { nombre_transacción | @variable_tran } ]
[ WITH MARK [ 'descripcion' ] ]
```

Veamos cada parte 🖐️

#### ◆ a) TRAN o TRANSACTION

Ambas palabras son equivalentes — podés usar cualquiera:

BEGIN TRAN

BEGIN TRANSACTION

#### ◆ b) { nombre\_transacción | @variable\_tran }

Podés **asignarle un nombre** a la transacción o usar una **variable**.

Esto sirve para identificar o manejar transacciones anidadas.

#### ■ Ejemplo:

```
BEGIN TRANSACTION ActualizacionStock
UPDATE Productos SET stock = stock - 10 WHERE id = 5;
COMMIT TRANSACTION ActualizacionStock;
```

👉 El nombre es **opcional**, pero útil si tenés varias transacciones en un mismo script.

#### ◆ c) WITH MARK ['descripcion']

Se usa para **marcar la transacción en el registro del sistema (log)** con una descripción textual.

Sirve en **administración o auditoría** para identificar puntos importantes.

 Ejemplo:

```
BEGIN TRANSACTION BackupMensual WITH MARK 'Inicio de respaldo mensual';
```

### ◆ 3. Ejemplo de la diapositiva

```
BEGIN TRANSACTION;
```

```
USE AdventureWorks;
```

```
DELETE FROM AdventureWorks.HumanResources.JobCandidate
```

```
WHERE JobCandidateID = 13;
```

```
COMMIT TRANSACTION;
```

### ◆ Paso a paso:

1. **BEGIN TRANSACTION** → comienza la transacción.
2. **USE AdventureWorks** → selecciona la base de datos.
3. **DELETE FROM ...** → elimina un registro de la tabla JobCandidate.
4. **COMMIT TRANSACTION** → confirma la eliminación y la hace permanente.

 Si todo se ejecuta correctamente → se guarda el cambio.

 Si hay un error (por ejemplo, restricción de clave foránea) → se puede hacer **ROLLBACK TRANSACTION** para revertir.

### 4. Importancia práctica

- Permite agrupar operaciones críticas (como transferencias o actualizaciones masivas).
- Evita inconsistencias si algo falla durante la ejecución.
- Da control total al programador sobre cuándo aplicar los cambios.

### 5. Resumen rápido

| Elemento                    | Descripción                                         |
|-----------------------------|-----------------------------------------------------|
| <b>BEGIN TRANSACTION</b>    | Inicia una transacción explícita.                   |
| <b>nombre_transacción</b>   | (Opcional) Identifica la transacción.               |
| <b>WITH MARK 'texto'</b>    | Marca la transacción en el log con una descripción. |
| <b>COMMIT TRANSACTION</b>   | Finaliza y guarda los cambios.                      |
| <b>ROLLBACK TRANSACTION</b> | Cancela y revierte los cambios.                     |

### ◆ 2. BEGIN DISTRIBUTED TRANSACTION

Inicia una **transacción distribuida**, que puede involucrar **múltiples servidores o bases de datos**.

Se usa principalmente en SQL Server con **Linked Servers**.

■ Ejemplo:

BEGIN DISTRIBUTED TRANSACTION

UPDATE ServidorA.BaseA.dbo.Cuentas

SET saldo = saldo - 500

WHERE id = 10;

UPDATE ServidorB.BaseB.dbo.Cuentas

SET saldo = saldo + 500

WHERE id = 20;

COMMIT TRANSACTION;

✅ Si uno de los servidores falla → se ejecuta un **rollback automático** en todos. Esto garantiza la **consistencia global** (usando el protocolo *Two-Phase Commit*).

■

## BEGIN DISTRIBUTED TRANSACTION

- MS DTC
- Ejemplo:
 

```

BEGIN DISTRIBUTED TRANSACTION;
-- Borra un registro sobre la base de datos local
DELETE AdventureWorks.HumanResources.JobCandidate
WHERE JobCandidateID = 13;
-- Borra un registro sobre la base de datos remota
DELETE
RemoteServer.AdventureWorks.HumanResources.JobCa
ndidate
WHERE JobCandidateID = 13;
COMMIT TRANSACTION;
          
```

### ◆ 1. Qué es una Transacción Distribuida

Es una transacción que **afecta a más de una base de datos o servidor**, y garantiza que **todas las operaciones se ejecuten correctamente en todos los sitios**,

o bien que **todas se reviertan** si ocurre algún error.

■ En otras palabras:

👉 “O todas las bases se actualizan, o ninguna lo hace.”

### ◆ 2. El rol del coordinador — MS DTC

- Las transacciones distribuidas son gestionadas por un **coordinador de transacciones**, en SQL Server llamado **MS DTC (Microsoft Distributed Transaction Coordinator)**.
- Este servicio se encarga de **coordinar la confirmación (COMMIT)** o la **cancelación (ROLLBACK)** en todos los servidores involucrados.

■ El protocolo que usa se llama **Two-Phase Commit (2PC)**:



### 1. Fase 1 – Preparación:

El coordinador pregunta a cada base si puede confirmar los cambios.

### 2. Fase 2 – Confirmación o Cancelación:

Si todas responden “sí”, se ejecuta el COMMIT.

Si alguna falla, se hace ROLLBACK en todas.

### ◆ 3. Sintaxis

BEGIN DISTRIBUTED TRANSACTION

-- operaciones en múltiples bases o servidores

COMMIT TRANSACTION

### ◆ 4. Ejemplo (como el de la diapositiva)

BEGIN DISTRIBUTED TRANSACTION;

-- ◆ Operación en la base local

DELETE AdventureWorks.HumanResources.JobCandidate

WHERE JobCandidateID = 13;

-- ◆ Operación en la base remota

DELETE RemoteServer.AdventureWorks.HumanResources.JobCandidate

WHERE JobCandidateID = 13;

COMMIT TRANSACTION;

### 🧠 Explicación paso a paso:

#### 1. BEGIN DISTRIBUTED TRANSACTION

→ Inicia una transacción controlada por **MS DTC**.

#### 2. Primera sentencia DELETE

→ Afecta una tabla en la **base local**.

#### 3. Segunda sentencia DELETE

→ Afecta una tabla idéntica en otro **servidor remoto (RemoteServer)**.

#### 4. COMMIT TRANSACTION

→ Confirma los cambios **solo si ambas operaciones fueron exitosas**.

Si alguna falla, **MS DTC ejecuta ROLLBACK en todas las bases** para mantener la consistencia.

### 💻 5. Ejemplo visual de ejecución

| Servidor      | Acción                              | Resultado              |
|---------------|-------------------------------------|------------------------|
| Local         | Elimina registro ID=13              | OK                     |
| Remoto        | Elimina registro ID=13              | OK                     |
| <b>MS DTC</b> | Ambas operaciones exitosas → COMMIT | ✅ Confirmadas en ambos |

Si el servidor remoto falla:

| Servidor | Acción           | Resultado |
|----------|------------------|-----------|
| Local    | Elimina registro | OK        |
| Remoto   | Falla conexión   | ❌         |

|               |                           |                                                                                                   |
|---------------|---------------------------|---------------------------------------------------------------------------------------------------|
| <b>MS DTC</b> | Ejecuta ROLLBACK en ambos |  Reversión total |
|---------------|---------------------------|---------------------------------------------------------------------------------------------------|

## ⚙️ 6. Requisitos técnicos

- El servicio **MSDTC** debe estar **habilitado y ejecutándose** en todos los servidores.
- Se deben configurar las conexiones de red y seguridad para permitir transacciones distribuidas.
- Todos los servidores deben tener **linked servers** o conexiones definidas.

## 🎯 7. Resumen rápido

| Elemento                             | Descripción                                             |
|--------------------------------------|---------------------------------------------------------|
| <b>BEGIN DISTRIBUTED TRANSACTION</b> | Inicia una transacción entre varias bases o servidores. |
| <b>MS DTC</b>                        | Servicio que coordina la transacción distribuida.       |
| <b>Protocolo usado</b>               | Two-Phase Commit (2PC).                                 |
| <b>Ventaja</b>                       | Garantiza integridad entre bases separadas.             |
| <b>Desventaja</b>                    | Más lenta y requiere configuración de red.              |

### 💎 3. COMMIT TRANSACTION

Finaliza correctamente una transacción, **confirmando** todos los cambios en la base de datos.

📘 Ejemplo:

```
BEGIN TRANSACTION
```

```
INSERT INTO Productos VALUES ('Yerba', 2500);
```

```
COMMIT TRANSACTION;
```

✅ Todo lo hecho dentro de la transacción **queda grabado de forma permanente**.



### 💎 1. Qué hace COMMIT TRANSACTION

Confirma (graba permanentemente) todos los cambios realizados desde que comenzó la transacción con BEGIN TRANSACTION.

📘 En palabras simples:

👉 “Guarda definitivamente todo lo que hiciste en la transacción.”

Después del COMMIT, los cambios **no pueden deshacerse**.

### 💎 2. Sintaxis general

```
COMMIT { TRAN | TRANSACTION }  
[ nombre_transaccion | @variable_tran ]
```

#### ◆ a) TRAN o TRANSACTION

Ambas formas son equivalentes:

```
COMMIT TRAN
```

```
COMMIT TRANSACTION
```

#### ◆ b) [ nombre\_transaccion | @variable\_tran ]

- Si le diste un nombre a la transacción al comenzar, podés repetirlo aquí.
- No es obligatorio, pero se usa cuando tenés **transacciones anidadas o controladas por variables**.

■ Ejemplo:

```
BEGIN TRANSACTION ActualizacionClientes
```

```
UPDATE Clientes SET saldo = saldo + 200 WHERE id = 1;
```

```
COMMIT TRANSACTION ActualizacionClientes;
```

#### ◆ 3. Ejemplo completo

```
BEGIN TRANSACTION;
```

```
DELETE FROM Candidatos WHERE JobCandidateID = 13;
```

```
COMMIT TRANSACTION;
```

→ Si el DELETE se ejecuta correctamente → los cambios se confirman.

→ Si ocurre un error y no se llega al COMMIT → la transacción **queda pendiente o se revierte**.

#### ◆ 4. Uso de @@TRANCOUNT

@@TRANCOUNT es una **variable de sistema** que indica cuántas transacciones están actualmente abiertas.

■ Ejemplo:

```
SELECT @@TRANCOUNT; -- Muestra el número de transacciones activas
```

```
BEGIN TRANSACTION;
```

```
SELECT @@TRANCOUNT; -- Devuelve 1
```

```
BEGIN TRANSACTION;
```

```
SELECT @@TRANCOUNT; -- Devuelve 2 (anidada)
```

```
COMMIT TRANSACTION;
```

```
SELECT @@TRANCOUNT; -- Devuelve 1
```

```
COMMIT TRANSACTION;
```

```
SELECT @@TRANCOUNT; -- Devuelve 0 (todas cerradas)
```

✅ Esto es muy útil para saber si hay transacciones abiertas o si una parte del código olvidó cerrarlas.

### 🧠 5. ¿Qué pasa si no haces COMMIT?

- Si una transacción queda abierta y el proceso termina (por error o apagado), el **SGBD ejecuta un rollback automático**.
- En sistemas críticos (bancos, ERP, etc.), no hacer COMMIT puede **bloquear registros** y generar **deadlocks**.

### 🎯 6. Resumen rápido

| Elemento                    | Descripción                                                               |
|-----------------------------|---------------------------------------------------------------------------|
| <b>Función</b>              | Confirma (graba) los cambios de una transacción.                          |
| <b>Sintaxis corta/larga</b> | COMMIT TRAN / COMMIT TRANSACTION                                          |
| <b>Parámetro opcional</b>   | nombre_transacción o @variable_tran                                       |
| <b>Variable útil</b>        | @@TRANCOUNT → indica cuántas transacciones están abiertas                 |
| <b>Efecto</b>               | Hace permanentes todos los cambios realizados desde el BEGIN TRANSACTION. |

### ♦ 4. ROLLBACK TRANSACTION

Cancela una transacción y **revierte todos los cambios** realizados desde el BEGIN TRANSACTION.

■ Ejemplo:

```
BEGIN TRANSACTION
```

```
UPDATE Stock SET cantidad = cantidad - 10 WHERE id_producto = 1;
```

```
IF @@ERROR <> 0
```

```
    ROLLBACK TRANSACTION;
```

```
COMMIT TRANSACTION;
```

♦ También puede usarse junto con **puntos de guardado (SAVE TRANSACTION)** para revertir **solo una parte** de la transacción.

■ ROLLBACK TRANSACTION

- ROLLBACK { TRAN | TRANSACTION }  
[ nombre\_transaction | @variable\_tran  
| nombre\_savepoint |  
@variable\_savepoint ]
- Control de Errores

## ◆ 1. Qué hace ROLLBACK TRANSACTION

Cancela una transacción activa y **revierte todos los cambios** realizados desde el último BEGIN TRANSACTION.

■ En palabras simples:

👉 "Deshace todo lo que hiciste dentro de la transacción, dejando la base de datos como estaba antes de empezar."

## ◆ 2. Sintaxis general

ROLLBACK { TRAN | TRANSACTION }

[ nombre\_transaccion | @variable\_tran

| nombre\_savepoint | @variable\_savepoint ]

### ◆ a) TRAN o TRANSACTION

Podés usar cualquiera de las dos palabras, son equivalentes:

ROLLBACK TRAN

ROLLBACK TRANSACTION

### ◆ b) [ nombre\_transaccion | @variable\_tran ]

Permite **especificar qué transacción** querés revertir (si hay más de una activa o anidada).

■ Ejemplo:

BEGIN TRANSACTION Principal

UPDATE Clientes SET saldo = saldo + 100;

BEGIN TRANSACTION Secundaria

UPDATE Clientes SET saldo = saldo - 500;

ROLLBACK TRANSACTION Secundaria; -- Solo cancela la interna

COMMIT TRANSACTION Principal;

✓ Solo la transacción **Secundaria** se deshace.

### ◆ c) [ nombre\_savepoint | @variable\_savepoint ]

Sirve para revertir hasta un **punto de guardado** definido con SAVE TRANSACTION. Esto **no cancela toda la transacción**, solo vuelve atrás parcialmente.

■ Ejemplo:

BEGIN TRANSACTION

INSERT INTO Clientes VALUES ('Ana');

SAVE TRANSACTION Punto1;

INSERT INTO Clientes VALUES ('Juan');

ROLLBACK TRANSACTION Punto1; -- Solo revierte desde el punto

COMMIT TRANSACTION;

✓ La inserción de Ana se mantiene, la de Juan se elimina.

### ◆ 3. Control de Errores (TRY...CATCH)

En SQL Server, el ROLLBACK se usa frecuentemente junto con **bloques TRY/CATCH** para manejar errores y mantener la base consistente.

■ Ejemplo:

```
BEGIN TRY
  BEGIN TRANSACTION
    UPDATE Productos SET stock = stock - 10 WHERE id = 1;
    UPDATE Ventas SET total = total + 10 WHERE id = 1;
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  ROLLBACK TRANSACTION;
  PRINT 'Error detectado. Cambios revertidos.';
END CATCH
```

✅ Si ocurre un error en cualquier UPDATE, el sistema **revierte automáticamente** todos los cambios anteriores dentro de esa transacción.

### 🧠 4. Ejemplo simple

```
BEGIN TRANSACTION;
  DELETE FROM Empleados WHERE id = 10;
ROLLBACK TRANSACTION;
```

➡ El registro **no se elimina** realmente porque el ROLLBACK deshace la operación.

### 🎯 5. Resumen rápido

| Elemento             | Descripción                                                  |
|----------------------|--------------------------------------------------------------|
| Función              | Deshace todos los cambios desde el último BEGIN TRANSACTION. |
| Sintaxis corta/larga | ROLLBACK TRAN / ROLLBACK TRANSACTION                         |
| Con nombre           | Revierte una transacción específica.                         |
| Con SAVEPOINT        | Revierte solo hasta un punto de guardado.                    |
| Uso común            | Control de errores, consistencia de datos.                   |

### 🧩 6. Comparativa visual

| Instrucción          | Acción           | Resultado                       |
|----------------------|------------------|---------------------------------|
| COMMIT TRANSACTION   | Confirma cambios | ✅ Se guardan permanentemente    |
| ROLLBACK TRANSACTION | Cancela cambios  | 🔄 Todo vuelve al estado inicial |

### ◆ 5. SAVE TRANSACTION



Crea un **punto de guardado (savepoint)** dentro de una transacción, para poder deshacer cambios **parciales** sin cancelar toda la transacción.

■ Ejemplo:

```
BEGIN TRANSACTION
```

```
INSERT INTO Clientes VALUES ('Ana');
```

```
SAVE TRANSACTION Punto1;
```

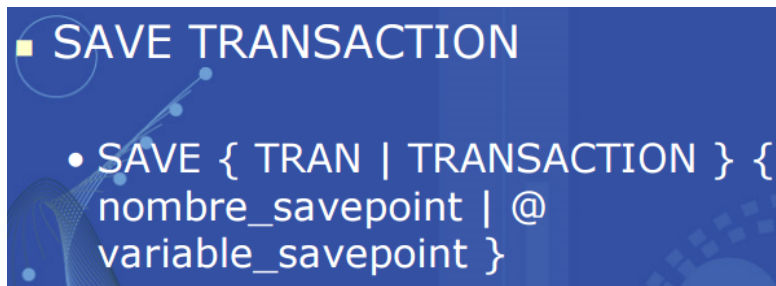
```
INSERT INTO Clientes VALUES ('Juan');
```

```
ROLLBACK TRANSACTION Punto1; -- deshace solo hasta Punto1
```

```
COMMIT TRANSACTION;
```

✓ Se guarda la inserción de Ana, pero se cancela la de Juan.

Sirve en procesos largos donde querés recuperar sin perder todo el trabajo.



#### ◆ 1. Qué hace SAVE TRANSACTION

Crea un punto de guardado (savepoint) dentro de una transacción activa.

Permite hacer un rollback parcial sin deshacer toda la transacción.

■ En palabras simples:

👉 "Guarda un punto intermedio al que podés volver si algo falla después."

#### ◆ 2. Sintaxis general

```
SAVE { TRAN | TRANSACTION } { nombre_savepoint | @variable_savepoint }
```

- TRAN o TRANSACTION → son equivalentes.
- nombre\_savepoint → nombre del punto de guardado.
- También se puede usar una variable (@variable\_savepoint).

#### ◆ 3. Cómo funciona

1. Iniciás una transacción con BEGIN TRANSACTION.
2. Hacés algunas operaciones.
3. Guardás un punto con SAVE TRANSACTION punto1.
4. Si algo sale mal, podés volver hasta ese punto, sin deshacer lo anterior.

✚ Ejemplo práctico

```
BEGIN TRANSACTION;
```

```
INSERT INTO Clientes VALUES ('Ana', 'Salta');
SAVE TRANSACTION Punto1;
```

```
INSERT INTO Clientes VALUES ('Juan', 'Jujuy');
INSERT INTO Clientes VALUES ('María', 'Tucumán');
```

```
-- Supongamos que ocurre un error con el último INSERT
ROLLBACK TRANSACTION Punto1;
```

COMMIT TRANSACTION;

◆ Resultado:

- Se mantiene la inserción de Ana (antes del punto de guardado).
- Se revierten las inserciones de Juan y María (después del savepoint).
- Luego se confirma lo restante con COMMIT.

💡 5. Ejemplo con variables

También podés guardar el nombre del punto en una variable:

```
BEGIN TRANSACTION;
DECLARE @Punto VARCHAR(20) = 'Mitad';
UPDATE Productos SET stock = stock - 10 WHERE id = 1;
SAVE TRANSACTION @Punto;
UPDATE Productos SET stock = stock + 5 WHERE id = 2;
ROLLBACK TRANSACTION @Punto;
COMMIT TRANSACTION;
```

✅ Solo el primer UPDATE se mantiene.

🧠 6. Casos de uso comunes

- Procesos largos o por lotes: te permite retroceder sin perder todo el progreso.
- Bloques TRY/CATCH: combinado con ROLLBACK TRANSACTION, se usa para manejar errores controladamente.
- Sistemas críticos: cuando solo parte del proceso debe deshacerse ante un fallo.

🎯 7. Resumen rápido

| Elemento         | Descripción                                                 |
|------------------|-------------------------------------------------------------|
| Comando          | SAVE TRANSACTION                                            |
| Función          | Crea un punto de guardado dentro de una transacción.        |
| Rollback parcial | Permite volver a un punto sin deshacer toda la transacción. |

|                       |                                       |
|-----------------------|---------------------------------------|
| Parámetros            | Nombre o variable del savepoint.      |
| Combinación frecuente | ROLLBACK TRANSACTION nombre_savepoint |

#### ♦ 8. Diferencia con ROLLBACK total

| Comando                     | Qué hace                         |
|-----------------------------|----------------------------------|
| ROLLBACK TRANSACTION        | Deshace toda la transacción.     |
| ROLLBACK TRANSACTION punto1 | Deshace solo hasta el savepoint. |

#### ♦ 6. XACT\_STATE()

Es una **función de sistema** que devuelve el **estado actual de la transacción**. Muy útil para manejar errores de forma controlada.

| Valor | Significado                                                         |
|-------|---------------------------------------------------------------------|
| 1     | Hay una transacción activa y válida (se puede hacer COMMIT).        |
| -1    | La transacción está activa pero con error (solo se puede ROLLBACK). |
| 0     | No hay ninguna transacción activa.                                  |

■ Ejemplo:

```
BEGIN TRANSACTION
```

```
UPDATE Ventas SET total = total + 500 WHERE id = 10;
```

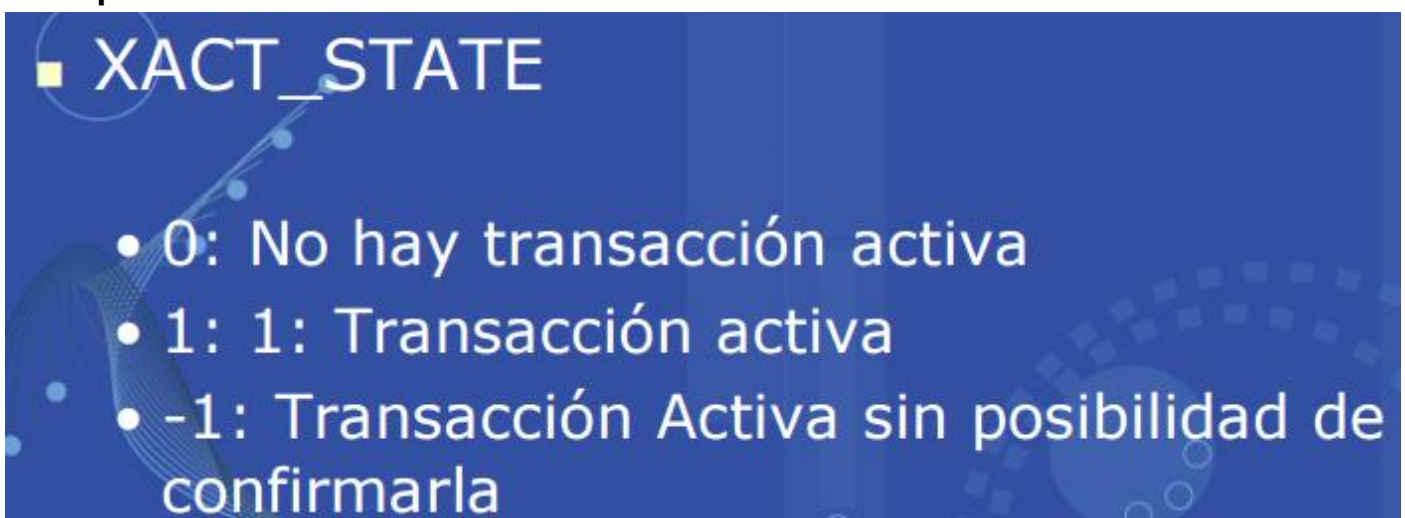
```
IF (XACT_STATE() = 1)
```

```
    COMMIT TRANSACTION;
```

```
ELSE IF (XACT_STATE() = -1)
```

```
    ROLLBACK TRANSACTION;
```

✅ Esto garantiza que tu aplicación **no quede con transacciones abiertas o corruptas**.



**XACT\_STATE**

- 0: No hay transacción activa
- 1: Transacción activa
- -1: Transacción Activa sin posibilidad de confirmarla

#### ♦ 1. Qué es XACT\_STATE()

Es una función de sistema en SQL Server que devuelve el estado actual de la transacción del usuario.

Sirve para saber en qué condición se encuentra la transacción antes de intentar confirmarla (COMMIT) o revertirla (ROLLBACK).

■ En palabras simples:

👉 "Te dice si hay una transacción activa, si está dañada o si no hay ninguna."

◆ 2. Valores posibles que devuelve

| Valor | Significado                        | Acción recomendada                       |
|-------|------------------------------------|------------------------------------------|
| 0     | ✗ No hay transacción activa        | No hay nada que confirmar ni revertir.   |
| 1     | ✓ Transacción activa y consistente | Se puede hacer COMMIT o ROLLBACK.        |
| -1    | ⚠ Transacción activa pero dañada   | Solo se puede hacer ROLLBACK, no COMMIT. |

◆ 3. Ejemplo práctico

BEGIN TRY

BEGIN TRANSACTION;

UPDATE Productos SET precio = precio / 0; -- Error intencional (división por cero)

COMMIT TRANSACTION;

END TRY

BEGIN CATCH

SELECT XACT\_STATE() AS Estado; -- Devuelve -1 (transacción dañada)

IF (XACT\_STATE()) = -1

ROLLBACK TRANSACTION;

END CATCH

◆ Explicación:

- Se inicia una transacción.
- Ocurre un error (división por cero).
- SQL entra al bloque CATCH.
- XACT\_STATE() devuelve -1 porque la transacción está incompleta y no puede confirmarse.
- Por lo tanto, solo se puede revertir con ROLLBACK.

◆ 4. Uso típico

Generalmente se usa en conjunto con TRY...CATCH para asegurar que la base de datos no quede bloqueada o inconsistente:

BEGIN TRY

BEGIN TRANSACTION;

```
-- Operaciones críticas
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF (XACT_STATE()) = -1
        ROLLBACK TRANSACTION; -- Daño total, revertir todo
    ELSE IF (XACT_STATE()) = 1
        ROLLBACK TRANSACTION; -- Error leve, aún se puede revertir
END CATCH
```

✅ Con esto te aseguras de manejar errores graves o leves sin corromper la transacción.

#### ♦ 5. Diferencia con @@TRANCOUNT

| Función      | Qué mide                          | Valores posibles |
|--------------|-----------------------------------|------------------|
| @@TRANCOUNT  | Cantidad de transacciones activas | 0, 1, 2, ...     |
| XACT_STATE() | Estado lógico de la transacción   | -1, 0, 1         |

#### 🎯 6. Resumen rápido

| Valor | Estado    | Descripción breve                                    |
|-------|-----------|------------------------------------------------------|
| 0     | No activa | No hay transacción en curso.                         |
| 1     | Activa    | Transacción válida → puede confirmarse o revertirse. |
| -1    | Dañada    | Transacción fallida → solo puede revertirse.         |

#### 💡 7. Uso práctico en sistemas reales

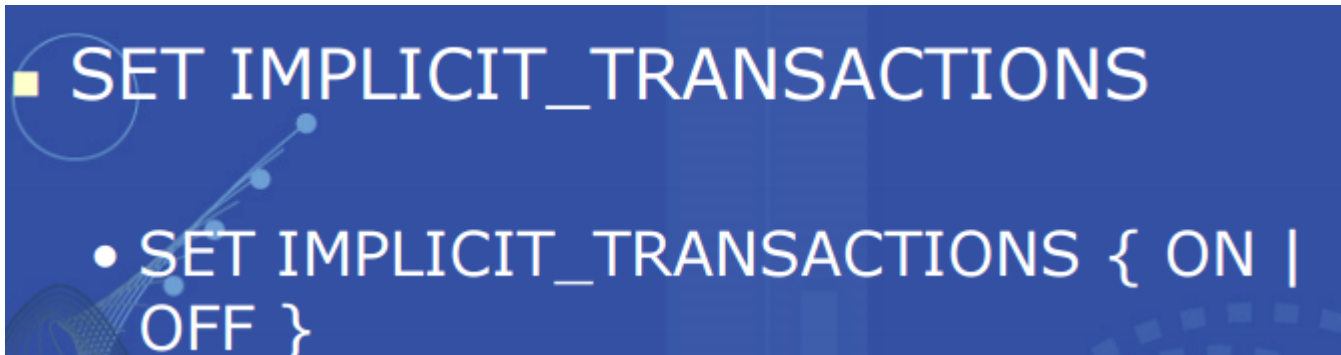
Se utiliza en **procesos de pago, inventario o reservas** donde los errores parciales pueden dañar datos críticos.

Ejemplo: si falla una inserción a mitad de una compra, XACT\_STATE() permite revertir solo esa parte y dejar todo consistente.

#### 🎯 Resumen general

| Instrucción                          | Descripción                                | Ejemplo / Uso                 |
|--------------------------------------|--------------------------------------------|-------------------------------|
| <b>BEGIN TRANSACTION</b>             | Inicia una transacción local.              | BEGIN TRANSACTION             |
| <b>BEGIN DISTRIBUTED TRANSACTION</b> | Inicia una transacción entre varias bases. | BEGIN DISTRIBUTED TRANSACTION |
| <b>COMMIT TRANSACTION</b>            | Confirma los cambios.                      | COMMIT                        |
| <b>ROLLBACK TRANSACTION</b>          | Revierte todos los cambios.                | ROLLBACK                      |

|                         |                                        |                                         |
|-------------------------|----------------------------------------|-----------------------------------------|
| <b>SAVE TRANSACTION</b> | Crea un punto de restauración parcial. | SAVE TRANSACTION punto1                 |
| <b>XACT_STATE()</b>     | Informa el estado de la transacción.   | Control de errores en bloques TRY-CATCH |



### ◆ 1. Qué es SET IMPLICIT\_TRANSACTIONS

Controla si las transacciones se inician automáticamente o no.

En otras palabras, define si las transacciones deben iniciarse **manual** o **automáticamente** cada vez que ejecutás una sentencia SQL.

■ En palabras simples:

👉 “Decide si SQL empieza una transacción solo cuando vos la pedís (manual), o si lo hace automáticamente cada vez que ejecutás algo (implícita).”

### ◆ 2. Sintaxis

SET IMPLICIT\_TRANSACTIONS { ON | OFF };

### ◆ 3. Modo OFF (por defecto)

- Las transacciones **no se inician automáticamente**.
- Debés usar explícitamente BEGIN TRANSACTION para empezar una.

■ Ejemplo:

```
SET IMPLICIT_TRANSACTIONS OFF;
BEGIN TRANSACTION;
UPDATE Productos SET stock = stock - 5 WHERE id = 1;
COMMIT TRANSACTION;
```

✅ Solo hay transacción si la definís con BEGIN TRANSACTION.

### ◆ 4. Modo ON (implícito)

- Cada sentencia de ciertos tipos (**INSERT, UPDATE, DELETE, SELECT INTO, etc.**) **inicia automáticamente una transacción**.
- Pero **no se confirma sola**: debés hacer COMMIT o ROLLBACK manualmente.
- Hasta que lo hagas, la transacción **permanece abierta**.

■ Ejemplo:

```
SET IMPLICIT_TRANSACTIONS ON;
UPDATE Productos SET stock = stock - 10 WHERE id = 2;
```



-- Aquí SQL Server abrió una transacción automáticamente

COMMIT TRANSACTION; -- confirmás los cambios manualmente

⚠ Si te olvidás de confirmar, la transacción queda abierta y puede **bloquear tablas**.

#### ♦ 5. Qué operaciones disparan una transacción implícita

Cuando está activado (ON), SQL crea una transacción automáticamente al ejecutar cualquiera de estas operaciones:

- INSERT
- UPDATE
- DELETE
- SELECT INTO
- CREATE TABLE
- DROP TABLE
- ALTER TABLE
- GRANT, REVOKE, DENY (de permisos)

#### ♦ 6. Cuándo usarlo

✅ **Útil en ambientes de desarrollo o pruebas**, donde querés controlar manualmente cuándo se confirman los cambios.

⚠ **No recomendable en producción** si el equipo no tiene cuidado con los commits, porque puede dejar bloqueos abiertos.

#### ♦ 7. Diferencia entre Implícita y Explícita

| Tipo de Transacción | Cómo se inicia                              | Cómo se confirma                  |
|---------------------|---------------------------------------------|-----------------------------------|
| <b>Explícita</b>    | Con BEGIN TRANSACTION                       | Con COMMIT o ROLLBACK             |
| <b>Implícita</b>    | Automáticamente (al ejecutar una sentencia) | Manualmente con COMMIT o ROLLBACK |

#### ✿ 8. Ejemplo comparativo

##### ♦ Implícita

SET IMPLICIT\_TRANSACTIONS ON;

DELETE FROM Empleados WHERE id = 1;

-- Se inicia automáticamente una transacción

COMMIT TRANSACTION;

##### ♦ Explícita

SET IMPLICIT\_TRANSACTIONS OFF;

BEGIN TRANSACTION;

DELETE FROM Empleados WHERE id = 1;

COMMIT TRANSACTION;

👉 Resultado final es el mismo, pero en la primera no escribís el BEGIN TRANSACTION.

## 9. Resumen rápido

| Valor | Significado                            | Efecto                                        |
|-------|----------------------------------------|-----------------------------------------------|
| ON    | Transacciones implícitas activadas     | SQL abre una transacción automáticamente.     |
| OFF   | Transacciones explícitas (por defecto) | Solo hay transacción si la iniciás con BEGIN. |

# Bloqueos:

## BLOQUEOS (LOCKS)

Los **bloqueos** son mecanismos que usa el **Sistema Gestor de Base de Datos (SGBD)** —por ejemplo, SQL Server— para **controlar el acceso concurrente a los datos**.

👉 Su objetivo principal es **garantizar la integridad y consistencia** de la información cuando varias transacciones acceden o modifican los mismos datos al mismo tiempo.

### ♦ 1. Modos de Bloqueos

Los principales **tipos o modos de bloqueo** son:

| Tipo de bloqueo                      | Nombre                           | Descripción                                                                                                                            |
|--------------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>S (Shared)</b>                    | Compartido                       | Permite leer datos, pero no modificarlos. Varias transacciones pueden tener este tipo de bloqueo simultáneamente.                      |
| <b>X (Exclusive)</b>                 | Exclusivo                        | Impide que otros lean o modifiquen el dato. Solo una transacción puede tenerlo.                                                        |
| <b>U (Update)</b>                    | Actualización                    | Intermedio entre lectura y escritura; se usa cuando una transacción planea actualizar datos que está leyendo. Evita <i>deadlocks</i> . |
| <b>IS (Intent Shared)</b>            | Intención compartida             | Indica que dentro de un conjunto (p. ej., una tabla) hay bloqueos de lectura en niveles más bajos (p. ej., filas).                     |
| <b>IX (Intent Exclusive)</b>         | Intención exclusiva              | Indica que habrá bloqueos de escritura en niveles inferiores.                                                                          |
| <b>SIX (Shared Intent Exclusive)</b> | Compartido + intención exclusiva | Lee datos, pero planea modificarlos más adelante.                                                                                      |

### ♦ 2. Compatibilidad de Bloqueos

SQL Server verifica si los bloqueos pueden **coexistir**.

Ejemplo clásico:

|   | S | U | X |
|---|---|---|---|
| S | ✓ | ✗ | ✗ |
| U | ✗ | ✗ | ✗ |
| X | ✗ | ✗ | ✗ |

✓ = compatible → puede coexistir

✗ = incompatible → debe esperar

👉 Si una transacción intenta obtener un bloqueo incompatible, **queda en espera** hasta que el otro lo libere o el tiempo de espera se agote.

### ♦ 3. Bloqueo Dinámico

SQL Server ajusta dinámicamente el nivel de bloqueo según el número de filas afectadas:

- Si se bloquean **pocas filas**, usa **bloqueos a nivel de fila** (granularidad fina).
- Si se afectan **muchas filas**, **eleva el bloqueo** al nivel de **página o tabla** (granularidad gruesa).

■ Esto mejora el rendimiento y reduce la sobrecarga de administrar muchos bloqueos pequeños.

### ♦ 4. Personalizar el tiempo de espera de bloqueo

Podés definir cuánto tiempo una transacción esperará un recurso bloqueado antes de generar error:

SET LOCK\_TIMEOUT 5000; -- 5000 milisegundos (5 segundos)

- ♦ Si el tiempo se agota, SQL Server lanza un error y cancela la operación.
- ♦ Por defecto, el tiempo de espera es **infinito** (espera hasta que el bloqueo se libere).

### ♦ 5. Granularidad y Jerarquías de Bloqueos

Los bloqueos pueden aplicarse en diferentes **niveles de granularidad**:

| Nivel de bloqueo   | Ejemplo                  | Cuándo se usa                                            |
|--------------------|--------------------------|----------------------------------------------------------|
| Fila (Row)         | Un registro de una tabla | Cuando solo se afecta una pequeña porción de datos.      |
| Página (Page)      | Grupo de filas (8 KB)    | Cuando varias filas están en la misma página de memoria. |
| Tabla (Table)      | Tabla completa           | Cuando se afecta una gran parte de los registros.        |
| Base de datos (DB) | Toda la base             | En operaciones globales como BACKUP o RESTORE.           |

■ **Jerarquía:** DB → Tabla → Página → Fila

El gestor usa bloqueos de intención (IS, IX) para saber qué nivel inferior está afectado.

### ♦ 6. Efectos de Simultaneidad (Problemas Clásicos)

Cuando no se controlan los bloqueos correctamente, surgen problemas de **concurrency**:

| Problema                                          | Descripción                                                              | Ejemplo                                                         |
|---------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Lectura sucia (Dirty Read)</b>                 | Una transacción lee datos que otra aún no confirmó (pueden revertirse).  | T1 modifica un valor, T2 lo lee antes del commit.               |
| <b>Lectura no repetible (Non-repeatable Read)</b> | Una transacción lee el mismo dato dos veces y obtiene valores distintos. | T1 lee, T2 modifica, T1 vuelve a leer.                          |
| <b>Lectura fantasma (Phantom Read)</b>            | Una transacción ve nuevas filas en una segunda lectura.                  | T1 lee todas las filas, T2 inserta una nueva, T1 vuelve a leer. |

#### ◆ 7. Niveles de Aislamiento de una Transacción

Definen **qué tan protegida está una transacción frente a cambios de otras**.

A mayor aislamiento, menor concurrency.

| Nivel                   | Lectura sucia | No repetible | Fantasma | Descripción                                                      |
|-------------------------|---------------|--------------|----------|------------------------------------------------------------------|
| <b>READ UNCOMMITTED</b> | ✗             | ✗            | ✗        | Máxima concurrency, pero sin control de consistencia.            |
| <b>READ COMMITTED</b>   | ✓             | ✗            | ✗        | Lecturas solo de datos confirmados. (Por defecto en SQL Server). |
| <b>REPEATABLE READ</b>  | ✓             | ✓            | ✗        | Nadie puede modificar los datos que leíste.                      |
| <b>SERIALIZABLE</b>     | ✓             | ✓            | ✓        | Máxima protección. Evita lecturas y escrituras simultáneas.      |
| <b>SNAPSHOT</b>         | ✓             | ✓            | ✓        | Usa versiones de datos (sin bloqueos físicos).                   |

✓ = Evita el problema

✗ = Puede ocurrir

#### ◆ 8. Interbloqueos (Deadlocks)

Ocurre cuando **dos transacciones se bloquean mutuamente** esperando un recurso que la otra posee.

■ Ejemplo:

- T1 bloquea la fila A y quiere la fila B.

- T2 bloquea la fila B y quiere la fila A.

⚠ Ninguna puede continuar → SQL Server detecta el *deadlock* y **cancela una de ellas**.

💡 Soluciones:

- Acceder siempre a los recursos en el mismo orden.
- Usar bloqueos más cortos o más específicos.
- Dividir operaciones grandes en transacciones más pequeñas.

## 🧠 Resumen Final

| Concepto                        | Qué controla                                 |
|---------------------------------|----------------------------------------------|
| <b>Bloqueos</b>                 | Acceso simultáneo a datos                    |
| <b>Modos (S, X, U, etc.)</b>    | Tipo de acceso permitido                     |
| <b>Compatibilidad</b>           | Qué bloqueos pueden coexistir                |
| <b>Granularidad</b>             | Nivel de detalle (fila, página, tabla)       |
| <b>Efectos de simultaneidad</b> | Problemas que surgen sin control de bloqueos |
| <b>Niveles de aislamiento</b>   | Qué tan protegida está la transacción        |
| <b>Interbloqueos</b>            | Cuando dos transacciones se esperan entre sí |

## ♦ Modos de Bloqueos en SQL Server

### ■ Modos de Bloqueos

- Compartido (S)
- Actualizar (U)
- Exclusivo (X)
- Intención (IS, IX, SIX)
- Esquema (Sch-S, Sch-M)
- Actualización Masiva (BU)
- Intervalo de Clases

Los **bloqueos** determinan qué operaciones puede realizar una transacción sobre un dato y cómo afecta eso a otras transacciones concurrentes.

■ En otras palabras:

Cada vez que una transacción lee o modifica datos, SQL Server coloca un “candado” que puede ser de distintos tipos

### 1. ■ Bloqueo Compartido (S – Shared Lock)

- Se coloca cuando **una transacción solo quiere leer** (SELECT).
- **Permite que otras transacciones también lean** el mismo dato.
- ❌ No permite modificaciones mientras dure el bloqueo.

■ Ejemplo:

```
BEGIN TRAN;
```

```
SELECT * FROM Empleados WHERE ID = 10; -- Bloqueo S
```

→ Otras transacciones pueden leer el mismo empleado, pero no modificarlo.

## 2. Bloqueo de Actualización (U – Update Lock)

- Es un **bloqueo intermedio** entre el compartido y el exclusivo.
- Se usa cuando una transacción **lee un dato con intención de modificarlo**.
- Evita que dos transacciones lean el mismo dato y ambas intenten actualizarlo luego (previene *deadlocks*).

 Ejemplo:

BEGIN TRAN;

SELECT \* FROM Empleados WITH (UPDLOCK) WHERE ID = 10;

UPDATE Empleados SET Sueldo = Sueldo + 1000 WHERE ID = 10;

→ Se coloca un bloqueo **U**, y cuando se actualiza el registro, pasa a **X (exclusivo)**.

## 3. Bloqueo Exclusivo (X – Exclusive Lock)

- Se usa para **operaciones de escritura** (INSERT, UPDATE, DELETE).
- Impide que **cualquier otra transacción lea o modifique** el dato bloqueado.
- Solo **una transacción** puede tenerlo sobre un mismo registro.

 Ejemplo:

BEGIN TRAN;

UPDATE Empleados SET Sueldo = Sueldo + 500 WHERE ID = 10; -- Bloqueo X

→ Mientras dure, nadie más puede leer ni modificar ese registro.

## 4. Bloqueos de Intención (IS, IX, SIX)

Son bloqueos que se aplican **a niveles superiores de la jerarquía (tabla, página)** para indicar que existen bloqueos más específicos dentro (por ejemplo, a nivel de fila).

| Tipo                                      | Significado                                                              | Uso                                                 |
|-------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------|
| <b>IS (Intent Shared)</b>                 | Intención de colocar bloqueos compartidos en registros específicos.      | Una transacción que lee filas de una tabla.         |
| <b>IX (Intent Exclusive)</b>              | Intención de colocar bloqueos exclusivos en registros.                   | Una transacción que actualiza o elimina filas.      |
| <b>SIX (Shared with Intent Exclusive)</b> | Combinación: lectura de toda la tabla + modificaciones en algunas filas. | Consultas que leen datos y actualizan parcialmente. |

 Ejemplo:

- Si bloqueas una fila de una tabla con X, SQL pone automáticamente un IX sobre la tabla entera para indicarlo.

## 5. Bloqueo de Esquema (Sch-S y Sch-M)

Controlan **cambios estructurales** en las tablas u objetos.

| Tipo | Significado | Cuándo se usa |
|------|-------------|---------------|
|------|-------------|---------------|

|                                    |                                                                                               |                                          |
|------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------|
| <b>Sch-S (Schema Stability)</b>    | Bloqueo de estabilidad. Permite consultas, pero evita cambios de estructura.                  | Durante un SELECT.                       |
| <b>Sch-M (Schema Modification)</b> | Bloqueo de modificación de esquema. Impide cualquier acceso mientras se cambia la estructura. | Durante un ALTER TABLE, DROP TABLE, etc. |

 Ejemplo:

ALTER TABLE Empleados ADD FechaIngreso DATE; -- Sch-M

→ Mientras dure este comando, nadie puede consultar la tabla.

## 6. Bloqueo de Actualización Masiva (BU – Bulk Update Lock)

- Se utiliza en operaciones de **carga masiva de datos** (BULK INSERT, bcp, etc.).
- Permite que **una sola conexión** inserte datos de forma masiva **sin conflictos**, mejorando el rendimiento.
- Evita que otros lean la tabla durante la operación.

 Ejemplo:

BULK INSERT Productos FROM 'C:\datos.csv' WITH (TABLOCK);

→ SQL Server bloquea la tabla completa con un **BU**.

## 7. Bloqueo de Intervalo de Clases (Key-Range Lock)

- Se usa en el nivel de aislamiento **SERIALIZABLE**.
- Bloquea **rangos de valores de clave** (no solo filas existentes).
- Impide la inserción de nuevas filas dentro del rango bloqueado (evita *lecturas fantasmas*).

 Ejemplo:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRAN;

SELECT \* FROM Empleados WHERE Sueldo BETWEEN 30000 AND 40000;

→ SQL bloquea ese rango de sueldos.

Nadie puede insertar nuevos empleados con sueldos dentro de ese rango hasta que se confirme la transacción.



## Resumen General

| Tipo de bloqueo | Código | Permite lectura          | Permite escritura     | Nivel típico  |
|-----------------|--------|--------------------------|-----------------------|---------------|
| Compartido      | S      | ✓                        | ✗                     | Fila / Página |
| Actualizar      | U      | ⚠ (solo una transacción) | ⚠ (se convierte en X) | Fila          |
| Exclusivo       | X      | ✗                        | ✗                     | Fila / Página |



| Intención | IS / IX / SIX    | Depende                                                                                    | Depende                                      | Tabla /<br>Página |
|-----------|------------------|--------------------------------------------------------------------------------------------|----------------------------------------------|-------------------|
| Esquema   | Sch-S /<br>Sch-M | Sch-S: <input checked="" type="checkbox"/> / Sch-M:<br><input checked="" type="checkbox"/> | Sch-M: <input checked="" type="checkbox"/>   | Objeto            |
| Masiva    | BU               | <input checked="" type="checkbox"/>                                                        | <input checked="" type="checkbox"/> (masivo) | Tabla             |
| Intervalo | Key-Range        | <input checked="" type="checkbox"/>                                                        | <input checked="" type="checkbox"/>          | Índice /<br>Rango |

## Compatibilidad de Bloqueos

| Modo Solicitado                             | Modo Otorgado |    |    |    |     |    |
|---------------------------------------------|---------------|----|----|----|-----|----|
|                                             | IS            | S  | U  | IX | SIX | X  |
| Intención Compartida (IS)                   | Sí            | Sí | Sí | Sí | Sí  | No |
| Compartido (S)                              | Sí            | Sí | Sí | No | No  | No |
| Actualizar (U)                              | Sí            | Sí | No | No | No  | No |
| Intención Exclusivo (IX)                    | Sí            | No | No | Sí | No  | No |
| Compartido con Intención<br>Exclusivo (SIX) | Sí            | No | No | No | No  | No |
| Exclusivo (X)                               | No            | No | No | No | No  | No |

## Compatibilidad de Bloqueos

### ¿Qué significa “compatibilidad”?

Cuando **dos transacciones acceden al mismo dato**, SQL Server verifica si los **modos de bloqueo** que usan son **compatibles entre sí**.

- Si son **compatibles (Sí)** → ambas pueden continuar simultáneamente.
- Si son **incompatibles (No)** → la segunda **espera** a que la primera libere el bloqueo (o falla si se supera el tiempo de espera).

### En resumen:

La compatibilidad determina si dos tipos de acceso (lectura o escritura) pueden ocurrir al mismo tiempo sobre el mismo recurso.

### Cómo leer la tabla

- Filas** → modo de bloqueo que **quiere solicitar** una transacción.
- Columnas** → modo de bloqueo que **ya fue otorgado** a otra transacción.
- En la intersección, “**Sí**” = compatible, “**No**” = incompatible.

### Interpretación de la tabla

| Modo Solicitado                    | Modo Otorgado                       | ¿Compatible?                                                         |
|------------------------------------|-------------------------------------|----------------------------------------------------------------------|
| <b>IS</b> con casi todos (menos X) | <input checked="" type="checkbox"/> | Varias transacciones pueden indicar intención de leer sin conflicto. |

|                            |   |                                                                               |
|----------------------------|---|-------------------------------------------------------------------------------|
| <b>S con S o IS</b>        | ✓ | Varias transacciones pueden leer el mismo dato.                               |
| <b>S con U, IX, SIX, X</b> | ✗ | No se puede leer mientras alguien lo modifica.                                |
| <b>U con S o IS</b>        | ✓ | Puede leer si nadie más actualiza.                                            |
| <b>U con U o X</b>         | ✗ | No se puede actualizar simultáneamente.                                       |
| <b>IX con IS o IX</b>      | ✓ | Varias transacciones pueden tener intención de escribir en diferentes partes. |
| <b>IX con S, SIX, X</b>    | ✗ | Incompatible si hay lectura o escritura activa.                               |
| <b>SIX con IS</b>          | ✓ | Permite lecturas globales y escrituras puntuales.                             |
| <b>SIX con otros</b>       | ✗ | Bloquea lecturas y escrituras simultáneas.                                    |
| <b>X con cualquiera</b>    | ✗ | Exclusivo total. Solo una transacción puede tenerlo.                          |

### ◆ Ejemplos Prácticos

#### ■ Ejemplo 1 — Bloqueos compatibles

- T1 ejecuta:
- `SELECT * FROM Empleados;` -- genera bloqueo S
- T2 ejecuta:
- `SELECT * FROM Empleados;` -- genera otro bloqueo S

✓ Ambos pueden leer al mismo tiempo (S con S = compatible).

#### ■ Ejemplo 2 — Bloqueos incompatibles

- T1 ejecuta:
- `UPDATE Empleados SET Sueldo = Sueldo + 1000 WHERE ID = 10;` -- genera bloqueo X
- T2 ejecuta al mismo tiempo:
- `SELECT * FROM Empleados WHERE ID = 10;` -- genera bloqueo S

✗ Incompatibles (X con S = No).

→ La segunda transacción **espera** a que la primera haga COMMIT o ROLLBACK.

#### ■ Ejemplo 3 — Intenciones de bloqueo

- T1 actualiza una fila → SQL pone **IX** sobre la tabla.
- T2 intenta leer toda la tabla (S).

✗ Incompatibles (S con IX = No).

→ T2 debe esperar hasta que T1 termine.

### ◆ Resumen de comportamiento

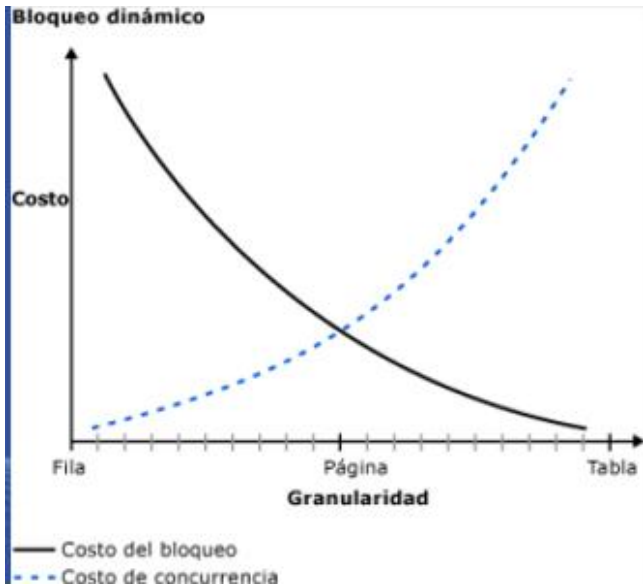
| Combinación  | Resultado | Descripción                           |
|--------------|-----------|---------------------------------------|
| <b>S + S</b> | ✓         | Lecturas simultáneas permitidas.      |
| <b>S + X</b> | ✗         | No se puede leer mientras se escribe. |

|                       |   |                                               |
|-----------------------|---|-----------------------------------------------|
| <b>U + U</b>          | ✗ | Previene deadlocks de actualización.          |
| <b>IX + IX</b>        | ✓ | Modificaciones diferentes en filas distintas. |
| <b>SIX + IS</b>       | ✓ | Lectura global + actualización parcial.       |
| <b>X + Cualquiera</b> | ✗ | Exclusivo total. Nadie más accede.            |

### 🧠 Conclusión clave

La compatibilidad de bloqueos es lo que determina:

- Qué transacciones pueden ejecutarse al mismo tiempo.
- Cuándo SQL Server debe **esperar o bloquear** un proceso.
- Cuándo pueden aparecer **problemas de rendimiento o deadlocks**.



### ⚙️ Bloqueo Dinámico

#### ◆ ¿Qué es?

El **bloqueo dinámico** es una estrategia que **SQL Server usa para decidir automáticamente el nivel de bloqueo más eficiente** (fila, página o tabla) según el tamaño de la operación y la concurrencia del sistema.

■ En otras palabras:

👉 Si estás modificando pocas filas, bloquea **solo esas filas**.

👉 Pero si modificás miles, bloquea **toda la tabla** para reducir el costo total de mantener muchos bloqueos individuales.

#### ◆ Concepto de Granularidad

La **granularidad** indica el nivel de detalle del bloqueo:

| Nivel             | Qué bloquea    | Ventajas                                                      | Desventajas                                    |
|-------------------|----------------|---------------------------------------------------------------|------------------------------------------------|
| <b>Fila (Row)</b> | Una sola tupla | Alta concurrencia (varios usuarios pueden trabajar a la vez). | Alto costo: se deben mantener muchos bloqueos. |

|                      |                       |                                              |                                               |
|----------------------|-----------------------|----------------------------------------------|-----------------------------------------------|
| <b>Página (Page)</b> | Grupo de filas (8 KB) | Equilibrio entre rendimiento y concurrencia. | Puede bloquear filas que no interesan.        |
| <b>Tabla (Table)</b> | Toda la tabla         | Bajo costo administrativo.                   | Baja concurrencia: bloquea a todos los demás. |

#### ◆ Cómo funciona el bloqueo dinámico

SQL Server **empieza con bloqueos finos (fila o página)** y, si la operación crece, los **“escala”** automáticamente a bloqueos más grandes (página o tabla).

Este proceso se llama **escalamiento de bloqueos (lock escalation)**.

■ Ejemplo:

BEGIN TRAN;

UPDATE Ventas SET Total = Total \* 1.05; -- Aumenta el 5%

- Si se modifican pocas filas → bloqueos a nivel de **fila**.
- Si se modifican miles → SQL Server “sube” a un bloqueo de **tabla completa** (para ahorrar memoria y tiempo).

#### ◆ Interpretación del gráfico

En el eje del gráfico:

- **Eje X → Granularidad (Fila → Página → Tabla)**
- **Eje Y → Costo**

Hay **dos curvas**:

1. **Línea continua (negra) → Costo del bloqueo**
  - Cuantos más bloqueos pequeños (por fila) se usan, **mayor es el costo de administración**.
  - A medida que se agrupan (página o tabla), ese costo **disminuye**.
2. **Línea punteada (azul) → Costo de concurrencia**
  - Si bloqueás gran parte de la tabla, **la concurrencia baja** (otros usuarios deben esperar).
  - Por eso, el costo de concurrencia **aumenta** con bloqueos más grandes.

📍 **El punto de cruce** representa el equilibrio ideal entre:

- Mantener pocos bloqueos (rendimiento)
- Permitir acceso concurrente (usuarios simultáneos)

Ese equilibrio es lo que busca el **bloqueo dinámico** automáticamente.

#### ◆ Beneficios

- ✓ **Eficiencia automática** → el sistema decide el mejor nivel sin intervención manual.
- ✓ **Mejor rendimiento global** → menos sobrecarga cuando hay muchas filas.
- ✓ **Menos deadlocks** → al reducir el número de bloqueos individuales.
- ✓ **Alta concurrencia** cuando solo se modifican pocas filas.

#### ◆ Riesgos o limitaciones

⚠ Si SQL Server eleva demasiado rápido a nivel de **tabla**, puede generar esperas innecesarias.

⚙ Podés controlar o desactivar la escalada automática con:

ALTER TABLE nombre\_tabla SET (LOCK\_ESCALATION = AUTO | TABLE | DISABLE);

## 🧠 Resumen final

| Concepto                    | Explicación                                                                  |
|-----------------------------|------------------------------------------------------------------------------|
| <b>Bloqueo Dinámico</b>     | Ajusta automáticamente el nivel del bloqueo según el tamaño de la operación. |
| <b>Objetivo</b>             | Equilibrar costo de bloqueo y concurrencia.                                  |
| <b>Pequeñas operaciones</b> | Bloqueos por fila/página.                                                    |
| <b>Grandes operaciones</b>  | Escala a bloqueo de tabla.                                                   |
| <b>Ventaja principal</b>    | Mejor rendimiento y menor sobrecarga.                                        |

- Personalizar el tiempo de espera de bloqueo
  - SET LOCK\_TIMEOUT período
  - SELECT @@lock\_timeout

## 🕒 Personalizar el tiempo de espera de bloqueo

### ♦ ¿Qué significa?

Por defecto, cuando una transacción intenta acceder a un recurso (una fila, tabla, etc.) que **ya está bloqueado por otra transacción**, **SQL Server espera indefinidamente** hasta que el bloqueo se libere.

■ Pero con SET LOCK\_TIMEOUT, vos podés **definir un límite máximo de espera (en milisegundos)**.

Si se supera ese tiempo y el recurso sigue bloqueado, la instrucción **falla automáticamente** y se genera un error.

### ♦ Sintaxis

SET LOCK\_TIMEOUT <milisegundos>;

📍 Donde <milisegundos> es el tiempo que la transacción esperará.

### ♦ Ejemplo

SET LOCK\_TIMEOUT 5000; -- Espera 5 segundos (5000 milisegundos)

BEGIN TRAN;

UPDATE Empleados SET Sueldo = Sueldo + 1000 WHERE ID = 1;

COMMIT TRAN;

→ Si la fila del empleado con ID = 1 está bloqueada por otra transacción:

- SQL Server esperará **hasta 5 segundos**.
- Si en ese tiempo no se libera el bloqueo → la instrucción **falla con error 1222** (“**Lock request time out period exceeded**”).

#### ◆ Consultar el valor actual

Podés saber cuál es el tiempo de espera actual con:

```
SELECT @@LOCK_TIMEOUT;
```

■ Por defecto, devuelve -1, lo que significa que **espera indefinidamente**.

#### ◆ Ejemplo completo de control

```
SET LOCK_TIMEOUT 3000; -- 3 segundos
```

```
BEGIN TRY
```

```
    UPDATE Ventas SET Total = Total * 1.1 WHERE ID = 10;
```

```
    COMMIT TRANSACTION;
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'Error: Tiempo de espera de bloqueo excedido.';
```

```
    ROLLBACK TRANSACTION;
```

```
END CATCH
```

✅ De esta forma, si hay un bloqueo prolongado, el sistema no se queda esperando indefinidamente y **se puede manejar el error controladamente**.

#### ◆ Valores posibles

| Valor | Significado                                                 |
|-------|-------------------------------------------------------------|
| -1    | Espera indefinidamente (por defecto).                       |
| 0     | No espera: si el recurso está bloqueado, falla al instante. |
| >0    | Espera la cantidad de milisegundos indicada.                |

#### ◆ Casos de uso comunes

- En **sistemas con alta concurrencia**, para evitar que una transacción quede colgada.
- En **aplicaciones web**, donde los usuarios no deben esperar más de unos segundos.
- En **bloques TRY/CATCH**, para capturar y manejar los errores de espera.

#### 🧠 Resumen

| Comando               | Función                                               |
|-----------------------|-------------------------------------------------------|
| SET LOCK_TIMEOUT n    | Define el tiempo máximo de espera (en milisegundos).  |
| SELECT @@LOCK_TIMEOUT | Muestra el valor actual del tiempo de espera.         |
| Valor por defecto     | -1 (espera indefinida).                               |
| Error si se supera    | 1222 – <i>Lock request time out period exceeded</i> . |

#### 🧱 Granularidad y Jerarquías de Bloqueos

##### ◆ ¿Qué es la *granularidad*?

La **granularidad** indica el **nivel de detalle** en el que el sistema aplica un bloqueo. Puede ser tan pequeño como una **fila individual**, o tan grande como **toda la base de datos**.

■ En otras palabras:

👉 Cuanto más “grosso” es el nivel, más datos quedan bloqueados, pero menos recursos consume el sistema.

◆ **¿Qué es la jerarquía de bloqueos?**

SQL Server mantiene una **estructura jerárquica** que organiza los bloqueos de menor a mayor nivel.

Esto le permite **administrar eficientemente los recursos** y evitar conflictos entre niveles.

📊 **Jerarquía de niveles de bloqueo (de menor a mayor)**

| Nivel                        | Descripción                                                                                                              | Qué bloquea                     |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <b>RID (Row Identifier)</b>  | Es el nivel más bajo. Bloquea una <b>fila específica</b> en una tabla sin índice.                                        | 1 registro                      |
| <b>KEY</b>                   | Bloquea una <b>clave de índice</b> en tablas indexadas. Se usa cuando la tabla tiene un índice clustered o nonclustered. | 1 valor de índice               |
| <b>PAGE</b>                  | Bloquea una <b>página de datos o de índice</b> (8 KB). Puede contener varias filas.                                      | 8 KB de datos                   |
| <b>EXTENT</b>                | Bloquea un <b>conjunto de 8 páginas contiguas</b> (64 KB).                                                               | Grupo de páginas                |
| <b>HOBT (Heap Or B-Tree)</b> | Bloquea un <b>árbol de índices (B-Tree)</b> o una <b>tabla heap (sin índice clustered)</b> .                             | Estructura interna completa     |
| <b>TABLE</b>                 | Bloquea <b>toda una tabla</b> .                                                                                          | Todos los registros de la tabla |
| <b>FILE</b>                  | Bloquea un <b>archivo físico</b> dentro de la base de datos (archivos .mdf, .ndf, etc.).                                 | Archivo completo                |
| <b>APPLICATION</b>           | Bloqueo definido por el <b>usuario o aplicación</b> para coordinar procesos externos.                                    | Recurso lógico                  |
| <b>METADATA</b>              | Bloquea <b>información del sistema</b> (por ejemplo, definiciones de tablas o índices).                                  | Catálogo del sistema            |
| <b>ALLOCATION_UNIT</b>       | Bloquea <b>áreas de espacio asignadas</b> (grupos de páginas). Se usa en operaciones internas.                           | Espacio de almacenamiento       |



|                 |                                                                                                |               |
|-----------------|------------------------------------------------------------------------------------------------|---------------|
| <b>DATABASE</b> | Nivel más alto. Bloquea <b>toda la base de datos</b> , impidiendo cualquier acceso simultáneo. | Base completa |
|-----------------|------------------------------------------------------------------------------------------------|---------------|

#### ◆ Cómo funciona la jerarquía

SQL Server utiliza **bloqueos de intención** (IS, IX, SIX) para **indicar en niveles superiores** que hay bloqueos activos en niveles inferiores.

#### ■ Ejemplo:

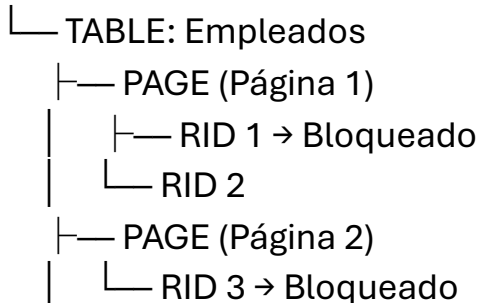
- Si bloqueás una **fila (RID)** con X,

SQL Server coloca automáticamente un **IX (Intención Exclusiva)** en la tabla.

Así, el sistema sabe que dentro de esa tabla hay registros bloqueados, y evita que otro proceso intente bloquear toda la tabla sin esperar.

#### ◆ Ejemplo visual simplificado

DATABASE



→ Si se bloquean filas específicas (RID), SQL Server:

- Coloca bloqueos **IS** en PAGE y TABLE.
- Avisa que “adentro” de esa tabla hay bloqueos activos.

#### ◆ Ventajas de esta jerarquía

- ✓ Permite **bloquear solo lo necesario**, evitando bloquear toda la tabla si no hace falta.
- ✓ Mejora la **concurrency** (más usuarios pueden trabajar a la vez).
- ✓ Reduce la **sobrecarga administrativa** cuando el sistema escala bloqueos grandes.
- ✓ Facilita el **control de compatibilidad** entre niveles.

#### ◆ Resumen de uso típico

| Nivel                         | Uso común                                                          |
|-------------------------------|--------------------------------------------------------------------|
| <b>RID / KEY</b>              | Lectura o actualización de registros individuales.                 |
| <b>PAGE / EXTENT</b>          | Operaciones moderadas o que afectan grupos de filas.               |
| <b>TABLE / DATABASE</b>       | Operaciones masivas o estructurales (DELETE masivo, BACKUP, etc.). |
| <b>METADATA / APPLICATION</b> | Procesos internos o definidos por el usuario.                      |

#### Resumen final

| Concepto              | Explicación                                                  |
|-----------------------|--------------------------------------------------------------|
| Granularidad          | Nivel de detalle del bloqueo (fila, página, tabla, etc.).    |
| Jerarquía             | Estructura que organiza bloqueos de menor a mayor alcance.   |
| Bloqueos de intención | Avisan a niveles superiores que hay bloqueos internos.       |
| Objetivo              | Mantener eficiencia y consistencia en el acceso concurrente. |

### ⚙️ Efectos de Simultaneidad

Cuando varias transacciones acceden a los mismos datos **al mismo tiempo**, pueden producirse inconsistencias.

Los principales **efectos de simultaneidad** son:

| Efecto                                                           | Descripción                                                                                                                       | Ejemplo                                                                                             |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Actualizaciones Perdidas</b>                                  | Dos transacciones modifican el mismo dato; la última sobrescribe el cambio de la otra.                                            | T1 y T2 actualizan el mismo salario; T1 guarda 4000, T2 guarda 3500 → el cambio de T1 se pierde.    |
| <b>Dependencia No Confirmada (Lectura No Confirmada o Sucia)</b> | Una transacción lee datos modificados por otra que aún no hizo COMMIT. Si la otra hace ROLLBACK, los datos quedan inconsistentes. | T1 cambia un saldo, T2 lo lee, pero luego T1 deshace la transacción.                                |
| <b>Análisis Incoherente (Lectura No Repetible)</b>               | Una transacción lee el mismo dato dos veces, pero entre ambas lecturas otra transacción lo modifica.                              | T1 consulta el stock, T2 actualiza el stock antes de que T1 vuelva a leerlo.                        |
| <b>Lecturas Ficticias (Lecturas Fantasma)</b>                    | Una transacción repite una consulta y obtiene filas nuevas o eliminadas por otra transacción.                                     | T1 cuenta las filas de una tabla, T2 inserta una nueva, y T1 vuelve a contar → el resultado cambia. |

■ Estos efectos son los que los **niveles de aislamiento** intentan controlar.

### 🏠 Nivel de Aislamiento de una Transacción

El **nivel de aislamiento** define **cuánto puede una transacción ver los cambios no confirmados** de otras.

Se establece con:

SET TRANSACTION ISOLATION LEVEL <nivel>;

◆ **Niveles de Aislamiento (de menor a mayor seguridad)**

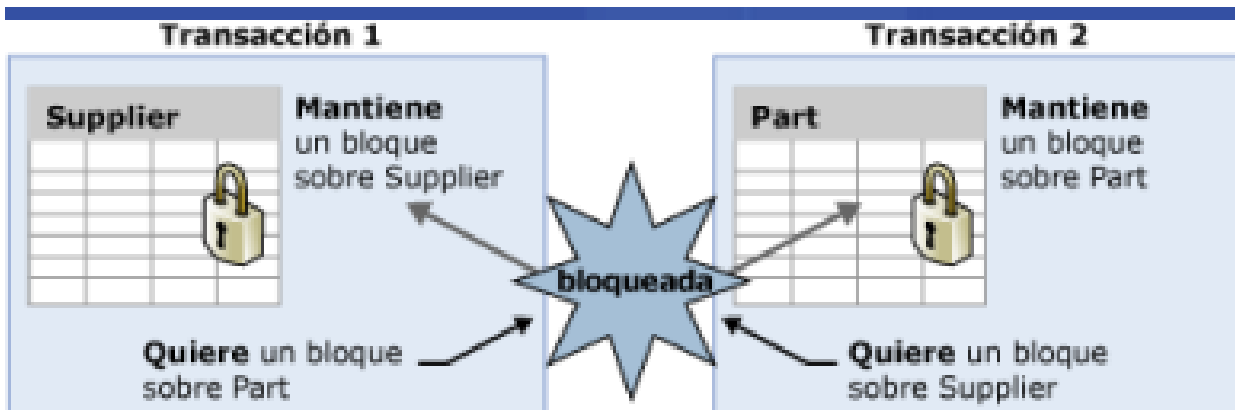
| Nivel                                        | Descripción                                                                | Permite                                                                     |
|----------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>READ UNCOMMITTED</b>                      | Lee datos sin confirmar (sin bloqueos de lectura).                         | Lecturas sucias, no repetibles, fantasmas.                                  |
| <b>READ COMMITTED</b> ( <i>por defecto</i> ) | Solo lee datos confirmados.                                                | Evita lecturas sucias, pero puede haber lecturas no repetibles y fantasmas. |
| <b>READ_COMMITTED_SNAPSHOT</b>               | Usa versiones en memoria (row versioning).                                 | Evita bloqueos; lecturas siempre consistentes.                              |
| <b>REPEATABLE READ</b>                       | Mantiene bloqueos sobre las filas leídas.                                  | Evita lecturas sucias y no repetibles, pero no fantasmas.                   |
| <b>SNAPSHOT</b>                              | Cada transacción ve una “foto” del estado de la BD al iniciar.             | Evita todos los efectos, sin bloquear lecturas.                             |
| <b>SERIALIZABLE</b>                          | Máximo aislamiento; ejecuta como si las transacciones fueran secuenciales. | Evita todos los efectos, pero reduce la concurrencia.                       |

◆ **Ejemplo práctico**

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRAN;
SELECT * FROM Productos WHERE Categoria = 'Mates';
-- otra transacción no puede modificar ni borrar estas filas
COMMIT TRAN;
```

■ Así garantizas que los datos no cambien durante la transacción.

🔒 **Interbloqueos (Deadlocks)**



### ◆ ¿Qué es un Deadlock?

Ocurre cuando **dos transacciones se bloquean mutuamente**, cada una esperando que la otra libere un recurso.

 Ejemplo del gráfico:

- **Transacción 1:** tiene bloqueado Supplier, pero necesita Part.
  - **Transacción 2:** tiene bloqueado Part, pero necesita Supplier.
-  Ninguna puede continuar → SQL Server **detecta el deadlock y cancela una** (la “víctima”).

### ◆ Cómo minimizar los Deadlocks

1.  **Acceder a los objetos siempre en el mismo orden.**  
Ej: si en todas las transacciones primero bloqueás Cliente y luego Factura, no habrá conflicto.
2.  **Mantener transacciones cortas y rápidas.**  
Cuanto más duran, más tiempo bloquean recursos.
3.  **Usar niveles de aislamiento inferiores** si la consistencia lo permite (por ejemplo READ COMMITTED).
4.  **Usar versiones de filas:**  
Activar READ\_COMMITTED\_SNAPSHOT o SNAPSHOT reduce la necesidad de bloqueos físicos.

ALTER DATABASE MiBase

SET READ\_COMMITTED\_SNAPSHOT ON;

### Monitoreo de Transacciones y Bloqueos

SQL Server provee vistas del sistema y procedimientos para diagnosticar bloqueos y deadlocks:

#### ◆ Vistas de catálogo dinámico (DMVs)

| Vista                             | Descripción                                  |
|-----------------------------------|----------------------------------------------|
| sys.dm_tran_locks                 | Muestra todos los bloqueos activos.          |
| sys.dm_tran_database_transactions | Detalla el estado de las transacciones.      |
| sys.dm_tran_active_transactions   | Lista las transacciones actualmente activas. |

#### ◆ Procedimientos almacenados útiles

##### Procedimiento Descripción

sp\_who Muestra sesiones activas y bloqueos.

sp\_lock Lista los recursos bloqueados y por quién.

 También podés usar Activity Monitor en SQL Server Management Studio para una vista visual.

### Resumen General

| Concepto                 | Qué controla                                       | Nivel de riesgo |
|--------------------------|----------------------------------------------------|-----------------|
| Efectos de simultaneidad | Problemas entre transacciones concurrentes         | Alto            |
| Nivel de aislamiento     | Cuánto se protegen los datos frente a esos efectos | Variable        |
| Deadlocks                | Bloqueos mutuos entre transacciones                | Medio           |
| DMVs / SPs               | Monitoreo y diagnóstico                            | Preventivo      |

## Unidad 3: Procedimientos Almacenados y Triggers

### Definición de Stored Procedure

Un **Stored Procedure (SP)** es un **conjunto de sentencias SQL precompiladas** que se almacenan directamente en el **servidor de base de datos** (por ejemplo, SQL Server). Su objetivo es **automatizar operaciones repetitivas, encapsular lógica y mejorar el rendimiento y la seguridad**.

#### ◆ 1. Colección de sentencias SQL

Un SP puede contener **una o varias instrucciones SQL**:

- Consultas (SELECT)
- Inserciones (INSERT)
- Actualizaciones (UPDATE)
- Eliminaciones (DELETE)
- Sentencias de control (IF, WHILE, BEGIN...END, etc.)
- Incluso **llamadas a otros procedimientos o funciones**.

#### Ejemplo:

```
CREATE PROCEDURE dbo.MostrarClientes
AS
BEGIN
    SELECT Nombre, Ciudad FROM Clientes;
END;
```

Este SP guarda una consulta que, al ejecutarla, devuelve todos los clientes.

#### ◆ 2. Aceptar parámetros

Los SP pueden **recibir valores de entrada** (parámetros) para trabajar con distintos datos sin tener que reescribir el código.

#### Ejemplo:

```
CREATE PROCEDURE dbo.ClientesPorCiudad
```

```
@Ciudad NVARCHAR(50)
AS
BEGIN
    SELECT Nombre, Telefono
    FROM Clientes
    WHERE Ciudad = @Ciudad;
END;
```

👉 Cuando se ejecuta:

```
EXEC dbo.ClientesPorCiudad @Ciudad = 'Salta';
```

Esto permite reutilizar el mismo SP con diferentes valores.

### ◆ 3. Devolver valores de estado

Un SP puede devolver:

- **Resultados de consultas** (tablas).
- **Parámetros de salida** (OUTPUT).
- Un **código de estado** (RETURN <número>).

■ Ejemplo:

```
CREATE PROCEDURE dbo.Sumar
    @A INT, @B INT, @Resultado INT OUTPUT
AS
BEGIN
    SET @Resultado = @A + @B;
    RETURN 0; -- 0 significa "éxito"
END;
```

Y se ejecuta así:

```
DECLARE @Suma INT;
EXEC dbo.Sumar @A = 5, @B = 3, @Resultado = @Suma OUTPUT;
SELECT @Suma AS Total;
```

### ◆ 4. Permisos de creación

Solo pueden crear SP los usuarios con **permisos adecuados**, por ejemplo:

```
GRANT CREATE PROCEDURE TO Usuario1;
```

Y para ejecutarlos:

```
GRANT EXECUTE ON dbo.MiProcedimiento TO Usuario1;
```

Esto **mejora la seguridad**, ya que los usuarios pueden ejecutar lógica del servidor **sin tener acceso directo a las tablas**.

### 🔍 Resumen general

| Característica           | Descripción                                                  |
|--------------------------|--------------------------------------------------------------|
| <b>Definición</b>        | Bloque de código SQL almacenado en el servidor.              |
| <b>Función principal</b> | Ejecutar lógica SQL de manera rápida, reutilizable y segura. |

|                   |                                                          |
|-------------------|----------------------------------------------------------|
| <b>Parámetros</b> | Permiten recibir valores para hacer el código flexible.  |
| <b>Salida</b>     | Devuelve resultados, mensajes o códigos de estado.       |
| <b>Seguridad</b>  | Se controla quién puede crear y ejecutar procedimientos. |

### **Guía para su creación (Buenas prácticas)**

#### ◆ **1. Especificar el esquema de los objetos**

Siempre que crees o llames a un procedimiento, **indica su esquema**, normalmente dbo (Database Owner).

 Ejemplo:

```
CREATE PROCEDURE dbo.ListarClientes
AS
SELECT * FROM dbo.Clientes;
GO
EXEC dbo.ListarClientes;
```

#### ✓ **Ventajas:**

- Mejora el **rendimiento** porque SQL Server no necesita buscar en otros esquemas.
- Evita **errores de ambigüedad** (si hay procedimientos con el mismo nombre en distintos esquemas).
- Refuerza la **seguridad y organización** del código.

#### ◆ **2. Una sola tarea**

Cada procedimiento debe tener **una única responsabilidad** o tarea principal.

Por ejemplo: un SP para registrar un cliente, otro para eliminarlo, otro para consultar.

 Ejemplo correcto:

```
CREATE PROCEDURE dbo.InsertarCliente
@Nombre NVARCHAR(100),
@email NVARCHAR(100)
AS
INSERT INTO Clientes (Nombre, Email)
VALUES (@Nombre, @Email);
```

 Ejemplo incorrecto:

-- Inserta, actualiza y borra clientes en el mismo SP → confuso, difícil de mantener

#### ✓ **Ventajas:**

- Código más **modular y reutilizable**.
- Más **fácil de probar y mantener**.
- Menor riesgo de errores y bloqueos.

#### ◆ **3. Crear y probar en el servidor antes de usarlos en las aplicaciones**

Antes de conectar el SP a una app (por ejemplo, un sistema web o escritorio), **próballo directamente en SQL Server**.



■ Ejemplo de prueba:

```
EXEC dbo.InsertarCliente @Nombre = 'Braian', @Email = 'braian@mail.com';
```

✓ **Motivo:**

- Verificás que **funcione correctamente**, que **compile sin errores**, y que los **resultados sean correctos** antes de exponerlo al código de la aplicación.
- Permite detectar **problemas de permisos**, de sintaxis o de rendimiento a tiempo.

◆ **4. Evitar el prefijo sp\_**

Nunca nombres tus SP con el prefijo sp\_, como por ejemplo sp\_InsertarCliente.

■ Ejemplo incorrecto:

```
CREATE PROCEDURE sp_InsertarCliente AS ...
```

⚠ **Motivo técnico:**

- SQL Server **busca primero** los SP del sistema (master) cuando ve ese prefijo.
- Esto **ralentiza la ejecución** porque el motor **revisa otras bases antes** de encontrar el SP correcto.
- Además, puede generar **conflictos de nombre** con procedimientos del sistema (por ejemplo, sp\_help).

✓ Ejemplo correcto:

```
CREATE PROCEDURE dbo.InsertarCliente AS ...
```

◆ **5. Valores establecidos en las conexiones**

Muchos valores de configuración (por ejemplo SET ANSI\_NULLS, SET QUOTED\_IDENTIFIER, SET XACT\_ABORT) **afectan la ejecución** del SP.

Por eso conviene **definir explícitamente dentro del SP** los valores importantes, en lugar de depender de la conexión del cliente.

■ Ejemplo:

```
CREATE PROCEDURE dbo.EliminarCliente
```

```
@Id INT
```

```
AS
```

```
BEGIN
```

```
SET NOCOUNT ON;      -- evita mensajes de "X filas afectadas"
```

```
SET XACT_ABORT ON;    -- asegura rollback automático si hay error
```

```
DELETE FROM Clientes WHERE Id = @Id;
```

```
END;
```

✓ **Ventajas:**

- El SP **se comporta igual** sin importar desde qué aplicación o usuario se ejecute.
- Mejora la **consistencia** y **fiabilidad** del código.

🧠 **Resumen general**

| Recomendación                    | Propósito                                               |
|----------------------------------|---------------------------------------------------------|
| Especificar esquema              | Evitar búsquedas innecesarias y errores de ambigüedad.  |
| Una sola tarea                   | Facilita mantenimiento y depuración.                    |
| Probar antes de usar             | Asegura que funcione correctamente antes de integrarlo. |
| Evitar sp_                       | Evita conflictos y mejora el rendimiento.               |
| Configurar valores dentro del SP | Garantiza comportamiento estable.                       |

### 💡 Ventajas de los Stored Procedures

#### ♦ 1. Se registran en el servidor

Los SP se **guardan directamente en el motor de la base de datos**, no en la aplicación.

■ Ejemplo:

```
CREATE PROCEDURE dbo.ListarProductos AS
SELECT * FROM Productos;
```

#### ✅ Ventajas:

- No hace falta reenviar el código SQL desde la app cada vez → **menos tráfico**.
- El **plan de ejecución se guarda en caché**, así que SQL Server no recompila el código cada vez → **más rápido**.
- Si cambiás la lógica dentro del SP, no necesitás modificar la aplicación → **fácil mantenimiento**.

#### ♦ 2. Seguridad – Cadena de Propiedad (Ownership Chain)

El SP **se ejecuta con los permisos del propietario** (por ejemplo dbo), no necesariamente del usuario que lo ejecuta.

Esto se llama **cadena de propiedad** y evita que el usuario acceda directamente a las tablas.

■ Ejemplo:

```
GRANT EXECUTE ON dbo.ListarClientes TO UsuarioApp;
```

-- El usuario puede ejecutar el SP, pero NO hacer SELECT directo sobre Clientes

#### ✅ Ventajas:

- Aísla la base de datos de accesos directos.
- Evita exponer tablas sensibles.
- Simplifica la administración de permisos.

#### ♦ 3. Encapsulan lógica de aplicación

Los SP permiten poner **reglas de negocio dentro de la base de datos**, por ejemplo cálculos, validaciones o verificaciones.

■ Ejemplo:

```
CREATE PROCEDURE dbo.RegistrarVenta
    @Clienteld INT,
    @Monto DECIMAL(10,2)
AS
BEGIN
    IF @Monto <= 0
        THROW 50001, 'El monto debe ser mayor a cero', 1;
    INSERT INTO Ventas (Clienteld, Fecha, Monto)
    VALUES (@Clienteld, GETDATE(), @Monto);
END;
```

#### ✓ **Ventajas:**

- Centraliza la lógica → todas las aplicaciones usan las mismas reglas.
- Evita inconsistencias.
- Facilita el mantenimiento y depuración.

#### ◆ **4. Protegen contra inyección de código SQL**

La **inyección SQL** ocurre cuando el usuario introduce código malicioso en un campo o variable.

Los SP ayudan a **prevenirlo** porque usan **parámetros tipados** en lugar de concatenar cadenas.

■ Ejemplo inseguro (mala práctica):

-- Vulnerable

```
EXEC('SELECT * FROM Clientes WHERE Nombre = ' + @nombre + ');
```

■ Ejemplo seguro:

```
CREATE PROCEDURE dbo.BuscarCliente
```

```
    @Nombre NVARCHAR(50)
```

```
AS
```

```
    SELECT * FROM Clientes WHERE Nombre = @Nombre;
```

#### ✓ **Ventajas:**

- Evita ataques del tipo '; DROP TABLE Clientes; --
- Mantiene la base de datos protegida.
- Mejora la integridad de los datos.

#### ◆ **5. Programación modular**

Permite crear **módulos reutilizables** que pueden ser llamados desde otros SP, triggers o aplicaciones.

■ Ejemplo:

```
CREATE PROCEDURE dbo.CalcularImpuesto
```

```
    @Monto DECIMAL(10,2), @Total DECIMAL(10,2) OUTPUT
```

```
AS
```

```
SET @Total = @Monto * 1.21;
```

Luego puede ser usado dentro de otro SP:

```
CREATE PROCEDURE dbo.Facturar
```

```
@Monto DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
DECLARE @Total DECIMAL(10,2);
```

```
EXEC dbo.CalcularImpuesto @Monto, @Total OUTPUT;
```

```
INSERT INTO Facturas (MontoTotal) VALUES (@Total);
```

```
END;
```

#### ✓ Ventajas:

- Divide el código en piezas pequeñas y reutilizables.
- Facilita la lectura, el mantenimiento y la prueba del sistema.

#### ◆ 6. Reducen el tráfico de red

Como el código SQL **se ejecuta directamente en el servidor**, la aplicación solo envía:

```
EXEC dbo.RegistrarVenta @Clienteld = 5, @Monto = 1000;
```

En lugar de mandar **todas las consultas** o instrucciones desde el cliente.

#### ✓ Ventajas:

- Menos datos enviados por la red.
- Mejor rendimiento general, especialmente en sistemas distribuidos o con muchas conexiones simultáneas.

#### 🧠 Resumen

| Ventaja                                | Explicación                              | Beneficio clave   |
|----------------------------------------|------------------------------------------|-------------------|
| <b>Registrados en el servidor</b>      | Se ejecutan directamente en el motor SQL | Rendimiento       |
| <b>Seguridad / Cadena de propiedad</b> | Ejecutan con permisos del dueño          | Control de acceso |
| <b>Encapsulan lógica</b>               | Centralizan reglas de negocio            | Mantenimiento     |
| <b>Protegen contra inyección SQL</b>   | Usan parámetros, no concatenación        | Seguridad         |
| <b>Modularidad</b>                     | Reutilizables en distintos procesos      | Organización      |
| <b>Menos tráfico de red</b>            | Solo se envían parámetros                | Eficiencia        |

#### 🔪 Inyección de Código SQL

##### ◆ ¿Qué es?

La **inyección SQL** es una **vulnerabilidad de seguridad** que ocurre cuando un atacante introduce **código SQL malicioso** dentro de una aplicación (por ejemplo, en un

formulario web o una variable), y ese código se ejecuta directamente en la base de datos.

👉 Es decir, el hacker logra **modificar la consulta original** para obtener, alterar o eliminar información sin autorización.

### ♦ 1. Código malicioso

El atacante aprovecha campos de entrada (como “usuario” o “contraseña”) para **inyectar comandos SQL**.

📘 Ejemplo vulnerable:

-- Supongamos que la aplicación construye la consulta así:

```
SELECT * FROM Usuarios
```

```
WHERE Usuario = 'braian' AND Clave = '1234';
```

Pero si el atacante escribe esto:

Usuario: admin' --

Clave: (vacío)

La consulta se convierte en:

```
SELECT * FROM Usuarios
```

```
WHERE Usuario = 'admin' --' AND Clave = '';
```

👉 El doble guion (--) comenta el resto de la línea, por lo tanto:

- Se ignora la parte del password.
- El atacante entra sin saber la contraseña. 🤖

### ♦ 2. Inserción de código por concatenación

Este error ocurre cuando la aplicación **arma la consulta SQL uniendo texto y variables del usuario** con concatenación de strings.

📘 Ejemplo inseguro:

```
DECLARE @SQL NVARCHAR(MAX);
```

```
SET @SQL = 'SELECT * FROM Clientes WHERE Nombre = ''' + @Nombre + ''';
```

```
EXEC(@SQL);
```

❌ Esto es **altamente peligroso**, porque si el usuario introduce:

```
@Nombre = 'José'; DELETE FROM Clientes; --
```

La consulta final sería:

```
SELECT * FROM Clientes WHERE Nombre = 'José'; DELETE FROM Clientes; --'
```

➡ ¡Y borra toda la tabla! 🤖

### ♦ 3. Los Stored Procedures con parámetros disminuyen el riesgo

Usar **procedimientos almacenados** que reciben parámetros **evita la concatenación de texto** y protege contra la inyección.

📘 Ejemplo seguro:

```
CREATE PROCEDURE dbo.BuscarCliente
```

@Nombre NVARCHAR(50)

AS

BEGIN

SELECT \* FROM Clientes WHERE Nombre = @Nombre;

END;

 Ejecución segura:

EXEC dbo.BuscarCliente @Nombre = 'José';

 Ventajas:

- Los parámetros son **validados y tipados** (NVARCHAR(50) en este caso).
- El motor SQL **no interpreta el valor como código**, sino como dato.
- La consulta **no se modifica dinámicamente**, por lo que es inmune a inyecciones.

#### ◆ 4. Validación de los datos ingresados

Además de usar parámetros, **siempre se debe validar la entrada del usuario**, tanto en la aplicación como en la base.

 Ejemplo de validación:

IF @Nombre LIKE '%--%' OR @Nombre LIKE '%;%'

THROW 50000, 'Entrada no válida', 1;

O incluso en la aplicación (por ejemplo, en C#, JavaScript, etc.), validar que:

- No se envíen caracteres peligrosos (', ', ', ', %, etc.)
- Se cumplan longitudes máximas.
- Los tipos de datos sean correctos.

 Esto **reduce riesgos** incluso ante errores de programación.

#### Resumen

| Concepto                       | Descripción                                   | Prevención                                   |
|--------------------------------|-----------------------------------------------|----------------------------------------------|
| <b>Código malicioso</b>        | Instrucciones añadidas a la consulta original | Nunca concatenar texto SQL                   |
| <b>Concatenación peligrosa</b> | Armar consultas con variables directas        | Usar parámetros tipados                      |
| <b>SP con parámetros</b>       | Separan datos del código SQL                  | El motor no interpreta el texto como comando |
| <b>Validación de datos</b>     | Controla que la entrada sea segura            | Validar antes de ejecutar                    |

 **Conclusión:**

La inyección SQL es una de las **vulnerabilidades más comunes** en bases de datos. Usar **Stored Procedures con parámetros** y **validar los datos ingresados** son las defensas más efectivas.

#### Resolución Diferida de Nombres

### ♦ ¿Qué significa?

Cuando **creás un procedimiento almacenado (Stored Procedure)** en SQL Server, el motor **verifica la sintaxis** (que esté bien escrito), pero **no valida si las tablas o columnas referenciadas existen** en ese momento.

👉 En otras palabras:

- **Sí verifica la forma del código (errores de sintaxis).**
- **✗ No verifica los objetos dentro de las sentencias (tablas, columnas, etc.).**

Esto es lo que se llama **resolución diferida de nombres**, porque la verificación **se pospone hasta el momento de la ejecución** del SP.

### ♦ Ejemplo práctico

■ Ejemplo 1 – Sin errores al crear:

```
CREATE PROCEDURE dbo.ListarClientes
AS
    SELECT * FROM Clientes;
GO
```

✅ El procedimiento se crea correctamente, **aunque la tabla Clientes no exista todavía.**

■ Pero al ejecutar:

```
EXEC dbo.ListarClientes;
```

✗ SQL Server arrojará un error:  
Msg 208, Level 16, State 1  
Invalid object name 'Clientes'.

➡ Esto ocurre porque recién en el momento de la **ejecución** se intenta resolver el nombre del objeto (Clientes).

### ♦ ¿Por qué SQL Server lo hace así?

Principalmente **por eficiencia y flexibilidad**:

1. Permite **crear procedimientos antes de crear las tablas** (útil en scripts de instalación o despliegue).
2. Acelera la **creación y compilación inicial**, porque no revisa toda la base.
3. Posibilita **referencias entre objetos que se crearán después**, evitando errores circulares.

■ Ejemplo:

```
CREATE PROCEDURE dbo.ReporteVentas
AS
    SELECT * FROM Ventas;
GO
-- Luego recién se crea la tabla
CREATE TABLE Ventas (
```



Id INT PRIMARY KEY,  
 Fecha DATE,  
 Total DECIMAL(10,2)  
 );

✓ Esto funciona porque la verificación de la tabla se hace **al ejecutar** el procedimiento, no al crearlo.

## ♦ 2. “Detecta errores sintácticos”

Durante la creación, SQL Server **sí valida la estructura del código**:

■ Ejemplo:

```
CREATE PROCEDURE dbo.PruebaError
AS
    SELECT * FORM Clientes; -- “FORM” en lugar de “FROM”
GO
```

✗ Resultado:

Msg 102, Level 15, State 1, Line 3  
 Incorrect syntax near 'FORM'.

➡ Este tipo de error se detecta **al crear** el procedimiento, porque es un **error de sintaxis** (la palabra está mal escrita).

## ♦ 3. “No verifica existencia de objetos”

■ Ejemplo:

```
CREATE PROCEDURE dbo.ListarEmpleados
AS
    SELECT * FROM Empleados;
GO
```

✓ Se crea correctamente aunque la tabla Empleados no exista todavía.

■ Pero al ejecutar:

```
EXEC dbo.ListarEmpleados;
```

✗ SQL Server devuelve:

Invalid object name 'Empleados'.

➡ Porque el nombre se **resuelve en tiempo de ejecución**, no en la creación.

## ♦ Resumen visual

| Etapas                  | Qué se verifica                           | Qué no se verifica                      |
|-------------------------|-------------------------------------------|-----------------------------------------|
| <b>Creación del SP</b>  | Sintaxis correcta (SELECT, FROM, etc.)    | Existencia de tablas, columnas o vistas |
| <b>Ejecución del SP</b> | Existencia y accesibilidad de los objetos | —                                       |

## ♦ 💡 Consejos prácticos

✓ Siempre probá tus procedimientos después de crearlos:

EXEC dbo.NombreProcedimiento;

- ✓ Si estás migrando bases o haciendo scripts de instalación, el orden correcto es:
  1. Crear tablas.
  2. Crear vistas.
  3. Crear procedimientos.
  4. Ejecutar pruebas.
- ✓ En proyectos grandes, podés usar herramientas (como Visual Studio Data Tools o SQL Projects) que validan dependencias entre objetos.

### En resumen

- **Resolución diferida de nombres** = SQL Server **no valida la existencia** de los objetos referenciados al crear un SP.
- Solo **detecta errores de sintaxis**, no errores de existencia.
- La validación de nombres se hace **en el momento de ejecución**.

### Compilación en Stored Procedures

#### ◆ 1. Ejecución (primera vez)

Cuando ejecutás un procedimiento almacenado por primera vez, SQL Server **no lo ejecuta directamente**: primero lo **compila**.

Ese proceso incluye varias etapas:

#### ✿ a) Verifica la existencia de los objetos

Ahora sí (a diferencia de la “resolución diferida de nombres”):

- SQL Server **comprueba que existan las tablas, vistas y columnas** referenciadas en el SP.
- Si algo no existe o cambió, el procedimiento **falla** al ejecutarse.

#### Ejemplo:

```
CREATE PROCEDURE dbo.ListarClientes
AS
    SELECT * FROM Clientes;
GO
```

EXEC dbo.ListarClientes; -- aquí se verifica que la tabla Clientes exista

#### ✿ b) Validación

Se verifica que las **sentencias SQL sean correctas** y que los **tipos de datos sean compatibles**.

Por ejemplo, si intentás comparar un VARCHAR con un INT, se marcará como error.

#### ✿ c) Análisis de las instrucciones

SQL analiza cómo ejecutar las consultas internamente:

- Qué índices usar.
- En qué orden unir tablas.

- Qué ruta de acceso a datos es más eficiente.

Este análisis se usa para construir un **plan de ejecución**.

#### ✿ d) Creación del Plan de Ejecución

El **plan de ejecución** es como un “mapa de ruta” que indica cómo el motor debe acceder y combinar los datos.

Se genera una vez y se **guarda en memoria caché (plan cache)** para que las próximas ejecuciones sean más rápidas.

■ Ejemplo:

EXEC dbo.ListarClientes; -- primera ejecución → se crea el plan

EXEC dbo.ListarClientes; -- siguientes ejecuciones → reutiliza el plan

✓ **Ventaja:** mejora el rendimiento porque evita recompilar cada vez.

#### ✿ e) Compilación

El motor transforma el SQL en un **código ejecutable interno (plan binario)** optimizado para esa base de datos.

#### ✿ f) Plan de ejecución permanece en memoria

El plan compilado se guarda en el **Plan Cache**.

Mientras no se modifique la estructura de la base (índices, estadísticas o tablas), **el plan se reutiliza** en futuras ejecuciones del mismo SP.

✓ **Resultado:** las siguientes ejecuciones son **más rápidas**, porque SQL Server ya sabe exactamente cómo resolver la consulta.

### ◆ 2. Recompilación

Hay casos donde el **plan de ejecución** deja de ser óptimo o válido, y SQL Server necesita **recompilar** el SP.

#### ◆ a) sp\_recompile

Comando que marca un procedimiento para recompilar la próxima vez que se ejecute.

■ Ejemplo:

EXEC sp\_recompile 'dbo.ListarClientes';

👉 La próxima vez que se ejecute, SQL Server generará un nuevo plan de ejecución.

#### ◆ b) WITH RECOMPILE (Definición del procedimiento)

Podés definir que un procedimiento **siempre se compile de nuevo** cada vez que se ejecute.

■ Ejemplo:

CREATE PROCEDURE dbo.ListarClientes

WITH RECOMPILE

AS

SELECT \* FROM Clientes WHERE Ciudad = 'Salta';

✓ Ventaja: el plan siempre estará actualizado.

✗ Desventaja: se pierde el beneficio del cache → **menor rendimiento** si se ejecuta muy seguido.

#### ♦ c) WITH RECOMPILE (en la ejecución)

También podés forzar la recompilación solo **en una ejecución específica**, sin modificar el procedimiento.

■ Ejemplo:

EXEC dbo.ListarClientes WITH RECOMPILE;

#### ♦ 3. ¿Cuándo se recompila automáticamente?

SQL Server recompila un SP automáticamente cuando:

- Cambia la estructura de una tabla (ALTER TABLE).
- Se actualizan índices o estadísticas.
- Se detectan valores de parámetros muy diferentes (para optimizar el plan).
- Cambia el esquema del usuario o los permisos.

#### 🧠 Resumen visual

| Etapas                        | Qué hace                                    | Resultado                          |
|-------------------------------|---------------------------------------------|------------------------------------|
| <b>Primera ejecución</b>      | Verifica objetos, analiza, crea plan        | Plan guardado en caché             |
| <b>Ejecuciones siguientes</b> | Usa el plan guardado                        | Más rápido                         |
| <b>Recompilación</b>          | Se crea un nuevo plan (manual o automática) | Optimiza rendimiento según cambios |

#### 💡 En resumen

- Un Stored Procedure **se compila la primera vez** que se ejecuta.
- SQL Server **guarda el plan de ejecución** en memoria para hacerlo más rápido después.
- Si cambian los datos o estructuras, se puede **recompilar** manual o automáticamente.
- WITH RECOMPILE fuerza una nueva compilación (útil en casos especiales).

#### 📌 Creación de Procedimientos Almacenados

Un **Stored Procedure (SP)** es un bloque de código SQL que se guarda en el servidor y se puede ejecutar cuantas veces quieras.

Se crea con la sentencia CREATE PROCEDURE (o su abreviatura CREATE PROC).

#### ♦ 1. Sintaxis general

CREATE { PROC | PROCEDURE } [Esquema.]Nombre\_Procedimiento

[ { @parametro [ tipo\_dato ] [ = valor\_default ] [ OUTPUT ] } [ ,...n ] ]

[ WITH <opciones> [ ,...n ] ]

AS

## Sentencias\_SQL

## ♦ 2. Partes de la sentencia

## ✚ a) Nombre del procedimiento

- Se debe colocar un **nombre único** dentro del esquema.
- Por convención se usa el esquema **dbo** (Database Owner) si no se especifica otro.

■ Ejemplo:

```
CREATE PROCEDURE dbo.ListarClientes
```

## ✚ b) Parámetros

Los procedimientos pueden recibir **parámetros de entrada, de salida** o ambos.

- **Entrada:** reciben valores cuando se ejecuta el SP.
- **Salida:** devuelven un valor al final de la ejecución.
- **Valor por defecto:** si no se pasa, usa ese valor.

■ Ejemplo:

```
CREATE PROCEDURE dbo.BuscarCliente
```

```
@Ciudad NVARCHAR(50) = 'Salta', -- parámetro con valor por defecto
```

```
@TotalVentas DECIMAL(10,2) OUTPUT -- parámetro de salida
```

```
AS
```

```
BEGIN
```

```
SELECT * FROM Clientes WHERE Ciudad = @Ciudad;
```

```
SELECT @TotalVentas = SUM(Ventas.Total) FROM Ventas WHERE Ciudad = @Ciudad;
```

```
END;
```

■ Ejecución con parámetros:

```
DECLARE @Monto DECIMAL(10,2);
```

```
EXEC dbo.BuscarCliente @Ciudad = 'Jujuy', @TotalVentas = @Monto OUTPUT;
```

```
SELECT @Monto AS Total;
```

## ✚ c) Opciones (WITH)

Se pueden definir ciertas **opciones opcionales** al crear el SP:

| Opción            | Descripción                                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>ENCRYPTION</b> | Encripta el texto del procedimiento en la base (no se puede ver con sp_helptext).                                                |
| <b>RECOMPILE</b>  | Fuerza recompilación cada vez que se ejecute (no usa caché).                                                                     |
| <b>EXECUTE AS</b> | Define el contexto de seguridad bajo el cual se ejecuta el procedimiento (por ejemplo, como otro usuario o con permisos de DBA). |

■ Ejemplo:

```
CREATE PROCEDURE dbo.ListarVentas
```

```
WITH ENCRYPTION, EXECUTE AS OWNER
```

AS

```
SELECT * FROM Ventas;
```

### d) Cuerpo del procedimiento

Después del AS van las **sentencias SQL** que ejecutará el SP.

Podés usar:

- SELECT, INSERT, UPDATE, DELETE
- Control de flujo: IF, WHILE, BEGIN...END
- Variables locales
- Otros SP dentro del mismo (anidación)

 Ejemplo simple:

```
CREATE PROCEDURE dbo.ContarClientes
```

```
AS
```

```
BEGIN
```

```
    SELECT COUNT(*) AS TotalClientes FROM Clientes;
```

```
END;
```

### Modificación

Si querés cambiar el contenido de un procedimiento sin eliminarlo, usás:

```
ALTER PROCEDURE dbo.NombreProcedimiento
```

```
AS
```

```
-- Nuevo código
```

 Ejemplo:

```
ALTER PROCEDURE dbo.ContarClientes
```

```
AS
```

```
BEGIN
```

```
    SELECT COUNT(*) AS TotalClientes, Ciudad
```

```
    FROM Clientes
```

```
    GROUP BY Ciudad;
```

```
END;
```

 ALTER conserva los permisos, parámetros y dependencias existentes.

### Borrado

Si ya no necesitás un SP, podés eliminarlo con:

```
DROP PROCEDURE dbo.NombreProcedimiento;
```

 Ejemplo:

```
DROP PROCEDURE dbo.ContarClientes;
```

### Resumen práctico

| Operación | Comando          | Ejemplo                             |
|-----------|------------------|-------------------------------------|
| Crear     | CREATE PROCEDURE | CREATE PROCEDURE dbo.ListarClientes |
| Modificar | ALTER PROCEDURE  | ALTER PROCEDURE dbo.ListarClientes  |

|                 |                       |                                          |
|-----------------|-----------------------|------------------------------------------|
| <b>Eliminar</b> | <b>DROP PROCEDURE</b> | <b>DROP PROCEDURE dbo.ListarClientes</b> |
|-----------------|-----------------------|------------------------------------------|



### Recomendaciones

- ✓ Usá nombres descriptivos (por ejemplo: sp\_InsertarEmpleado, sp\_ListarVentas).
- ✓ Evitá el prefijo **sp\_**, porque SQL Server lo reserva para procedimientos del sistema.
- ✓ Siempre **probá** el SP después de crearlo o modificarlo (EXEC dbo.NombreProcedimiento).
- ✓ Documentá los parámetros y su uso (especialmente los OUTPUT).



### Información sobre los Stored Procedures

#### ◆ 1. SQL Server Management Studio (SSMS)

Es la herramienta gráfica principal para trabajar con SQL Server.

Podés usarla para:

- **Ver** todos los procedimientos almacenados.
- **Editar o modificar** uno existente.
- **Ejecutarlos manualmente.**
- **Ver su definición (el código SQL).**



Pasos:

1. Abrí el **Object Explorer**.
2. Expandí la base de datos → carpeta **Programmability** → **Stored Procedures**.
3. Desde ahí podés:
  - Hacer clic derecho → **Modify** (para editar).
  - O **Script Stored Procedure as** → **CREATE/ALTER/DROP** (para ver o copiar su código).

#### ◆ 2. Vistas de Catálogo

SQL Server guarda toda la información de los objetos (tablas, SP, funciones, etc.) en **vistas del sistema** dentro de la base de datos.

Estas vistas te permiten **consultar metadatos** directamente con SQL.



**Principales vistas relacionadas con los procedimientos almacenados:**

| Vista                 | Descripción                                                                     | Ejemplo                                                                          |
|-----------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>sys.procedures</b> | Lista todos los procedimientos almacenados definidos por el usuario.            | SELECT name, create_date FROM sys.procedures;                                    |
| <b>sys.parameters</b> | Muestra los parámetros de cada SP (nombre, tipo, dirección, valor por defecto). | SELECT * FROM sys.parameters WHERE object_id = OBJECT_ID('dbo.MiProcedimiento'); |



|                             |                                                                             |                                                                                            |
|-----------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>sys.sql_modules</b>      | Contiene el texto (definición SQL) de cada módulo (SP, función o trigger).  | SELECT definition FROM sys.sql_modules WHERE object_id = OBJECT_ID('dbo.MiProcedimiento'); |
| <b>sys.sql_dependencies</b> | Muestra las dependencias entre objetos (qué tablas usa un SP, por ejemplo). | SELECT * FROM sys.sql_dependencies WHERE object_id = OBJECT_ID('dbo.MiProcedimiento');     |

### Ejemplo práctico:

-- Ver todos los procedimientos almacenados

```
SELECT name, object_id, create_date, modify_date
FROM sys.procedures;
```

-- Ver los parámetros de un procedimiento específico

```
SELECT name, parameter_id, system_type_id, is_output
FROM sys.parameters
WHERE object_id = OBJECT_ID('dbo.BuscarCliente');
```

-- Ver el código fuente del procedimiento

```
SELECT definition
FROM sys.sql_modules
WHERE object_id = OBJECT_ID('dbo.BuscarCliente');
```

-- Ver qué tablas usa el procedimiento

```
SELECT referenced_major_id, referenced_minor_id
FROM sys.sql_dependencies
WHERE object_id = OBJECT_ID('dbo.BuscarCliente');
```

### ◆ 3. Procedimientos almacenados del sistema

SQL Server incluye varios **stored procedures internos** (prefijo sp\_) que sirven para **consultar información del sistema**, sin necesidad de usar las vistas directamente.

#### ◆ Principales SP del sistema:

| Comando            | Descripción                                                      | Ejemplo                               |
|--------------------|------------------------------------------------------------------|---------------------------------------|
| <b>sp_helptext</b> | Muestra el código fuente del SP, función o vista.                | EXEC sp_helptext 'dbo.BuscarCliente'; |
| <b>sp_help</b>     | Muestra información general del objeto (tipo, parámetros, etc.). | EXEC sp_help 'dbo.BuscarCliente';     |
| <b>sp_depends</b>  | Muestra qué objetos dependen del SP o de qué objetos depende él. | EXEC sp_depends 'dbo.BuscarCliente';  |

### Ejemplo práctico completo:

-- Mostrar la definición del procedimiento

```
EXEC sp_helptext 'dbo.BuscarCliente';
```

```
-- Mostrar los detalles (parámetros, tipo de objeto)
```

```
EXEC sp_help 'dbo.BuscarCliente';
```

```
-- Mostrar las dependencias (tablas o SP que usa)
```

```
EXEC sp_depends 'dbo.BuscarCliente';
```

### Resumen visual

| Método                    | Qué permite hacer                                    | Ejemplo                                    |
|---------------------------|------------------------------------------------------|--------------------------------------------|
| <b>SSMS (gráfico)</b>     | Ver, modificar y ejecutar SP fácilmente              | Menú → Programmability → Stored Procedures |
| <b>Vistas de catálogo</b> | Consultar detalles específicos mediante SQL          | sys.procedures, sys.parameters, etc.       |
| <b>SP del sistema</b>     | Obtener información sin escribir consultas complejas | sp_helptext, sp_help, sp_depends           |

### Consejos prácticos

- ✓ Usá **vistas de catálogo** para crear scripts automatizados o reportes de metadatos.
- ✓ Usá **sp\_helptext** cuando necesites leer o copiar el código de un SP.
- ✓ Recordá que los SP del sistema (sp\_...) están disponibles en todas las bases.
- ✓ Evitá nombrar tus procedimientos con sp\_ para no generar conflictos.

### Parámetros en Stored Procedures

#### ◆ 1. Manejo de Parámetros

Los **parámetros** son variables que se declaran en la definición del procedimiento y se utilizan para pasar o devolver valores.

Permiten que el mismo SP se ejecute con diferentes datos sin tener que reescribirlo.

 Ejemplo general:

```
CREATE PROCEDURE dbo.BuscarCliente
```

```
  @Ciudad NVARCHAR(50) -- parámetro
```

```
AS
```

```
BEGIN
```

```
  SELECT * FROM Clientes WHERE Ciudad = @Ciudad;
```

```
END;
```

 Ejecución:

```
EXEC dbo.BuscarCliente @Ciudad = 'Salta';
```

- ✓ Así, el mismo procedimiento sirve para distintas ciudades.

#### ◆ 2. Dirección del parámetro

Los parámetros pueden tener **tres direcciones**:

| Tipo | Palabra clave | Función |
|------|---------------|---------|
|------|---------------|---------|

|         |               |                                                                      |
|---------|---------------|----------------------------------------------------------------------|
| Entrada | (por defecto) | Envía valores al SP.                                                 |
| Salida  | OUTPUT        | Devuelve valores del SP al programa o script que lo llama.           |
| Retorno | RETURN        | Devuelve un único valor numérico (generalmente un código de estado). |

### ◆ 3. Valores predeterminados

Podés definir un valor por defecto para un parámetro.

Si el usuario **no lo especifica** al ejecutar el SP, se usa ese valor.

■ Ejemplo:

```
CREATE PROCEDURE dbo.BuscarCliente
```

```
    @Ciudad NVARCHAR(50) = 'Salta'
```

```
AS
```

```
BEGIN
```

```
    SELECT * FROM Clientes WHERE Ciudad = @Ciudad;
```

```
END;
```

■ Ejecución:

```
EXEC dbo.BuscarCliente;      -- Usa 'Salta' (por defecto)
```

```
EXEC dbo.BuscarCliente 'Jujuy'; -- Usa 'Jujuy' (reemplaza el valor)
```

✓ Útil para dar comportamientos por defecto o valores más usados.

### ◆ 4. Parámetros de entrada

Son los **más comunes**.

Se usan para **enviar información** al procedimiento.

■ Ejemplo:

```
CREATE PROCEDURE dbo.ClientesPorCiudad
```

```
    @Ciudad NVARCHAR(50)
```

```
AS
```

```
BEGIN
```

```
    SELECT * FROM Clientes WHERE Ciudad = @Ciudad;
```

```
END;
```

```
EXEC dbo.ClientesPorCiudad 'Buenos Aires';
```

### ◆ 5. Parámetros de salida

Permiten **devolver valores** desde el procedimiento hacia el programa o script que lo ejecuta.

■ Ejemplo:

```
CREATE PROCEDURE dbo.TotalVentasPorCiudad
```

```
    @Ciudad NVARCHAR(50),
```

```
    @TotalVentas DECIMAL(10,2) OUTPUT
```

```
AS
```

BEGIN

```
SELECT @TotalVentas = SUM(Monto) FROM Ventas WHERE Ciudad = @Ciudad;
```

END;

 Ejecución:

```
DECLARE @Total DECIMAL(10,2);
```

```
EXEC dbo.TotalVentasPorCiudad @Ciudad = 'Salta', @TotalVentas = @Total OUTPUT;
```

```
SELECT @Total AS 'Total de Ventas';
```

✓ Ideal cuando querés que el SP **te devuelva un resultado calculado o un valor de estado**.

#### ◆ 6. Parámetro de retorno (RETURN)

Se usa para devolver un **valor único (numérico)**, normalmente para indicar si la ejecución fue correcta o hubo errores.

Difiere del OUTPUT porque no devuelve datos, sino un **código de estado**.

 Ejemplo:

```
CREATE PROCEDURE dbo.ValidarStock
```

```
@ProductoID INT
```

```
AS
```

```
BEGIN
```

```
DECLARE @Stock INT;
```

```
SELECT @Stock = Cantidad FROM Productos WHERE ID = @ProductoID;
```

```
IF @Stock > 0
```

```
    RETURN 1; -- éxito
```

```
ELSE
```

```
    RETURN 0; -- sin stock
```

```
END;
```

 Ejecución:

```
DECLARE @Resultado INT;
```

```
EXEC @Resultado = dbo.ValidarStock @ProductoID = 12;
```

```
SELECT @Resultado AS 'Disponible'; -- 1 o 0
```

✓ RETURN solo devuelve **un entero**, mientras que OUTPUT puede devolver **cualquier tipo de dato**.



#### Resumen General

| Tipo de parámetro | Palabra clave | Se usa para...            | Ejemplo                              |
|-------------------|---------------|---------------------------|--------------------------------------|
| Entrada           | (ninguna)     | Enviar datos al SP        | @Ciudad NVARCHAR(50)                 |
| Salida            | OUTPUT        | Devolver datos calculados | @TotalVentas<br>DECIMAL(10,2) OUTPUT |

|                       |         |                                       |                                   |
|-----------------------|---------|---------------------------------------|-----------------------------------|
| <b>Retorno</b>        | RETURN  | Devolver un código de estado (entero) | RETURN 1 o RETURN 0               |
| <b>Predeterminado</b> | = valor | Usar un valor si no se pasa nada      | @Ciudad NVARCHAR(50)<br>= 'Salta' |

### 💡 En resumen

- Los **parámetros hacen a los SP reutilizables**.
- Pueden **recibir o devolver información** según la necesidad.
- OUTPUT es para devolver resultados específicos.
- RETURN es para indicar éxito o error.
- Los **valores predeterminados** simplifican la ejecución.

### ✖ Manejo de Parámetros

#### ◆ 1. Nombre de los parámetros

En SQL Server, **todo parámetro debe comenzar con el símbolo @**, igual que las variables.

■ Ejemplo:

```
CREATE PROCEDURE dbo.CalcularTotal
    @Precio DECIMAL(10,2),
    @Cantidad INT
AS
BEGIN
    SELECT @Precio * @Cantidad AS Total;
END;
```

Aquí:

- @Precio y @Cantidad son los parámetros.
- Dentro del procedimiento, se los puede usar igual que una variable normal.

#### ◆ 2. Pasar valores al ejecutar el procedimiento

Cuando ejecutás un SP, podés enviarle los valores de **dos maneras: por nombre o por posición**.

##### ✅ Por nombre (más recomendable)

Podés indicar explícitamente qué valor corresponde a cada parámetro. Esto mejora la legibilidad y evita errores si el orden cambia.

■ Ejemplo:

```
EXEC dbo.CalcularTotal @Cantidad = 3, @Precio = 250.50;
```

✓ No importa el orden, mientras uses los nombres de los parámetros.

##### ✅ Por posición

Los valores se envían en el mismo **orden** en que fueron definidos los parámetros. No se usa el nombre, solo el valor.

■ Ejemplo:

EXEC dbo.CalcularTotal 250.50, 3;

⚠ Deben coincidir:

- La cantidad de parámetros.
- El orden exacto en que fueron definidos.
- Los tipos de datos esperados.

### Comparación rápida

| Modo         | Ventajas                                      | Riesgos                                                                  |
|--------------|-----------------------------------------------|--------------------------------------------------------------------------|
| Por nombre   | Más claro, flexible, permite cambiar el orden | Es más largo de escribir                                                 |
| Por posición | Más corto                                     | Si el orden o cantidad cambia, puede fallar o dar resultados incorrectos |

### Dirección del Parámetro

Como indica la segunda diapositiva, los parámetros pueden tener **dirección de entrada o salida**:

- **Entrada:** se usan para enviar valores al SP (por defecto).
- **Salida (OUTPUT):** se usan para devolver información al llamador.

 Ejemplo con ambos tipos:

```
CREATE PROCEDURE dbo.CalcularImporte
  @Precio DECIMAL(10,2),
  @Cantidad INT,
  @Total DECIMAL(10,2) OUTPUT
```

AS

BEGIN

SET @Total = @Precio \* @Cantidad;

END;

 Ejecución:

```
DECLARE @Resultado DECIMAL(10,2);
```

```
EXEC dbo.CalcularImporte @Precio = 100, @Cantidad = 5, @Total = @Resultado
OUTPUT;
```

```
SELECT @Resultado AS 'Total a Pagar';
```

### Valores Predeterminados

Podés asignar **un valor por defecto** a un parámetro.

Si al ejecutar el SP no se envía ese parámetro, SQL Server usa el valor establecido.

 Ejemplo:

```
CREATE PROCEDURE dbo.ListarProductos
  @Categoria NVARCHAR(50) = 'General'
```

AS

BEGIN

```
SELECT * FROM Productos WHERE Categoria = @Categoria;
END;
```

 Ejecución:

```
EXEC dbo.ListarProductos;          -- Usa 'General'
```

```
EXEC dbo.ListarProductos 'Electrónica'; -- Usa el valor enviado
```

 Esto hace que el parámetro sea **opcional**.

### Resumen General

| Concepto              | Descripción                          | Ejemplo                        |
|-----------------------|--------------------------------------|--------------------------------|
| <b>Nombre</b>         | Siempre comienza con @               | @Ciudad NVARCHAR(50)           |
| <b>Por nombre</b>     | Se indica el nombre al pasar valores | EXEC SP @Param1 = 1            |
| <b>Por posición</b>   | Se pasan los valores en orden        | EXEC SP 1, 2, 3                |
| <b>Entrada</b>        | Envía valores al SP                  | @Cantidad INT                  |
| <b>Salida</b>         | Devuelve valores al exterior         | @Total DECIMAL(10,2) OUTPUT    |
| <b>Predeterminado</b> | Valor opcional si no se pasa otro    | @Ciudad NVARCHAR(50) = 'Salta' |

### Consejo práctico

- Usá **por nombre** en proyectos reales, mejora la legibilidad y mantenimiento.
- Combiná **OUTPUT** con **RETURN** para devolver resultados y estados.
- Si hacés pruebas, usá PRINT o SELECT dentro del SP para ver los valores de los parámetros.

### Parámetros de Entrada

#### ◆ ¿Qué son?

Los **parámetros de entrada** son variables que el procedimiento almacenado **recibe como datos** desde el exterior (por ejemplo, desde una aplicación o una consulta SQL). Se usan cuando el SP necesita **filtrar, calcular o ejecutar una operación personalizada** según el valor recibido.

#### ◆ Ejemplo del slide explicado

```
ALTER PROCEDURE Production.LongLeadProducts
```

```
    @MinimumLength INT = 1
```

```
AS
```

```
IF (@MinimumLength < 0)
```

```
BEGIN
```

```
    RAISERROR('Dato inválido', 14, 1)
```

```
    RETURN
```

```
END
```

```
SELECT Name, ProductNumber, DaysToManufacture
```

```
FROM Production.Product
```



```
WHERE DaysToManufacture >= @MinimumLength
ORDER BY DaysToManufacture DESC, Name;
```

### Explicación paso a paso

#### 1. ALTER PROCEDURE ...

Modifica un procedimiento ya existente.

Si fuera nuevo, se usaría CREATE PROCEDURE.

#### 2. @MinimumLength INT = 1

- Se define un parámetro de **entrada** llamado @MinimumLength.
- Su tipo de dato es INT.
- Tiene un **valor predeterminado** (= 1), lo que significa que **es opcional**.  
Si no se le pasa ningún valor, usará **1**.

#### 3. Validación de datos

```
IF (@MinimumLength < 0)
```

```
BEGIN
```

```
RAISERROR('Dato inválido', 14, 1)
```

```
RETURN
```

```
END
```

- Se verifica que el parámetro no sea negativo.
- Si lo es, se usa RAISERROR para mostrar un mensaje de error personalizado.
- RETURN detiene la ejecución del procedimiento.

 **RAISERROR** muestra un mensaje en pantalla y puede registrar el error.

 Parámetros del RAISERROR:

- 'Dato inválido' → mensaje.
- 14 → nivel de severidad (indica error del usuario).
- 1 → número de estado (puede ser cualquiera).

#### 4. Consulta principal

```
SELECT Name, ProductNumber, DaysToManufacture
```

```
FROM Production.Product
```

```
WHERE DaysToManufacture >= @MinimumLength
```

```
ORDER BY DaysToManufacture DESC, Name;
```

- Usa el parámetro @MinimumLength para **filtrar los productos** que tardan **al menos esa cantidad de días** en fabricarse.
- Los ordena de mayor a menor tiempo de fabricación.

### Ejemplo práctico

Imaginá que la tabla Production.Product tiene estos datos:

| Name        | DaysToManufacture |
|-------------|-------------------|
| Tornillo M5 | 1                 |
| Motor A100  | 5                 |
| Chasis Base | 10                |

### ✖ Ejecución 1 — por nombre:

EXEC Production.LongLeadProducts @MinimumLength = 4;

✓ Muestra los productos con 4 días o más de fabricación:

Motor A100

Chasis Base

### ✖ Ejecución 2 — por posición:

EXEC Production.LongLeadProducts 4;

✓ Hace exactamente lo mismo, pero pasando el valor **sin nombrar el parámetro**.

### ✖ Ejecución 3 — sin argumentos:

EXEC Production.LongLeadProducts;

✓ Usa el **valor predeterminado = 1**, por lo tanto muestra **todos los productos con 1 día o más** (todos).

### 📊 Resumen general

| Concepto               | Descripción                                           | Ejemplo                           |
|------------------------|-------------------------------------------------------|-----------------------------------|
| Entrada                | Valor que se pasa al SP desde el exterior.            | @MinimumLength INT                |
| Valor por defecto      | Si no se pasa nada, usa ese valor.                    | @MinimumLength INT = 1            |
| Validación             | Controla que el parámetro sea válido antes de usarlo. | IF @MinimumLength < 0             |
| Error controlado       | Muestra un mensaje personalizado.                     | RAISERROR('Dato inválido', 14, 1) |
| Ejecución por nombre   | Se especifica cada parámetro.                         | EXEC SP @Param=valor              |
| Ejecución por posición | Se pasa el valor en orden.                            | EXEC SP valor                     |
| Sin argumento          | Usa los valores predeterminados.                      | EXEC SP                           |

### 💡 En resumen

- Los **parámetros de entrada** permiten **personalizar** el comportamiento de un SP.
- Se pueden validar antes de usarlos.
- Podés ejecutarlos **por nombre**, **por posición**, o **sin argumentos** si tienen valores por defecto.
- RAISERROR y RETURN son útiles para **manejo de errores** dentro del procedimiento.

### ✖ Parámetros de Salida (OUTPUT)

#### ◆ ¿Qué son?

Los **parámetros de salida** permiten que un procedimiento almacenado **devuelva un valor** a la aplicación o script que lo llama.

Son muy útiles cuando el SP realiza una acción (como insertar, actualizar o calcular algo) y querés conocer el **resultado** sin tener que hacer otra consulta aparte.

### ■ Ejemplo típico:

Insertar un nuevo registro y devolver el **ID generado** automáticamente por la base de datos.

#### ◆ Ejemplo del slide explicado

```
CREATE PROC HumanResources.AddDepartment
    @Name NVARCHAR(50),
    @GroupName NVARCHAR(50),
    @DeptID SMALLINT OUTPUT
AS
INSERT INTO HumanResources.Department (Name, GroupName)
VALUES (@Name, @GroupName);
SET @DeptID = SCOPE_IDENTITY();
```

### 🧠 Explicación paso a paso

#### 1. CREATE PROC ...

Define un nuevo procedimiento almacenado llamado AddDepartment.

#### 2. Parámetros de entrada

```
@Name NVARCHAR(50),
@GroupName NVARCHAR(50),
```

Estos dos **reciben datos** que se insertarán en la tabla Department.

#### 3. Parámetro de salida

```
@DeptID SMALLINT OUTPUT
```

Este parámetro **no recibe** un valor, sino que **devuelve** uno al finalizar el SP.

La palabra clave OUTPUT indica que será de **salida**.

#### 4. Sentencia INSERT INTO

```
INSERT INTO HumanResources.Department (Name, GroupName)
VALUES (@Name, @GroupName);
```

Inserta un nuevo departamento en la tabla.

#### 5. Capturar el ID generado

```
SET @DeptID = SCOPE_IDENTITY();
```

- SCOPE\_IDENTITY() devuelve el **último valor autogenerado (IDENTITY)** en la sesión actual.
- Ese valor se **asigna al parámetro de salida @DeptID**, que luego podrá ser leído desde afuera.

#### ◆ Ejecución del procedimiento con parámetro de salida

```
DECLARE @dept INT;
EXEC AddDepartment 'Refunds', '', @dept OUTPUT;
SELECT @dept;
```

## ✖ Paso a paso

### 1. Declarar una variable:

2. DECLARE @dept INT;

Esta variable almacenará el valor que el SP devolverá.

### 3. Ejecutar el SP:

4. EXEC AddDepartment 'Refunds', '', @dept OUTPUT;

- 'Refunds' y '' son los parámetros de **entrada** (@Name y @GroupName).
- @dept OUTPUT recibe el valor del parámetro de **salida** del SP.

### 5. Mostrar el resultado:

6. SELECT @dept;

Muestra el valor devuelto, que debería ser el **ID del nuevo departamento**.

## 📊 Ejemplo completo ilustrativo

Supongamos que la tabla Department contiene:

| DeptID | Name       | GroupName |
|--------|------------|-----------|
| 1      | Accounting | Admin     |
| 2      | Sales      | Business  |

Después de ejecutar:

DECLARE @NuevoID INT;

EXEC AddDepartment 'IT Support', 'Technology', @NuevoID OUTPUT;

SELECT @NuevoID AS 'Nuevo ID';

📊 Resultado:

| Nuevo ID |
|----------|
| 3        |

👉 Significa que se insertó el departamento “IT Support” con ID = 3, y el SP lo devolvió mediante el parámetro OUTPUT.

## 🧠 Diferencias con RETURN

| Característica                            | OUTPUT | RETURN                                  |
|-------------------------------------------|--------|-----------------------------------------|
| Puede devolver varios valores             | ✅ Sí   | ❌ No (solo uno)                         |
| Puede devolver cualquier tipo de dato     | ✅ Sí   | ❌ Solo INT                              |
| Se usa para datos calculados o resultados | ✅      | ⚠ Se usa para códigos de estado o error |
| Palabra clave                             | OUTPUT | RETURN                                  |

📊 Ejemplo de uso combinado:

CREATE PROCEDURE dbo.CalcularTotal

@Precio DECIMAL(10,2),

@Cantidad INT,

@Total DECIMAL(10,2) OUTPUT

AS

BEGIN

IF @Cantidad <= 0 RETURN -1; -- código de error

SET @Total = @Precio \* @Cantidad;

RETURN 0; -- éxito

END;

### Resumen rápido

| Concepto          | Descripción                          | Ejemplo                  |
|-------------------|--------------------------------------|--------------------------|
| <b>Definición</b> | Devuelve valores desde un SP         | @DeptID INT OUTPUT       |
| <b>Asignación</b> | Se le asigna dentro del SP           | SET @DeptID = valor      |
| <b>Recepción</b>  | Se declara y se usa con OUTPUT       | EXEC SP ..., @var OUTPUT |
| <b>Uso común</b>  | Obtener IDs, resultados o cálculos   | SCOPE_IDENTITY()         |
| <b>Ventaja</b>    | No hace falta una consulta adicional | Más eficiente            |

### En resumen

- Los **parámetros de salida** son ideales para devolver valores **personalizados o generados** dentro del procedimiento.
- Se declaran con la palabra **OUTPUT** y se leen fuera del SP.
- Podés devolver **varios valores a la vez** usando múltiples parámetros OUTPUT.
- Son diferentes de RETURN, que solo puede devolver **un entero** (por ejemplo, código de error o estado).

### Parámetro de Retorno (RETURN)

#### ◆ ¿Qué es?

- Es un **valor entero** (INT) que un procedimiento almacenado puede devolver al final de su ejecución.
- Se utiliza para **indicar el estado** del procedimiento:
  - 0 → ejecución exitosa.
  - Cualquier otro número (ej. -1, 1, 99) → indica un tipo de error o situación especial.
- A diferencia de los parámetros OUTPUT, solo se puede devolver **un único valor entero**.

#### ◆ Ejemplo del slide explicado

ALTER PROC HumanResources.AddDepartment

@Name NVARCHAR(50),

@GroupName NVARCHAR(50),

@DeptID SMALLINT OUTPUT

AS

IF ((@Name = '') OR (@GroupName = ''))

RETURN -1

```
INSERT INTO HumanResources.Department (Name, GroupName)
VALUES (@Name, @GroupName)
SET @DeptID = SCOPE_IDENTITY()
RETURN 0
```

## Explicación paso a paso

### 1. Parámetros

- @Name y @GroupName: son **de entrada**.
- @DeptID: es **de salida**, devolverá el ID generado.
- El RETURN devolverá un **código de estado** (éxito o error).

### 2. Validación

```
IF ((@Name = '') OR (@GroupName = ''))
RETURN -1
```

👉 Si el nombre o grupo están vacíos, el SP **termina** inmediatamente devolviendo -1. No ejecuta el INSERT.

### 3. Inserción

```
INSERT INTO HumanResources.Department (Name, GroupName)
VALUES (@Name, @GroupName)
```

✅ Si los datos son válidos, inserta el nuevo departamento.

### 4. Captura del ID

```
SET @DeptID = SCOPE_IDENTITY()
```

Asigna al parámetro OUTPUT el ID del nuevo registro.

### 5. Devolver éxito

```
RETURN 0
```

✔ Devuelve **0** para indicar que la ejecución fue correcta.

## Ejecución con Parámetro de Retorno

```
DECLARE @dept INT, @result INT;
EXEC @result = AddDepartment 'Refunds', '', @dept OUTPUT;
IF (@result = 0)
    SELECT @dept AS 'ID del Nuevo Departamento';
ELSE
    SELECT 'Error' AS 'Estado';
```

## Paso a paso

### 1. Declaración de variables

2. DECLARE @dept INT, @result INT;

- @dept guardará el valor del parámetro OUTPUT (el ID insertado).
- @result almacenará el valor devuelto por RETURN.

### 3. Ejecución

4. EXEC @result = AddDepartment 'Refunds', '', @dept OUTPUT;

- @result recibe el valor de retorno (0 o -1).

- @dept recibe el ID si la inserción fue exitosa.

## 5. Evaluación

- IF (@result = 0)
- SELECT @dept;
- ELSE
- SELECT 'Error';
  - Si @result = 0, muestra el ID insertado.
  - Si @result = -1, muestra un mensaje de error.

## Resultado esperado

| Ejecución                                          | Retorno (@result) | OUTPUT (@dept) | Resultado mostrado |
|----------------------------------------------------|-------------------|----------------|--------------------|
| EXEC AddDepartment 'Ventas', 'Admin', @dept OUTPUT | 0                 | nuevo ID       | Muestra el ID      |
| EXEC AddDepartment '', 'Admin', @dept OUTPUT       | -1                | NULL           | Muestra "Error"    |

## Comparación: OUTPUT vs RETURN

| Característica | OUTPUT                      | RETURN                      |
|----------------|-----------------------------|-----------------------------|
| Tipo de dato   | Cualquiera                  | Solo INT                    |
| Cantidad       | Múltiples                   | Solo uno                    |
| Uso típico     | Devolver datos o resultados | Indicar estado o error      |
| Ejemplo        | @DeptID OUTPUT              | RETURN 0                    |
| Lectura        | Con variable @var OUTPUT    | Con variable antes del EXEC |

## En resumen

- OUTPUT devuelve **datos** (valores calculados, IDs, totales, etc.).
- RETURN devuelve un **código de estado** (éxito o error).
- Ambos pueden usarse juntos (como en el ejemplo).
- RETURN **interrumpe** la ejecución del procedimiento al instante.
- Siempre devuelve un **entero**.

## Manejo Estructurado de Excepciones en SQL Server

El **manejo estructurado de excepciones** permite **capturar y controlar errores** que se produzcan durante la ejecución de sentencias SQL, sin que el procedimiento falle abruptamente.

Esto se logra mediante los bloques TRY...CATCH.

## Estructura básica de TRY / CATCH

BEGIN TRY

-- Sentencias SQL que podrían generar errores

END TRY

BEGIN CATCH



-- Código que se ejecuta si ocurre un error

END CATCH

### ■ Explicación:

- El código dentro del bloque **TRY** se ejecuta normalmente.
- Si ocurre un error **controlable**, el flujo pasa automáticamente al bloque **CATCH**, sin detener la ejecución general.
- Desde el **CATCH**, podés registrar el error, devolver mensajes personalizados o incluso revertir transacciones.

### 🧠 Ejemplo práctico (del slide)

```
CREATE PROCEDURE dbo.AgregarDatos
    @a INT,
    @b VARCHAR(50)
AS
BEGIN TRY
    INSERT INTO TablaPrueba VALUES (@a, @b);
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS NumError,
        ERROR_MESSAGE() AS MsgError;
END CATCH;
```

### ▶ Ejecución:

EXEC dbo.AgregarDatos 1, 'Primero';

### ✅ Explicación paso a paso:

#### 1. Intento (TRY):

SQL intenta ejecutar el INSERT.  
Si todo sale bien, termina allí.

#### 2. Captura (CATCH):

Si ocurre un error (por ejemplo, tipo de dato incorrecto, violación de clave primaria, etc.),  
el control pasa al bloque CATCH.

#### 3. Funciones del bloque CATCH:

- ERROR\_NUMBER() → Devuelve el número del error.
- ERROR\_MESSAGE() → Devuelve el texto descriptivo del error.

#### 💡 Otros disponibles:

- ERROR\_LINE() → Línea donde ocurrió el error.
- ERROR\_PROCEDURE() → Nombre del procedimiento que falló.
- ERROR\_SEVERITY() → Nivel de gravedad.
- ERROR\_STATE() → Estado interno del error.

## Guía para el control de errores

| Regla                                                       | Explicación                                                                   |
|-------------------------------------------------------------|-------------------------------------------------------------------------------|
| TRY y CATCH deben estar en el mismo lote                    | No pueden separarse con GO ni definirse en diferentes scripts.                |
| CATCH viene inmediatamente después de TRY                   | No puede haber código entre ambos bloques.                                    |
| Pueden anidarse                                             | TRY/CATCH dentro de otro TRY/CATCH (por ejemplo, en transacciones complejas). |
| Errores <b>graves (&gt;20)</b> no se capturan               | Si el error desconecta la sesión, no pasa al CATCH.                           |
| Errores <b>menores (&lt;10)</b> no son errores de ejecución | Son advertencias, no activan el bloque CATCH.                                 |

### Transacciones y TRY / CATCH

Cuando usás transacciones dentro de un procedimiento, es muy común combinar TRY/CATCH con ROLLBACK para **mantener la integridad de los datos**.

 Ejemplo:

```
BEGIN TRY
    BEGIN TRANSACTION
        INSERT INTO Clientes VALUES ('Juan', 'Perez');
        INSERT INTO Ventas VALUES (1, 5000);
    COMMIT TRANSACTION
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    SELECT
        ERROR_NUMBER() AS NumError,
        ERROR_MESSAGE() AS MsgError;
END CATCH
```

 Explicación:

- Si **todo sale bien**, se ejecuta COMMIT TRANSACTION.
- Si **algo falla**, se cancela todo con ROLLBACK, y se muestra el error.
- Así evitás dejar datos incompletos o inconsistentes.

### Transacciones no confirmables

Una **transacción no confirmable** ocurre cuando se produce un error dentro de un bloque TRY/CATCH que **daña la transacción actual**.

Ejemplo:

- Violaciones de constraints (CHECK, FOREIGN KEY, etc.).
- Errores en triggers.

➡ En esos casos, aunque uses ROLLBACK, la transacción ya queda marcada como inválida y no puede confirmarse (ni siquiera con COMMIT).

### Funciones de Error más comunes

| Función           | Descripción               | Ejemplo                                  |
|-------------------|---------------------------|------------------------------------------|
| ERROR_NUMBER()    | Código numérico del error | 2627 (clave duplicada)                   |
| ERROR_MESSAGE()   | Mensaje descriptivo       | 'Violation of PRIMARY KEY constraint...' |
| ERROR_SEVERITY()  | Nivel de gravedad (0–25)  | 16 = error del usuario                   |
| ERROR_LINE()      | Línea donde ocurrió       | 12                                       |
| ERROR_PROCEDURE() | Nombre del SP             | 'AgregarDatos'                           |

### En resumen

| Concepto                | Descripción                                                         |
|-------------------------|---------------------------------------------------------------------|
| <b>TRY/CATCH</b>        | Estructura para capturar y manejar errores sin detener el programa. |
| <b>Ubicación</b>        | Ambos bloques deben estar en el mismo lote y en orden.              |
| <b>Usos comunes</b>     | Validaciones, registro de errores, control de transacciones.        |
| <b>Limitaciones</b>     | No captura errores graves (nivel >20) ni advertencias (<10).        |
| <b>Funciones útiles</b> | ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE(), etc.                 |

### Ejemplo completo de aplicación práctica

```
CREATE PROCEDURE dbo.RegistrarEmpleado
    @Nombre NVARCHAR(50),
    @Edad INT
AS
BEGIN TRY
    BEGIN TRANSACTION
        IF @Edad < 18
            THROW 50001, 'El empleado debe ser mayor de edad', 1;

        INSERT INTO Empleados (Nombre, Edad) VALUES (@Nombre, @Edad);
    COMMIT TRANSACTION
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    SELECT
        ERROR_NUMBER() AS NumError,
```

```
ERROR_MESSAGE() AS Mensaje,
ERROR_SEVERITY() AS Gravedad;
```

```
END CATCH;
```

- ◆ THROW lanza un error personalizado.
- ◆ Si algo falla, el bloque CATCH muestra información del error y revierte la transacción.

## ⚙️ Transacciones No Confirmables

### 🌱 1 Qué es una transacción no confirmable

Una **transacción no confirmable** es una transacción que **entró en un estado de error** tal que **no puede ser confirmada (COMMIT)**, aunque intentes hacerlo.

Esto suele pasar cuando dentro de la transacción ocurre un error grave o una violación de integridad (por ejemplo, CHECK, FOREIGN KEY, etc.).

En ese caso, **SQL Server marca la transacción como dañada y solo permite revertirla (ROLLBACK)**.

### ◆ 2 Qué hacer cuando ocurre

En el bloque CATCH, lo correcto es:

- **Llamar siempre a ROLLBACK TRAN** para liberar recursos y deshacer los cambios.
- **Usar funciones como XACT\_STATE()** para determinar si la transacción puede confirmarse o no.

### ◆ 3 XACT\_STATE()

Devuelve el **estado actual de la transacción**.

Los posibles valores son:

| Valor | Significado                                           | Acción recomendada                |
|-------|-------------------------------------------------------|-----------------------------------|
| 1     | Hay una transacción activa y <b>confirmable</b>       | Podés hacer COMMIT                |
| -1    | Hay una transacción activa pero <b>no confirmable</b> | Solo podés hacer ROLLBACK         |
| 0     | No hay transacción activa                             | No se requiere COMMIT ni ROLLBACK |

### 👉 Resumen visual:

XACT\_STATE() = -1 → No confirmable (hacer ROLLBACK)

XACT\_STATE() = 1 → Confirmable (hacer COMMIT)

XACT\_STATE() = 0 → Sin transacción activa

### ■ Ejemplo práctico

```
BEGIN TRY
```

```
  BEGIN TRANSACTION
```

```
  INSERT INTO Clientes VALUES ('Juan', 'Perez');
```

```
  -- Genera error (por ejemplo, columna duplicada)
```

```
INSERT INTO Clientes VALUES ('Juan', 'Perez');  
COMMIT TRANSACTION  
END TRY  
BEGIN CATCH  
    PRINT 'Error detectado';  
    PRINT 'Estado de la transacción: ' + CAST(XACT_STATE() AS NVARCHAR);  
  
    IF XACT_STATE() = -1  
        ROLLBACK TRANSACTION;  
    ELSE IF XACT_STATE() = 1  
        COMMIT TRANSACTION;  
END CATCH;
```

💡 Explicación:

- Si el segundo INSERT falla, XACT\_STATE() será -1.
- Como la transacción no es confirmable, el sistema solo permite **ROLLBACK**.

#### 💠 📌 XACT\_ABORT

XACT\_ABORT es una **opción que controla el comportamiento de las transacciones ante errores**.

SET XACT\_ABORT ON;

🧠 Explicación:

- Si está **ON**, **cualquier error** que ocurra dentro de la transacción **provoca automáticamente un ROLLBACK**.
- Si está **OFF** (por defecto), el error solo cancela la sentencia actual, pero la transacción sigue abierta.

👉 **Recomendación práctica:**

Siempre usar SET XACT\_ABORT ON en procedimientos críticos para garantizar que no queden transacciones abiertas.

#### ✖ Ejemplo completo

```
SET XACT_ABORT ON;  
BEGIN TRY  
    BEGIN TRANSACTION;  
  
    INSERT INTO Productos VALUES ('Mate', 100);  
    -- Error: columna no existe  
    INSERT INTO Productos (Nombre, PrecioInvalido) VALUES ('Termo', 200);  
    COMMIT TRANSACTION;  
END TRY  
BEGIN CATCH  
    PRINT 'Error N°: ' + CAST(ERROR_NUMBER() AS NVARCHAR);
```

```
PRINT 'Mensaje: ' + ERROR_MESSAGE();
PRINT 'Estado Transacción: ' + CAST(XACT_STATE() AS NVARCHAR);
IF XACT_STATE() = -1
    ROLLBACK TRANSACTION;
END CATCH;
```

 Resultado:

- El segundo INSERT genera error.
- XACT\_ABORT ON fuerza el **ROLLBACK automático**.
- XACT\_STATE() = 0 (no hay transacción activa tras el rollback automático).

### Funciones de Error

Estas funciones se usan dentro del bloque CATCH para **obtener información detallada del error**.

| Función           | Descripción                            | Ejemplo                                      |
|-------------------|----------------------------------------|----------------------------------------------|
| ERROR_NUMBER()    | Devuelve el número de error            | Ej: 2627 (clave duplicada)                   |
| ERROR_MESSAGE()   | Mensaje completo del error             | Ej: 'Violation of PRIMARY KEY constraint...' |
| ERROR_SEVERITY()  | Nivel de gravedad (0–25)               | Ej: 16 (error del usuario)                   |
| ERROR_STATE()     | Estado interno del error               | Ej: 1                                        |
| ERROR_LINE()      | Línea donde ocurrió el error           | Ej: 12                                       |
| ERROR_PROCEDURE() | Nombre del procedimiento donde ocurrió | Ej: 'AgregarDatos'                           |

 Estas funciones solo devuelven valores **cuando se usan dentro de un bloque CATCH**.

### Ejemplo didáctico

```
BEGIN TRY
    INSERT INTO Ventas (ID, Total) VALUES (1, 'Texto');
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS NumError,
        ERROR_MESSAGE() AS Mensaje,
        ERROR_SEVERITY() AS Gravedad,
        ERROR_STATE() AS Estado,
        ERROR_LINE() AS Linea,
        ERROR_PROCEDURE() AS Procedimiento;
END CATCH;
```

 Resultado esperado:

| NumError | Mensaje | Gravedad | Estado | Línea | Procedimiento |
|----------|---------|----------|--------|-------|---------------|
|----------|---------|----------|--------|-------|---------------|

|     |                                           |    |   |   |      |
|-----|-------------------------------------------|----|---|---|------|
| 245 | Error de conversión al tipo de datos int. | 16 | 1 | 2 | NULL |
|-----|-------------------------------------------|----|---|---|------|

### Resumen General

| Concepto                          | Descripción                                   | Acción                               |
|-----------------------------------|-----------------------------------------------|--------------------------------------|
| <b>Transacción no confirmable</b> | SQL detectó un error dentro de la transacción | Solo ROLLBACK posible                |
| <b>XACT_STATE()</b>               | Devuelve el estado actual de la transacción   | 1 = confirmable, -1 = no confirmable |
| <b>XACT_ABORT</b>                 | Cancela toda la transacción ante un error     | Recomendado: ON                      |
| <b>ROLLBACK TRAN</b>              | Reversa todos los cambios                     | Obligatorio en errores               |
| <b>Funciones de error</b>         | Proveen detalle del error capturado           | Usar dentro del bloque CATCH         |

## Triggers O Desencadenadores

### Desencadenadores (Triggers)

#### Definición

Un **trigger** (desencadenador) es un tipo especial de **procedimiento almacenado** que se **ejecuta automáticamente** en respuesta a un evento específico en una tabla o vista.

Los eventos que lo activan pueden ser:

- INSERT
- UPDATE
- DELETE

 En otras palabras:

“Un trigger se *dispara* cuando ocurre una acción determinada sobre una tabla o vista.”

#### Consideraciones y limitaciones

| Aspecto                                                      | Detalle                                    |
|--------------------------------------------------------------|--------------------------------------------|
| Activación                                                   | Automática (no se ejecutan con EXEC)       |
| No devuelven valores                                         | No tienen parámetros ni valores de retorno |
| Pueden afectar rendimiento                                   | Se ejecutan cada vez que ocurre el evento  |
| No pueden crear tablas temporales locales dentro del trigger |                                            |
| No deben contener transacciones largas                       |                                            |



|                                                          |  |
|----------------------------------------------------------|--|
| Se usan para auditoría, validación o integridad compleja |  |
|----------------------------------------------------------|--|

### Creación básica

```
CREATE TRIGGER NombreTrigger
ON NombreTabla
[FOR | AFTER | INSTEAD OF] [INSERT, UPDATE, DELETE]
AS
BEGIN
    -- Bloque de sentencias SQL
END
```

#### ◆ Ejemplo simple

```
CREATE TRIGGER trg_AuditoriaEmpleados
ON Empleados
AFTER INSERT
AS
BEGIN
    INSERT INTO AuditoriaEmpleados (EmpleadoID, Fecha, Accion)
    SELECT ID, GETDATE(), 'Nuevo registro'
    FROM inserted;
END
```

### Explicación:

- Cada vez que se inserta un empleado nuevo en la tabla Empleados, el trigger se ejecuta y guarda la acción en la tabla AuditoriaEmpleados.
- inserted es una **tabla lógica temporal** que contiene los nuevos registros.

### Tipos de Triggers

| Tipo          | Momento de ejecución                                            | Uso común                                                                    |
|---------------|-----------------------------------------------------------------|------------------------------------------------------------------------------|
| AFTER (o FOR) | Se ejecuta <b>después</b> de la acción (INSERT, UPDATE, DELETE) | Auditoría o control posterior                                                |
| INSTEAD OF    | Se ejecuta <b>en lugar de</b> la acción                         | Permite redefinir la acción (usado en vistas o restricciones personalizadas) |

#### ◆ Ejemplo AFTER

```
CREATE TRIGGER trg_BorrarEmpleado
ON Empleados
AFTER DELETE
AS
BEGIN
    INSERT INTO AuditoriaEmpleados (EmpleadoID, Fecha, Accion)
    SELECT ID, GETDATE(), 'Empleado borrado'
    FROM deleted;
```

```
SELECT ID, GETDATE(), 'Eliminado'
FROM deleted;
END;
```

■ La tabla lógica deleted contiene los registros **antes** de ser borrados.

#### ◆ Ejemplo INSTEAD OF

```
CREATE TRIGGER trg_EnVezDeDelete
ON Departamentos
INSTEAD OF DELETE
AS
BEGIN
```

```
    PRINT 'No se permite borrar departamentos. Use la columna Activo = 0';
END;
```

✓ Este tipo impide que el DELETE se ejecute realmente y reemplaza su acción.

#### ⚙️ Cómo trabajan internamente

Cuando se ejecuta una sentencia DML (INSERT, UPDATE, DELETE):

1. SQL Server crea dos **tablas lógicas temporales**:
  - inserted: con los registros nuevos.
  - deleted: con los registros eliminados o modificados.
2. El trigger accede a esas tablas para procesar la lógica necesaria (auditoría, validación, etc.).
3. Se ejecuta automáticamente dentro de la misma transacción de la operación.

#### 🔍 Ejemplo con UPDATE

```
CREATE TRIGGER trg_LogCambiosSalario
ON Empleados
AFTER UPDATE
AS
BEGIN
```

```
    INSERT INTO LogCambios (EmpleadoID, SalarioAnterior, SalarioNuevo, Fecha)
    SELECT d.ID, d.Salario, i.Salario, GETDATE()
    FROM deleted d
    INNER JOIN inserted i ON d.ID = i.ID;
```

```
END;
```

💡 Permite registrar **qué valor tenía antes y después** de la actualización.

#### 🌀 Desencadenadores anidados y recursivos

| Tipo           | Descripción                                                | Control                      |
|----------------|------------------------------------------------------------|------------------------------|
| <b>Anidado</b> | Un trigger que ejecuta una acción que dispara otro trigger | Por defecto <b>permitido</b> |

|                  |                                                                                       |                                   |
|------------------|---------------------------------------------------------------------------------------|-----------------------------------|
| <b>Recursivo</b> | Un trigger que se dispara <b>a sí mismo</b> (por una acción dentro del mismo trigger) | Controlado con RECURSIVE_TRIGGERS |
|------------------|---------------------------------------------------------------------------------------|-----------------------------------|

### Ejemplo de configuración

EXEC sp\_configure 'nested triggers', 1; -- Habilitar anidados

ALTER DATABASE MiBase SET RECURSIVE\_TRIGGERS OFF; -- Desactivar recursivos

### Buenas prácticas

- Evitá usar triggers para lógica de negocio compleja (mejor usar procedimientos).
- Documentá bien qué hace cada trigger, ya que se ejecutan automáticamente.
- Mantené triggers simples y específicos (una sola tarea).
- Evitá modificaciones en masa dentro del trigger.
- Usá triggers principalmente para **auditorías, logs y restricciones automáticas**.

### Resumen general

| Concepto              | Descripción                                           |
|-----------------------|-------------------------------------------------------|
| <b>Definición</b>     | Procedimiento automático ante INSERT, UPDATE o DELETE |
| <b>Tipos</b>          | AFTER / FOR y INSTEAD OF                              |
| <b>Tablas lógicas</b> | inserted, deleted                                     |
| <b>Usos comunes</b>   | Auditoría, validación, control de cambios             |
| <b>Limitaciones</b>   | No tienen parámetros, no devuelven valores            |
| <b>Ejecución</b>      | Automática dentro de la misma transacción             |

### Desencadenadores (Triggers): Consideraciones y Limitaciones

#### 1 Consideraciones

Los triggers tienen ciertas características y comportamientos que conviene tener en cuenta al diseñarlos, ya que afectan el rendimiento y la lógica de las transacciones.

| Consideración               | Explicación                                                                                                                                                                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Una sola transacción</b> | El trigger se ejecuta <b>dentro de la misma transacción</b> que la sentencia que lo activó (INSERT, UPDATE, DELETE). Si el trigger falla, <b>toda la operación se revierte</b> (rollback automático).                                              |
| <b>Cascada</b>              | Los triggers pueden <b>disparar otros triggers</b> , generando efectos en cadena. Por ejemplo, un AFTER UPDATE en la tabla Empleados podría activar otro AFTER INSERT en una tabla de auditoría. Esto se conoce como <b>ejecución en cascada</b> . |
| <b>Checks más complejos</b> | Permiten realizar <b>validaciones lógicas</b> más avanzadas que las restricciones simples CHECK. Por ejemplo, verificar una condición basada en múltiples tablas antes de aceptar un cambio.                                                       |

|                                           |                                                                                                                                                                                                                                                                  |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Evaluación del estado de una tabla</b> | Los triggers pueden acceder a las tablas lógicas inserted y deleted para conocer el <b>estado anterior y posterior</b> de los registros. Esto permite detectar qué cambió y actuar en consecuencia.                                                              |
| <b>Varios triggers por acción</b>         | En SQL Server se pueden definir <b>varios triggers</b> para el mismo evento (INSERT, UPDATE, DELETE) sobre una misma tabla. SQL los ejecuta <b>en orden no garantizado</b> , por lo que se recomienda <b>controlar la secuencia manualmente</b> si es necesario. |



### Ejemplo: Validación dentro de una sola transacción

```
CREATE TRIGGER trg_ValidarStock
ON DetalleVenta
AFTER INSERT
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN Productos p ON i.ProductoID = p.ID
        WHERE i.Cantidad > p.Stock
    )
    BEGIN
        RAISERROR('Stock insuficiente', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

#### Explicación:

- Todo se ejecuta en **una sola transacción**.
- Si no hay suficiente stock, el trigger lanza un error (RAISERROR) y **revierte la venta** automáticamente.



### 2 Limitaciones

Aunque los triggers son potentes, tienen restricciones técnicas y de diseño que hay que conocer para evitar errores o comportamientos no deseados.

| Limitación                            | Descripción                                                                                                                                                                            |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Acceso a otras bases de datos</b>  | Un trigger puede leer datos de otras bases, pero esto puede generar <b>dependencias cruzadas</b> o <b>problemas de rendimiento</b> .                                                   |
| <b>Múltiples acciones por trigger</b> | Un mismo trigger puede definirse para <b>más de una acción</b> , por ejemplo: FOR INSERT, UPDATE. Sin embargo, dentro del cuerpo del trigger se debe diferenciar qué operación ocurrió |

|                                                        |                                                                                                                                                                                                                                                        |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                        | (usando IF (COLUMNS_UPDATED()) o verificando inserted/deleted).                                                                                                                                                                                        |
| <b>Restricción con claves foráneas (FK) en cascada</b> | Los triggers INSTEAD OF DELETE o INSTEAD OF UPDATE <b>no pueden usarse en tablas que tengan relaciones en cascada</b> (ON DELETE CASCADE o ON UPDATE CASCADE), porque SQL Server no permite anular esas operaciones automáticas.                       |
| <b>Instrucciones SET</b>                               | Es posible usar comandos como SET NOCOUNT ON, SET XACT_ABORT ON, etc., dentro del trigger, pero deben usarse con cuidado porque afectan el comportamiento transaccional.                                                                               |
| <b>Uso de SELECT</b>                                   | Dentro de un trigger se pueden hacer consultas SELECT, pero si devuelven resultados al cliente pueden provocar errores o afectar el rendimiento. Por eso, se recomienda <b>no devolver resultados</b> , sino guardar los datos en tablas de auditoría. |
| <b>No se ejecutan con TRUNCATE TABLE</b>               | Los triggers <b>no se activan</b> cuando se usa TRUNCATE TABLE, ya que este comando <b>no genera eventos DML</b> (DELETE) sino que borra directamente las páginas de datos. Si querés auditar borrados masivos, usá DELETE FROM en su lugar.           |

### Ejemplo de trigger para múltiples acciones

```
CREATE TRIGGER trg_LogCambios
ON Productos
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    IF EXISTS (SELECT * FROM inserted)
        PRINT 'Hubo inserciones o actualizaciones';
    IF EXISTS (SELECT * FROM deleted)
        PRINT 'Hubo eliminaciones o modificaciones';
END;
```

 Este trigger se ejecuta ante **cualquiera de las tres acciones**, diferenciando el evento según el uso de las tablas lógicas inserted y deleted.

### Ejemplo de limitación con TRUNCATE

```
CREATE TRIGGER trg_BorradoMasivo
ON Ventas
AFTER DELETE
AS
BEGIN
    INSERT INTO AuditoriaBorrado VALUES (GETDATE(), USER_NAME());
END;
```

📌 Si ejecutás:

DELETE FROM Ventas; -- ✅ dispara el trigger

TRUNCATE TABLE Ventas; -- ❌ NO dispara el trigger

SQL Server no permite interceptar el TRUNCATE, por eso **los triggers no lo detectan.**

### 📄 Resumen comparativo

| Categoría              | Punto clave                                  | Ejemplo / Comentario                       |
|------------------------|----------------------------------------------|--------------------------------------------|
| <b>Consideraciones</b> | Se ejecutan en la misma transacción          | Si falla el trigger, se revierte la acción |
|                        | Pueden ser en cascada                        | Un trigger que llama a otro                |
|                        | Pueden contener validaciones complejas       | Control de stock, precios, etc.            |
|                        | Pueden coexistir varios triggers             | Misma tabla, misma acción                  |
| <b>Limitaciones</b>    | No se ejecutan con TRUNCATE                  | Solo con DELETE                            |
|                        | No pueden usarse en tablas con FK en cascada | Incompatibles con INSTEAD OF               |
|                        | Pueden acceder a otras bases (riesgo)        | Genera dependencias                        |
|                        | Evitar SELECTs que devuelvan datos           | Afecta rendimiento                         |
|                        | Pueden definirse para más de una acción      | AFTER INSERT, UPDATE                       |

### ⚙️ Creación, Modificación y Borrado de Triggers

#### 🔧 1 Creación de un Trigger

##### 📘 Sintaxis general

CREATE TRIGGER [nombre\_esquema.]nombre\_trigger

ON { tabla | vista }

[ WITH <opciones\_trigger\_dml> [ ,...n ] ]

{ FOR | AFTER | INSTEAD OF }

{ [ INSERT ] [,] [ UPDATE ] [,] [ DELETE ] }

[ WITH APPEND ]

[ NOT FOR REPLICATION ]

AS

{ sentencias\_sql [ ; ] [ ...n ] | EXTERNAL NAME <method\_specifier> [ ; ] }

#### 🔍 Explicación de cada parte

| Elemento          | Descripción                                       |
|-------------------|---------------------------------------------------|
| CREATE TRIGGER    | Indica que se va a crear un nuevo desencadenador. |
| [nombre_esquema.] | Es opcional. Indica el esquema (por defecto dbo). |
| ` ON {tabla       | vista}`                                           |

|                              |                                                                            |
|------------------------------|----------------------------------------------------------------------------|
| FOR / AFTER                  | Se ejecuta <b>después</b> del evento (INSERT, UPDATE, DELETE).             |
| INSTEAD OF                   | Se ejecuta <b>en lugar de</b> la acción (usado sobre todo en vistas).      |
| [INSERT], [UPDATE], [DELETE] | Indican qué eventos activan el trigger.                                    |
| WITH APPEND                  | Permite agregar este trigger <b>sin reemplazar</b> otros existentes.       |
| NOT FOR REPLICATION          | Evita que el trigger se ejecute durante procesos de <b>replicación</b> .   |
| AS                           | Indica el comienzo del código SQL del trigger.                             |
| EXTERNAL NAME                | Se usa para triggers definidos fuera de SQL (en .NET u otro lenguaje CLR). |

### Ejemplo práctico: auditoría simple

```
CREATE TRIGGER trg_AuditoriaProductos
ON Productos
AFTER INSERT, UPDATE
AS
BEGIN
    INSERT INTO AuditoriaProductos (ProductoID, Fecha, Accion)
    SELECT i.ID, GETDATE(),
        CASE
            WHEN d.ID IS NULL THEN 'INSERT'
            ELSE 'UPDATE'
        END
    FROM inserted i
    LEFT JOIN deleted d ON i.ID = d.ID;
END;
```

### Explicación:

- Se dispara automáticamente después de cada **INSERT o UPDATE** sobre Productos.
- Usa las tablas lógicas:
  - inserted → nuevos registros.
  - deleted → registros previos a la modificación.
- Inserta en la tabla AuditoriaProductos un registro indicando qué acción ocurrió.

### Ejemplo con INSTEAD OF

```
CREATE TRIGGER trg_EvitarBorradoClientes
ON Clientes
INSTEAD OF DELETE
```



```
AS
BEGIN
    PRINT 'No se permite eliminar clientes. Marcar como inactivo.';
END;
```

➡ Este reemplaza el DELETE y **bloquea el borrado real**.

## 🔧 2 Modificación de un Trigger

### 📘 Sintaxis

```
ALTER TRIGGER nombre_trigger
ON { tabla | vista }
[ WITH <opciones_trigger_dml> [ ,...n ] ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [,] [ UPDATE ] [,] [ DELETE ] }
AS
```

```
    sentencias_sql;
```

📘 Sirve para **editar el código o el comportamiento** de un trigger ya existente sin borrarlo.

### 🧠 Ejemplo de modificación

```
ALTER TRIGGER trg_AuditoriaProductos
ON Productos
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    INSERT INTO AuditoriaProductos (ProductoID, Fecha, Accion)
    SELECT COALESCE(i.ID, d.ID), GETDATE(),
        CASE
            WHEN i.ID IS NOT NULL AND d.ID IS NULL THEN 'INSERT'
            WHEN i.ID IS NOT NULL AND d.ID IS NOT NULL THEN 'UPDATE'
            WHEN i.ID IS NULL AND d.ID IS NOT NULL THEN 'DELETE'
        END
    FROM inserted i
    FULL JOIN deleted d ON i.ID = d.ID;
END;
```

💡 Ahora también audita eliminaciones (DELETE).

## 🔧 3 Borrado de un Trigger

### 📘 Sintaxis

```
DROP TRIGGER [ IF EXISTS ] nombre_trigger [ ,...n ];
```

📘 Elimina el trigger completamente del sistema.

### 🧠 Ejemplo

DROP TRIGGER IF EXISTS trg\_AuditoriaProductos;

💡 Usar IF EXISTS evita errores si el trigger no existía.

### 📄 Resumen general

| Acción          | Comando                          | Descripción                                       |
|-----------------|----------------------------------|---------------------------------------------------|
| Crear           | CREATE TRIGGER                   | Define un nuevo trigger en una tabla o vista.     |
| Modificar       | ALTER TRIGGER                    | Cambia el comportamiento o la lógica del trigger. |
| Eliminar        | DROP TRIGGER                     | Elimina el trigger del sistema.                   |
| Tipos           | AFTER / FOR / INSTEAD OF         | Define el momento de ejecución.                   |
| Eventos         | INSERT, UPDATE, DELETE           | Acciones que lo activan.                          |
| Opciones útiles | WITH APPEND, NOT FOR REPLICATION | Configuraciones avanzadas.                        |

### ⚙️ Tipos de Desencadenadores (Triggers)

En SQL Server existen **dos tipos principales de triggers**, que se diferencian por **cuándo** se ejecutan en relación con la operación que los dispara (INSERT, UPDATE o DELETE).

#### ♦️ **❑FOR / AFTER Triggers**

#### 📘 Definición:

Se ejecutan **después** de que la operación que los activó se haya completado exitosamente.

Esto significa que la acción (INSERT, UPDATE o DELETE) **ya se realizó** y los datos fueron modificados.

📌 Se pueden usar en:

- **Tablas**
- (No en vistas)

#### 🧠 Ejemplo práctico: Trigger AFTER

```
CREATE TRIGGER trg_AuditoriaVentas
ON Ventas
AFTER INSERT
AS
BEGIN
    INSERT INTO Auditoria (VentaID, Fecha, Accion)
    SELECT ID, GETDATE(), 'INSERT'
    FROM inserted;
END;
```

#### 🔗 Explicación:

- Se ejecuta después de que se inserta un registro en Ventas.

- Usa la tabla lógica inserted (que contiene los datos nuevos).
- Inserta un registro de auditoría automáticamente.

#### Caso típico de uso:

Registrar, auditar o realizar cálculos secundarios **después de una operación DML**.

#### ♦ INSTEAD OF Triggers

#### Definición:

Se ejecutan **en lugar de** la operación que los disparó.

Es decir, **reemplazan la acción original** (el INSERT, UPDATE o DELETE).

• Se pueden usar en:

- **Tablas**
- **Vistas** (donde a veces no se puede realizar un INSERT o DELETE directo)

#### Ejemplo práctico: Trigger INSTEAD OF

```
CREATE TRIGGER trg_BorrarCliente
ON Clientes
INSTEAD OF DELETE
AS
BEGIN
    UPDATE Clientes
    SET Activo = 0
    WHERE ID IN (SELECT ID FROM deleted);
END;
```

#### Explicación:

- En lugar de borrar físicamente un cliente, el trigger **marca el campo Activo como 0**.
- La operación DELETE original **no se ejecuta**.
- Usa la tabla lógica deleted, que contiene los registros que se iban a eliminar.

#### Caso típico de uso:

Evitar eliminaciones o modificaciones directas en tablas o vistas importantes, aplicando una **lógica personalizada**.

#### Resumen comparativo

| Tipo               | Momento de ejecución     | Permite en vistas                                                                      | Uso típico                                               |
|--------------------|--------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>FOR / AFTER</b> | Después de la operación  |  No | Auditoría, cálculos, validaciones posteriores            |
| <b>INSTEAD OF</b>  | En lugar de la operación |  Sí | Validaciones, control de lógica, restricciones en vistas |

#### Nota importante:

Un mismo trigger puede responder a **varios eventos** (por ejemplo: AFTER INSERT,

UPDATE)

y dentro del código se puede usar IF EXISTS (SELECT \* FROM inserted) o IF EXISTS (SELECT \* FROM deleted)

para determinar qué tipo de cambio se produjo.

## Desencadenadores (Triggers) por tipo de operación

### 1 Trigger de INSERT

#### Concepto

Se activa automáticamente **cuando se inserta un registro** en una tabla.

 SQL Server **crea una tabla temporal lógica llamada inserted**, que contiene **los registros nuevos** que se acaban de insertar.

#### Ejemplo

```
CREATE TRIGGER insrtWorkOrder
ON Production.WorkOrder
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO Production.TransactionHistory
        (ProductID, ReferenceOrderID, TransactionType,
        TransactionDate, Quantity, ActualCost)
    SELECT inserted.ProductID, inserted.WorkOrderID,
        'W', GETDATE(), inserted.OrderQty, 0
    FROM inserted;
END;
```

#### Explicación paso a paso:

1. Se ejecuta **después** de un INSERT en la tabla WorkOrder.
2. Toma los nuevos registros de la tabla lógica inserted.
3. Los usa para **insertar automáticamente** una fila en TransactionHistory.
4. Agrega información de auditoría o control.

 *Uso típico:* registrar operaciones, auditorías, actualizaciones automáticas de stock o logs.

### 2 Trigger de DELETE

#### Concepto

Se activa **cuando se eliminan registros** de una tabla.

 SQL Server crea una tabla temporal lógica llamada **deleted**, que contiene **los registros que fueron borrados**.

#### Ejemplo

```
CREATE TRIGGER delCustomer
```

```

ON Sales.Customer
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    EXEC master..xp_sendmail
        @recipients = N'SalesManagers@Adventure-Works.com',
        @message = N'Customers have been deleted!!';
END;

```

### Explicación:

1. El trigger se ejecuta **luego de un borrado (DELETE)**.
2. Puede acceder a la tabla deleted para conocer qué registros se eliminaron.
3. En este caso, envía un correo (usando xp\_sendmail) notificando que se borraron clientes.

### Uso típico:

Registrar o notificar eliminaciones, mantener logs de auditoría, o mover los datos eliminados a una tabla histórica.

## 3 Trigger de UPDATE

### Concepto

Se activa **cuando se actualiza** un registro de una tabla.

-  SQL Server genera dos tablas lógicas:
  - **deleted** → contiene los datos **antes** del cambio.
  - **inserted** → contiene los datos **después** del cambio.

### Propiedades importantes

- Permite comparar los valores **viejos** y **nuevos** de un campo.
- Se puede usar la función **UPDATE(<columna>)** para saber si una columna específica fue modificada.

### Ejemplo

```

CREATE TRIGGER uWorkOrder
ON Production.WorkOrder
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF UPDATE(ProductID) OR UPDATE(OrderQty)
    BEGIN
        INSERT INTO Production.TransactionHistory

```

```
(ProductID, ReferenceOrderID, TransactionType,
TransactionDate, Quantity)
SELECT inserted.ProductID, inserted.WorkOrderID,
'W', GETDATE(), inserted.OrderQty
FROM inserted;
```

```
UPDATE Production.WorkOrder
SET ModifiedDate = GETDATE()
FROM inserted
WHERE inserted.WorkOrderID = Production.WorkOrder.WorkOrderID;
END;
END;
```

### Explicación paso a paso:

1. El trigger se ejecuta **después de un UPDATE**.
2. Usa IF UPDATE(ProductID) para detectar si se modificó esa columna.
3. Inserta en TransactionHistory un registro de la modificación.
4. Finalmente, actualiza el campo ModifiedDate para reflejar la fecha del cambio.

 **Uso típico:** auditorías, mantener históricos, recalcular totales, sincronizar datos o actualizar fechas de modificación.

### Resumen general

| Tipo de Trigger | Tabla lógica creada | Momento de ejecución | Datos disponibles          | Uso común                            |
|-----------------|---------------------|----------------------|----------------------------|--------------------------------------|
| <b>INSERT</b>   | inserted            | Después del INSERT   | Nuevos datos               | Auditoría de altas                   |
| <b>DELETE</b>   | deleted             | Después del DELETE   | Datos eliminados           | Registro de bajas o notificaciones   |
| <b>UPDATE</b>   | deleted y inserted  | Después del UPDATE   | Antes y después del cambio | Auditoría, sincronización, recalculo |

### Tips para recordar

- inserted = datos **nuevos**
- deleted = datos **anteriores**
- En UPDATE, existen **ambas** tablas.
- Usar SET NOCOUNT ON para evitar resultados intermedios.
- IF UPDATE(columna) es ideal para detectar modificaciones específicas.
- Los triggers se ejecutan **por lote** (no por fila), así que se debe usar INSERTED y DELETED con **consultas basadas en conjuntos**, no con variables escalares.

### Desencadenadores Anidados y Recursivos

## 1 Desencadenadores Anidados (Nested Triggers)

### Definición

Un **trigger anidado** ocurre cuando un trigger ejecuta otra acción DML (INSERT, UPDATE o DELETE) que **dispara otro trigger**.

SQL Server puede manejar hasta **32 niveles de anidamiento**.

 Por defecto, **los triggers anidados están habilitados** en SQL Server, pero se pueden activar o desactivar.

### Configuración y control

- **Habilitar o deshabilitar triggers anidados globalmente:**

EXEC sp\_configure 'nested triggers', 1; -- 1 = habilitado, 0 = deshabilitado  
RECONFIGURE;

- **Saber el nivel actual de anidamiento:**

SELECT @@NESTLEVEL AS NivelActual;

 Devuelve cuántos niveles de triggers se han activado en una cadena de ejecución.

### Ejemplo

Supongamos que:

- El trigger A hace un INSERT en otra tabla.
- Esa tabla tiene su propio trigger B.

 Entonces, **el trigger B se ejecuta automáticamente después de A** → eso es *anidamiento*.

-- Trigger A

```
CREATE TRIGGER trg_A
ON Tabla1
AFTER INSERT
AS
BEGIN
    INSERT INTO Tabla2 (columna)
    SELECT columna FROM inserted;
END;
GO
```

-- Trigger B

```
CREATE TRIGGER trg_B
ON Tabla2
AFTER INSERT
AS
BEGIN
    PRINT 'Se ejecutó el Trigger B después del A';
END;
```



💡 **SQL Server permite hasta 32 triggers encadenados**, pero normalmente se recomienda evitar más de 2–3 niveles por mantenimiento.

## 🔄 2 Desencadenadores Recursivos

### 📖 Definición

Un **trigger recursivo** es aquel que **se ejecuta a sí mismo directa o indirectamente**. Hay dos tipos:

| Tipo                        | Descripción                                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Repetición Directa</b>   | El trigger ejecuta una acción (por ejemplo, UPDATE) que <b>dispara nuevamente el mismo trigger</b> sobre la misma tabla.  |
| <b>Repetición Indirecta</b> | Un trigger A ejecuta una acción que <b>activa otro trigger B</b> , y B ejecuta una acción que <b>vuelve a activar A</b> . |

### 🧠 Configuración

Para permitir o impedir recursividad, se usa la opción de base de datos:

```
ALTER DATABASE NombreBD
```

```
SET RECURSIVE_TRIGGERS ON; -- Permite recursividad
```

o

```
ALTER DATABASE NombreBD
```

```
SET RECURSIVE_TRIGGERS OFF; -- Bloquea recursividad (por defecto)
```

### 🔍 Ejemplo de recursividad directa

```
CREATE TRIGGER trg_UpdateStock
```

```
ON Productos
```

```
AFTER UPDATE
```

```
AS
```

```
BEGIN
```

```
    UPDATE Productos
```

```
    SET Stock = Stock - 1
```

```
    WHERE ID IN (SELECT ID FROM inserted);
```

```
END;
```

⚠️ Si RECURSIVE\_TRIGGERS está **activado**, este trigger se ejecutará **infinitamente**, porque el UPDATE dentro del trigger vuelve a disparar el mismo trigger.

📖 Por eso, normalmente **se deja deshabilitada** la recursividad, o se controla con condiciones:

```
IF TRIGGER_NESTLEVEL() = 1
```

```
-- ejecutar lógica
```

### 📄 Resumen general

| Tipo | Característica | Límite / Control | Palabra clave / Configuración |
|------|----------------|------------------|-------------------------------|
|------|----------------|------------------|-------------------------------|

|                  |                                                             |                                  |                                             |
|------------------|-------------------------------------------------------------|----------------------------------|---------------------------------------------|
| <b>Anidado</b>   | Un trigger dispara otro                                     | Hasta 32 niveles                 | sp_configure 'nested triggers', @@NESTLEVEL |
| <b>Recursivo</b> | Un trigger se dispara a sí mismo (directa o indirectamente) | Controlado a nivel base de datos | RECURSIVE_TRIGGERS                          |

 **Tips para el examen**

-  SQL Server **permite triggers anidados** hasta **32 niveles**.
-  Se controlan con sp\_configure 'nested triggers'.
-  Los **recursivos** pueden ser peligrosos (riesgo de bucles infinitos).
-  RECURSIVE\_TRIGGERS controla si se permiten.
-  @@NESTLEVEL devuelve el nivel de anidamiento actual.